St. Wilfred's College of Computer Sciences

Affiliated to University of Mumbai, DTE Code :

3539 Near MBMC Garden, Sanghvi Nagar, Mira Road(East),Thane-401107, Maharashtra

Name:Shubham Vaity

Class:FY-MCA

Subject: ADVANCED DATABASE MANAGEMENT SYSTEM LAB

Roll No: MCA2014203

Year: 2025-2026

ST. WILFRED'S COLLEGE OF COMPUTER SCIENCES

Affiliated to Mumbai University, Approved by AICTE- New Delhi,

Near MBMC garden, Sanghvi Nagar, Mira Bhayandar Road, Mira Road, Thane – 401107

CERTIFICATE

DATE:

This is to certify that Mr./~~Ms~~**. Shubahm Hemchandra Vaity**
Roll No.**MCA2014203** is a student of FYMCA Semester-I has completed successfully full-semester practical/assignments of subject Advanced Database Management System Lab for the academic year 2025 – 26.

CO	R1 (JOURNAL)	R2 (PERFORMANCE DURING LAB SESSION)	R3 (IMPLEMENTATION USING DIFFERENT PROBLEM SOLVING TECHNIQUES)	R4 (MOCK VIVA)	ATTENDANCE
CO1					
CO2					
CO3					
CO4					

Subject In-Charge

Director
Dr. Shubhi Lall Agarwal

External Examiner

MCAL13 ADVANCED DATABASE MANAGEMENT SYSTEM LAB
INDEX

NO	PROBLEM STATEMENT	DATE	CO	SIGNATURE
1.	Implementation of Data partitioning through Hash, Range and List partitioning	**18-10-25**	**CO1**	
2.	Implementation of Analytical queries like Roll_UP, CUBE, First, Last, Lead, Lag, Rank AND Dense Rank and Windowing functions- preceding rows, following and rows between	**25-10-25**	**CO3**	
3.	Implementation of ORDBMS concepts like ADT (Abstract Data Types), Object table and Inheritance	**08-11-25**	**CO1**	

4.	**ETL Transformation with Pentaho** 1. Copy data from Source & store to target 2. Adding Sequence 3. Adding Calculator 4. Concatenation of Two Fields 5. Splitting of Two Fields 6. Number Range 7. String Operations 8. Sorting Data 9. Implement the Merge Join 10. Implement data validations on table data 11. Replace Strings 12. Splitting Fields to Rows	**08-11-25,** **13-11-25**	**CO2**	
5.	**Introduction to R,** Install packages, Loading packages Data types, checking variable type, printing variable and	**13-11-25**	**CO4**	

	objects (Vector, Matrix, List, Factor, Data frame, Table) c-binding and rbinding **Reading and Writing data** Setw( ), getw( ), data( ), rm( ) **Attaching and Detaching data** Reading data from the console Loading data from d ifferent data sources (CSV, Excel)	**14-11-25**		
6.	Data preprocessing techniques in R Naming and Renaming variables Adding a new variables Dealing with missing value Dealing with categorical data Data reduction using subsetting	**22-11-25,** **23-11-25**	**CO4**	
7.	Implementation and analysis of Linear regression.	**04-12-25**	**CO4**	

8.	Implementation and Analysis Classification algorithms -Naïve Bayesian	**10-12-25**	**CO5**	
9.	Implementation and Analysis Classification algorithms - K-Nearest Neighbour	**10-12-25**	**CO5**	
10.	Implementation and Analysis Classification algorithms - ID3, C4.5	**04-12-25**	**CO5**	
11.	Implementation and analysis of Apriori Algorithm using Market Basket Analysis	**11-12-25**	**CO5**	
12.	Implementation and analysis of clustering algorithms - K-Means	**12-12-25**	**CO5**	
13.	Implementation and analysis of clustering algorithms - Agglomerative	**12-12-25**	**CO5**	

PRACTICAL NO 1

Aim: Implementation of Data partitioning through Range , List And Hash partitioning

RANGE PARTITIONING

```
create table Accounts
(
acctno  number,
custname varchar(23),
branch varchar(23),
acctbal number
)partition by range (acctbal)
(
partition p1 values less than (2000),
partition p2 values less than (5000),
partition p3 values less than (9999)
);
select * from Accounts;
```

OUTPUT:

```
SQL> desc Accounts;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 ACCTNO                                             NUMBER
 CUSTNAME                                           VARCHAR2(23)
 BRANCH                                             VARCHAR2(23)
 ACCTBAL                                            NUMBER

SQL>  insert into Accounts values(4,'Sagar','Bhuvneshwar',5000);

1 row created.

SQL>
SQL>
SQL>
SQL>   insert into Accounts values(2,'Arman','Fun Fiesta',9998);

1 row created.

SQL>
SQL>
SQL>  insert into Accounts values(3,'Shashikala','Airport',7000);

1 row created.

SQL>
SQL>
SQL>   insert into Accounts values(1,'Siddharth','Thane',6000);

1 row created.

SQL>
SQL>
SQL>
SQL>   insert into Accounts values(5,'Pratik','Kalyan',2000);

1 row created.

SQL>
SQL>
SQL>
SQL>  insert into Accounts values(6,'Sarvesh','Airport',1000);

1 row created.

SQL>
SQL> select * from Accounts;

    ACCTNO CUSTNAME                BRANCH                       ACCTBAL
---------- ----------------------- ----------------------- ----------
         6 Sarvesh                 Airport                        1000
         5 Pratik                  Kalyan                         2000
         4 Sagar                   Bhuvneshwar                    5000
         2 Arman                   Fun Fiesta                     9998
         3 Shashikala              Airport                        7000
         1 Siddharth               Thane                          6000

6 rows selected.
```

select * from Accounts partition (p1);
select * from Accounts partition (p2);
select * from Accounts partition (p3);

OUTPUT:

```
SQL> select * from Accounts partition(p1);

    ACCTNO CUSTNAME                BRANCH                       ACCTBAL
---------- ----------------------- ----------------------- ----------
         6 Sarvesh                 Airport                        1000

SQL>
SQL>  select * from Accounts partition(p2);

    ACCTNO CUSTNAME                BRANCH                       ACCTBAL
---------- ----------------------- ----------------------- ----------
         5 Pratik                  Kalyan                         2000

SQL>
SQL>  select * from Accounts partition(p3);

    ACCTNO CUSTNAME                BRANCH                       ACCTBAL
---------- ----------------------- ----------------------- ----------
         4 Sagar                   Bhuvneshwar                    5000
         2 Arman                   Fun Fiesta                     9998
         3 Shashikala              Airport                        7000
         1 Siddharth               Thane                          6000

---.
```

alter table Accounts merge Partitions p1,p2 into partition p6;
select * from Accounts partition(p6);
OUTPUT:

```
SQL>  alter table Accounts merge partitions p1,p2 into partition p6;

Table altered.

SQL> select * from Accounts partition(p6);

    ACCTNO CUSTNAME                BRANCH                       ACCTBAL
---------- ----------------------- ----------------------- ----------
         6 Sarvesh                 Airport                        1000
         5 Pratik                  Kalyan                         2000
```

```
SQL> alter table Accounts add partition p4 values less than (10000);

Table altered.

SQL> alter table Accounts add partition p5 values less than (12000);

Table altered.

SQL> insert into Accounts values(7,'Ziya','Mahim',11000);

1 row created.

SQL> insert into Accounts values(8,'Sairaj','Airport',10500);

1 row created.

SQL> select * from Accounts partition(p5);

    ACCTNO CUSTNAME                BRANCH                       ACCTBAL
---------- ----------------------- ----------------------- ----------
         7 Ziya                    Mahim                         11000
         8 Sairaj                  Airport                       10500

SQL> alter table Accounts drop partition p6 ;

Table altered.
```

select * from user_tab_partitions where table_name ='ACCOUNTS';

OUTPUT:

```
SQL> select * from user_tab_partitions where table_name ='ACCOUNTS';

TABLE_NAME                     COM PARTITION_NAME
------------------------------ --- ------------------------------
SUBPARTITION_COUNT
------------------
HIGH_VALUE
--------------------------------------------------------------------------------
HIGH_VALUE_LENGTH PARTITION_POSITION TABLESPACE_NAME                    PCT_FREE
----------------- ------------------ ------------------------------ ----------
  PCT_USED  INI_TRANS  MAX_TRANS INITIAL_EXTENT NEXT_EXTENT MIN_EXTENT
---------- ---------- ---------- -------------- ----------- ----------
MAX_EXTENT PCT_INCREASE  FREELISTS FREELIST_GROUPS LOGGING COMPRESS   NUM_ROWS
---------- ------------ ---------- --------------- ------- -------- ----------
    BLOCKS EMPTY_BLOCKS  AVG_SPACE  CHAIN_CNT AVG_ROW_LEN SAMPLE_SIZE LAST_ANAL
---------- ------------ ---------- ---------- ----------- ----------- ---------
BUFFER_ GLO USE
------- --- ---
ACCOUNTS                       NO  P3

TABLE_NAME                     COM PARTITION_NAME
------------------------------ --- ------------------------------
SUBPARTITION_COUNT
------------------
HIGH_VALUE
--------------------------------------------------------------------------------
HIGH_VALUE_LENGTH PARTITION_POSITION TABLESPACE_NAME                    PCT_FREE
----------------- ------------------ ------------------------------ ----------
  PCT_USED  INI_TRANS  MAX_TRANS INITIAL_EXTENT NEXT_EXTENT MIN_EXTENT
---------- ---------- ---------- -------------- ----------- ----------
MAX_EXTENT PCT_INCREASE  FREELISTS FREELIST_GROUPS LOGGING COMPRESS   NUM_ROWS
---------- ------------ ---------- --------------- ------- -------- ----------
    BLOCKS EMPTY_BLOCKS  AVG_SPACE  CHAIN_CNT AVG_ROW_LEN SAMPLE_SIZE LAST_ANAL
---------- ------------ ---------- ---------- ----------- ----------- ---------
BUFFER_ GLO USE
------- --- ---
              0

TABLE_NAME                     COM PARTITION_NAME
------------------------------ --- ------------------------------
SUBPARTITION_COUNT
------------------
HIGH_VALUE
--------------------------------------------------------------------------------
HIGH_VALUE_LENGTH PARTITION_POSITION TABLESPACE_NAME                    PCT_FREE
----------------- ------------------ ------------------------------ ----------
  PCT_USED  INI_TRANS  MAX_TRANS INITIAL_EXTENT NEXT_EXTENT MIN_EXTENT
---------- ---------- ---------- -------------- ----------- ----------
MAX_EXTENT PCT_INCREASE  FREELISTS FREELIST_GROUPS LOGGING COMPRESS   NUM_ROWS
---------- ------------ ---------- --------------- ------- -------- ----------
    BLOCKS EMPTY_BLOCKS  AVG_SPACE  CHAIN_CNT AVG_ROW_LEN SAMPLE_SIZE LAST_ANAL
---------- ------------ ---------- ---------- ----------- ----------- ---------
BUFFER_ GLO USE
------- --- ---
9999

TABLE_NAME                     COM PARTITION_NAME
------------------------------ --- ------------------------------
SUBPARTITION_COUNT
------------------
HIGH_VALUE
--------------------------------------------------------------------------------
HIGH_VALUE_LENGTH PARTITION_POSITION TABLESPACE_NAME                    PCT_FREE
----------------- ------------------ ------------------------------ ----------
```

alter table Accounts drop partition p3;

OUTPUT:

```
SQL> alter table Accounts drop partition p3;

Table altered.
```

update Accounts set acctbal = acctbal+(acctbal*7/100) where acctbal<=100

OUTPUT:

```
SQL> update Accounts set acctbal = acctbal+(acctbal*7/100) where acctbal<=100;

0 rows updated.

SQL> select * from Accounts;

    ACCTNO CUSTNAME                BRANCH                       ACCTBAL
---------- ----------------------- ----------------------- ----------
         7 Ziya                    Mahim                        11000
         8 Sairaj                  Airport                      10500
```

LIST PARTITIONING

```
create table Product
(
prdno varchar(10),
prodname varchar(23),
rate number,
qty_available number
)
partition by list(prdno)
(
partition pd1 values('p001','p002','p003','p004'),
partition pd2 values('b001','b002','b003','b004'),
partition pd3 values('c001','c002','c003','c004'),
partition pd4 values('d001','d002','d003','d004')
);
```

OUTPUT:

```
SQL> create table Product
  2  (
  3  prdno varchar(10),
  4  prodname varchar(23),
  5  rate number,
  6  qty_available number
  7  )
  8  partition by list(prdno)
  9  (
 10  partition pd1 values('p001','p002','p003','p004'),
 11  partition pd2 values('b001','b002','b003','b004'),
 12  partition pd3 values('c001','c002','c003','c004'),
 13  partition pd4 values('d001','d002','d003','d004')
 14  );

Table created.

SQL> desc Product;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 PRDNO                                              VARCHAR2(10)
 PRODNAME                                           VARCHAR2(23)
 RATE                                               NUMBER
 QTY_AVAILABLE                                      NUMBER

SQL> INSERT INTO Product (prdno, prodname, rate, qty_available) VALUES ('p001', 'Wireless Mouse', 10
.50, 100);

1 row created.

SQL> INSERT INTO Product (prdno, prodname, rate, qty_available) VALUES ('p002', 'Mechanical Keyboard
', 15.00, 200);

1 row created.

SQL> INSERT INTO Product (prdno, prodname, rate, qty_available) VALUES ('p003', 'Gaming Headset', 20
.00, 150);

1 row created.

SQL> INSERT INTO Product (prdno, prodname, rate, qty_available) VALUES ('p004', 'USB-C Hub', 25.00,
120);

1 row created.

SQL>
SQL> INSERT INTO Product (prdno, prodname, rate, qty_available) VALUES ('b001', 'Bluetooth Speaker',
 30.00, 90);

1 row created.

SQL> INSERT INTO Product (prdno, prodname, rate, qty_available) VALUES ('b002', 'Smartphone Charger'
, 35.00, 80);

1 row created.

SQL> INSERT INTO Product (prdno, prodname, rate, qty_available) VALUES ('b003', 'Portable SSD', 40.0
0, 70);

1 row created.

SQL> INSERT INTO Product (prdno, prodname, rate, qty_available) VALUES ('b004', 'Tablet Stand', 45.0
0, 60);
```

```
SQL>
SQL> INSERT INTO Product (prdno, prodname, rate, qty_available) VALUES ('c001', 'Laptop Bag', 50.00,
 50);

1 row created.

SQL> INSERT INTO Product (prdno, prodname, rate, qty_available) VALUES ('c002', 'Monitor Stand', 55.
00, 40);

1 row created.

SQL> INSERT INTO Product (prdno, prodname, rate, qty_available) VALUES ('c003', 'Webcam', 60.00, 30)
;

1 row created.

SQL> INSERT INTO Product (prdno, prodname, rate, qty_available) VALUES ('c004', 'Wireless Charger',
65.00, 20);

1 row created.

SQL>
SQL> INSERT INTO Product (prdno, prodname, rate, qty_available) VALUES ('d001', 'Graphics Card', 70.
00, 10);

1 row created.

SQL> INSERT INTO Product (prdno, prodname, rate, qty_available) VALUES ('d002', 'External Monitor',
75.00, 5);

1 row created.

SQL> INSERT INTO Product (prdno, prodname, rate, qty_available) VALUES ('d003', 'Gaming Console', 80
.00, 8);

1 row created.

SQL> INSERT INTO Product (prdno, prodname, rate, qty_available) VALUES ('d004', 'VR Headset', 85.00,
 3);

1 row created.
```

```
SQL> select * from Product;

PRDNO      PRODNAME                       RATE QTY_AVAILABLE
---------- ---------------------- ---------- -------------
p001       Wireless Mouse               10.5           100
p002       Mechanical Keyboard            15           200
p003       Gaming Headset                 20           150
p004       USB-C Hub                      25           120
b001       Bluetooth Speaker              30            90
b002       Smartphone Charger             35            80
b003       Portable SSD                   40            70
b004       Tablet Stand                   45            60
c001       Laptop Bag                     50            50
c002       Monitor Stand                  55            40
c003       Webcam                         60            30

PRDNO      PRODNAME                       RATE QTY_AVAILABLE
---------- ---------------------- ---------- -------------
c004       Wireless Charger               65            20
d001       Graphics Card                  70            10
d002       External Monitor               75             5
d003       Gaming Console                 80             8
d004       VR Headset                     85             3

16 rows selected.
```

select * from user_tab_partitions where table_name ='PRODUCT';

OUTPUT:

```
SQL> select * from user_tab_partitions where table_name ='PRODUCT';

TABLE_NAME                     COM PARTITION_NAME
------------------------------ --- ------------------------------
SUBPARTITION_COUNT
------------------
HIGH_VALUE
--------------------------------------------------------------------------------
HIGH_VALUE_LENGTH PARTITION_POSITION TABLESPACE_NAME                  PCT_FREE
----------------- ------------------ ------------------------------ ----------
  PCT_USED  INI_TRANS  MAX_TRANS INITIAL_EXTENT NEXT_EXTENT MIN_EXTENT
---------- ---------- ---------- -------------- ----------- ----------
MAX_EXTENT PCT_INCREASE  FREELISTS FREELIST_GROUPS LOGGING COMPRESS   NUM_ROWS
---------- ------------ ---------- --------------- ------- -------- ----------
    BLOCKS EMPTY_BLOCKS  AVG_SPACE  CHAIN_CNT AVG_ROW_LEN SAMPLE_SIZE LAST_ANAL
---------- ------------ ---------- ---------- ----------- ----------- ---------
BUFFER_ GLO USE
------- --- ---
PRODUCT                        NO  PD1

TABLE_NAME                     COM PARTITION_NAME
------------------------------ --- ------------------------------
SUBPARTITION_COUNT
------------------
HIGH_VALUE
--------------------------------------------------------------------------------
HIGH_VALUE_LENGTH PARTITION_POSITION TABLESPACE_NAME                  PCT_FREE
----------------- ------------------ ------------------------------ ----------
  PCT_USED  INI_TRANS  MAX_TRANS INITIAL_EXTENT NEXT_EXTENT MIN_EXTENT
---------- ---------- ---------- -------------- ----------- ----------
MAX_EXTENT PCT_INCREASE  FREELISTS FREELIST_GROUPS LOGGING COMPRESS   NUM_ROWS
---------- ------------ ---------- --------------- ------- -------- ----------
    BLOCKS EMPTY_BLOCKS  AVG_SPACE  CHAIN_CNT AVG_ROW_LEN SAMPLE_SIZE LAST_ANAL
---------- ------------ ---------- ---------- ----------- ----------- ---------
BUFFER_ GLO USE
------- --- ---
                0
```

alter table product add partition PD5 values('f001','f002','f003','f004');

OUTPUT:

```
SQL> alter table product add partition PD5 values('f001','f002','f003','f004');

Table altered.

SQL> select * from Product Partition(pd5);

no rows selected

SQL> INSERT INTO Product (prdno, prodname, rate, qty_available) VALUES ('f001', 'Fitness Tracker', 4
0.00, 150);

1 row created.

SQL> INSERT INTO Product (prdno, prodname, rate, qty_available) VALUES ('f002', 'Yoga Mat', 25.00, 2
00);

1 row created.

SQL> INSERT INTO Product (prdno, prodname, rate, qty_available) VALUES ('f003', 'Dumbbell Set', 60.0
0, 100);

1 row created.

SQL> INSERT INTO Product (prdno, prodname, rate, qty_available) VALUES ('f004', 'Water Bottle', 15.0
0, 300);

1 row created.

SQL>  select * from Product Partition(pd5);

PRDNO      PRODNAME                       RATE QTY_AVAILABLE
---------- ---------------------- ---------- -------------
f001       Fitness Tracker                40           150
f002       Yoga Mat                       25           200
f003       Dumbbell Set                   60           100
f004       Water Bottle                   15           300
```

alter table PRODUCT rename partition PD5 to PD6;

OUTPUT:

```
SQL> alter table PRODUCT rename partition PD5 to PD6;

Table altered.

SQL>  select * from Product Partition(pd6);

PRDNO      PRODNAME                       RATE QTY_AVAILABLE
---------- ---------------------- ---------- -------------
f001       Fitness Tracker                40           150
f002       Yoga Mat                       25           200
f003       Dumbbell Set                   60           100
f004       Water Bottle                   15           300
```

alter table PRODUCT Modify partition PD6 add values('f005');

OUTPUT:

```
SQL> alter table PRODUCT Modify partition PD6 add values('f005');

Table altered.

SQL>   select * from Product Partition(pd6);

PRDNO      PRODNAME                       RATE QTY_AVAILABLE
---------- ---------------------- ---------- -------------
f001       Fitness Tracker                40           150
f002       Yoga Mat                       25           200
f003       Dumbbell Set                   60           100
f004       Water Bottle                   15           300

SQL> INSERT INTO Product (prdno, prodname, rate, qty_available) VALUES ('f005', 'Fitness
00, 80);

1 row created.

SQL> select * from Product Partition(pd6);

PRDNO      PRODNAME                       RATE QTY_AVAILABLE
---------- ---------------------- ---------- -------------
f001       Fitness Tracker                40           150
f002       Yoga Mat                       25           200
f003       Dumbbell Set                   60           100
f004       Water Bottle                   15           300
f005       Fitness Watch                  50            80
```

Alter table PRODUCT Modify Partition PD6 Drop values('f005');

OUTPUT:

```
SQL> DELETE FROM Product WHERE prdno = 'f005';

1 row deleted.

SQL> select * from Product Partition(pd6);

PRDNO      PRODNAME                       RATE QTY_AVAILABLE
---------- ---------------------- ---------- -------------
f001       Fitness Tracker                40           150
f002       Yoga Mat                       25           200
f003       Dumbbell Set                   60           100
f004       Water Bottle                   15           300
```

select * from product partition(pd4);
OUTPUT:

```
SQL> select * from product partition(pd4);

PRDNO      PRODNAME                       RATE QTY_AVAILABLE
---------- ---------------------- ---------- -------------
d001       Graphics Card                  70            10
d002       External Monitor               75             5
d003       Gaming Console                 80             8
d004       VR Headset                     85             3
```

Alter table PRODUCT Merge Partitions PD3, PD6 into Partition PD8;
OUTPUT:

```
SQL> Alter table PRODUCT Merge Partitions PD3, PD6 into Partition PD8;

Table altered.

SQL> select * from Product Partition (pd8);

PRDNO      PRODNAME                       RATE QTY_AVAILABLE
---------- ---------------------- ---------- -------------
c001       Laptop Bag                     50            50
c002       Monitor Stand                  55            40
c003       Webcam                         60            30
c004       Wireless Charger               65            20
f001       Fitness Tracker                40           150
f002       Yoga Mat                       25           200
f003       Dumbbell Set                   60           100
f004       Water Bottle                   15           300

8 rows selected.
```

hash partition

```
create table employees
(
 empno number,
 empname varchar(30),
 department varchar(30),
 salary number
)


partition by hash(empno)
(
 partition p1,
 partition p2,
 partition p3,
 partition p4
);
```

```
Connected to:
Oracle Database 10g Enterprise Edition Release 10.2.0.1.0 - Production
With the Partitioning, OLAP and Data Mining options

SQL> create table employees
  2  (
  3  empno number,
  4  empname varchar(30),
  5  department varchar(30),
  6  salary number
  7  )
  8  partition by hash (empno)
  9  (
 10  partition p1,
 11  partition p2,
 12  partition p3,
 13  partition p4
 14  );

Table created.

SQL> desc employees;
 Name                                      Null?    Type
 ----------------------------------------- -------- ------------------------
 EMPNO                                              NUMBER
 EMPNAME                                            VARCHAR2(30)
 DEPARTMENT                                         VARCHAR2(30)
 SALARY                                             NUMBER
```

insert into employees values (1, 'Alisha', 'hr', 4000);

insert into employees values (2, 'John', 'engineering', 6000);

insert into employees values (3, 'Sonali', 'marketing', 3000);

insert into employees values (4, 'Sam', 'finance', 7000);

insert into employees values (5, 'Vicky', 'hr', 2000);

insert into employees values (6, 'Feriha', 'engineering', 5000);

OUTPUT:

```
SQL> insert into employees values(1,'Siddharth','hr',4000);
1 row created.
SQL> insert into employees values(2,'Sarvesh','enginerring',6000);
1 row created.
SQL> insert into employees values(3,'Ziya','marketing',3000);
1 row created.
SQL> insert into employees values(4,'Vanshika','finance',7000);
1 row created.
SQL> insert into employees values(5,'Akash','hr',2000);
1 row created.
SQL> insert into employees values(6,'Pratik','engineering',5000);
1 row created.
```

* from employees;

OUTPUT:

select * from employees partition (p1);

select * from employees partition (p2);

select * from employees partition (p3);

select * from employees partition (p4);

OUTPUT:

```
SQL> select * from employees;

     EMPNO EMPNAME                        DEPARTMENT
---------- ------------------------------ ------------------------------
    SALARY
----------
         6 Pratik                         engineering
      5000

         2 Sarvesh                        enginerring
      6000

         5 Akash                          hr
      2000


     EMPNO EMPNAME                        DEPARTMENT
---------- ------------------------------ ------------------------------
    SALARY
----------
         1 Siddharth                      hr
      4000

         3 Ziya                           marketing
      3000

         4 Vanshika                       finance
      7000


6 rows selected.
```

```
SQL> select * from employees partition (p1);

     EMPNO EMPNAME                        DEPARTMENT
---------- ------------------------------ ------------------------------
    SALARY
----------
         6 Pratik                         engineering
      5000


SQL> select * from employees partition (p2);

no rows selected

SQL> select * from employees partition (p3);

     EMPNO EMPNAME                        DEPARTMENT
---------- ------------------------------ ------------------------------
    SALARY
----------
         2 Sarvesh                        enginerring
      6000

         5 Akash                          hr
      2000


SQL> select * from employees partition (p4);

     EMPNO EMPNAME                        DEPARTMENT
---------- ------------------------------ ------------------------------
    SALARY
----------
         1 Siddharth                      hr
      4000

         3 Ziya                           marketing
      3000

         4 Vanshika                       finance
      7000
```

update employees set salary = salary + (salary * 0.05) where salary < 5000;

OUTPUT:

```
SQL> update employees set salary = salary + (salary * 0.05) where salary < 5000;

3 rows updated.

SQL> select * from employees
  2  ;

     EMPNO EMPNAME                        DEPARTMENT
---------- ------------------------------ ------------------------------
    SALARY
----------
         6 Pratik                         engineering
      5000

         2 Sarvesh                        enginerring
      6000

         5 Akash                          hr
      2100


     EMPNO EMPNAME                        DEPARTMENT
---------- ------------------------------ ------------------------------
    SALARY
----------
         1 Siddharth                      hr
      4200

         3 Ziya                           marketing
      3150

         4 Vanshika                       finance
      7000

6 rows selected.
```

PRACTICAL NO 2

AIM- Implementation of Analytical queries like Roll_UP, CUBE, First, Last, Lead, Lag, Rank AND Dense Rank and Windowing functions- preceding rows, following and rows between

```
SQL> create table employee
  2  (
  3   empno number,
  4   empname varchar(30),
  5   department varchar(30),
  6   salary number
  7  ) ;

Table created.

SQL> select * from employee;

no rows selected

SQL> desc employee;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 EMPNO                                              NUMBER
 EMPNAME                                            VARCHAR2(30)
 DEPARTMENT                                         VARCHAR2(30)
 SALARY                                             NUMBER

SQL> insert into employee values (1, 'Siddharth', 'hr', 4000);

1 row created.

SQL> insert into employee values (2, 'Sarvesh', 'engineering', 6000);

1 row created.

SQL>  insert into employee values (3, 'Ziya', 'marketing', 3000);

1 row created.

SQL> insert into employee values (4, 'Vanshika', 'finance', 7000);

1 row created.

SQL>  insert into employee values (5, 'Akash', 'hr', 2000);

1 row created.

SQL> insert into employee values (6, 'Pratik', 'engineering', 5000);

1 row created.
```

```
SQL> insert into employee values (7, 'Tanveer', 'engineering', 6000);

1 row created.

SQL>  insert into employee values (8, 'Amey', 'marketing', 3000);

1 row created.

SQL> insert into employee values (9, 'Sarah', 'finance', 7000);

1 row created.

SQL> insert into employee values (10, 'Virendra', 'finance', 2000);

1 row created.
```

```
SQL> set pagesize 2000;
SQL> set linesize 1000;
SQL> select * from employee;

     EMPNO EMPNAME                        DEPARTMENT                          SALARY
---------- ------------------------------ ------------------------------ ----------
         1 Siddharth                      hr                                   4000
         2 Sarvesh                        engineering                          6000
         3 Ziya                           marketing                            3000
         4 Vanshika                       finance                              7000
         5 Akash                          hr                                   2000
         6 Pratik                         engineering                          5000
         7 Tanveer                        engineering                          6000
         8 Amey                           marketing                            3000
         9 Sarah                          finance                              7000
        10 Virendra                       finance                              2000

10 rows selected.
```

Rank

select Empno, empname, department ,salary, rank() over (order by salary) "Rank" from employee;

```
SQL> select Empno,  empname, department ,salary, rank() over (order by salary) "Rank" from employee;

     EMPNO EMPNAME                        DEPARTMENT                       SALARY       Rank
---------- ------------------------------ ------------------------------ ---------- ----------
         5 Akash                          hr                                 2000          1
        10 Virendra                       finance                            2000          1
         3 Ziya                           marketing                          3000          3
         8 Amey                           marketing                          3000          3
         1 Siddharth                      hr                                 4000          5
         6 Pratik                         engineering                        5000          6
         7 Tanveer                        engineering                        6000          7
         2 Sarvesh                        engineering                        6000          7
         4 Vanshika                       finance                            7000          9
         9 Sarah                          finance                            7000          9

10 rows selected.
```

Dense_Rank

select empno, empname, department, salary, dense_rank() over(order by salary)"den_rank" from employee;

```
SQL> set pagesize 2000;
SQL> set linesize 1000;
SQL> select empno, empname, department, salary, dense_rank() over(order by salary)"den_rank" from em
ployee;

     EMPNO EMPNAME                        DEPARTMENT                       SALARY   den_rank
---------- ------------------------------ ------------------------------ ---------- ----------
         5 Akash                          hr                                 2000          1
        10 Virendra                       finance                            2000          1
         3 Ziya                           marketing                          3000          2
         8 Amey                           marketing                          3000          2
         1 Siddharth                      hr                                 4000          3
         6 Pratik                         engineering                        5000          4
         7 Tanveer                        engineering                        6000          5
         2 Sarvesh                        engineering                        6000          5
         4 Vanshika                       finance                            7000          6
         9 Sarah                          finance                            7000          6

10 rows selected.
```

cube

select empno, department, sum(salary) from employee Group by cube(empno, department) order by department, empno;

```
SQL> set pagesize 2000;
SQL> set linesize 1000;
SQL> select empno, department, sum(salary) from employee Group by cube(empno, department) order by d
epartment, empno;

     EMPNO DEPARTMENT                     SUM(SALARY)
---------- ------------------------------ -----------
         2 engineering                           6000
         6 engineering                           5000
         7 engineering                           6000
           engineering                          17000
         4 finance                              7000
         9 finance                              7000
        10 finance                              2000
           finance                             16000
         1 hr                                   4000
         5 hr                                   2000
           hr                                   6000
         3 marketing                            3000
         8 marketing                            3000
           marketing                            6000
         1                                      4000
         2                                      6000
         3                                      3000
         4                                      7000
         5                                      2000
         6                                      5000
         7                                      6000
         8                                      3000
         9                                      7000
        10                                      2000
                                               45000

25 rows selected.
```

rollup

select empno, department, sum(salary) from employees Group by rollup(empno, department) order by empno, department;

```
SQL> set pagesize 2000;
SQL> set linesize 1000;
SQL> select empno, department, sum(salary) from employee Group by rollup(empno, department) order by
 empno, department;

     EMPNO DEPARTMENT                     SUM(SALARY)
---------- ------------------------------ -----------
         1 hr                                   4000
         1                                      4000
         2 engineering                          6000
         2                                      6000
         3 marketing                            3000
         3                                      3000
         4 finance                              7000
         4                                      7000
         5 hr                                   2000
         5                                      2000
         6 engineering                          5000
         6                                      5000
         7 engineering                          6000
         7                                      6000
         8 marketing                            3000
         8                                      3000
         9 finance                              7000
         9                                      7000
        10 finance                              2000
        10                                      2000
                                               45000

21 rows selected.
```

last

select department, last_value(salary) over (partition by department order by salary) as last_salary from employee;

```
SQL> set pagesize 2000;
SQL> set linesize 1000;
SQL> select department, last_value(salary) over (partition by department order by salary) as last_sa
lary from employee;

DEPARTMENT                     LAST_SALARY
------------------------------ -----------
engineering                           5000
engineering                           6000
engineering                           6000
finance                               2000
finance                               7000
finance                               7000
hr                                    2000
hr                                    4000
marketing                             3000
marketing                             3000

10 rows selected.
```

first

select department, first_value(salary) over (partition by department order by salary) as first_salary from employee;

```
SQL> set pagesize 2000;
SQL> set linesize 1000;
SQL> select department, first_value(salary) over (partition by department order by salary) as first_
salary from employee;

DEPARTMENT                     FIRST_SALARY
------------------------------ ------------
engineering                            5000
engineering                            5000
engineering                            5000
finance                                2000
finance                                2000
finance                                2000
hr                                     2000
hr                                     2000
marketing                              3000
marketing                              3000

10 rows selected.
```

lead

select empname, salary, lead(salary, 1) over (order by salary asc) as next_salary from employee;

```
SQL> set pagesize 2000;
SQL> set linesize 1000;
SQL> select empname, salary, lead(salary, 1) over (order by salary asc) as next_salary from employee
;

EMPNAME                          SALARY NEXT_SALARY
------------------------------ ---------- -----------
Akash                              2000        2000
Virendra                           2000        3000
Ziya                               3000        3000
Amey                               3000        4000
Siddharth                          4000        5000
Pratik                             5000        6000
Tanveer                            6000        6000
Sarvesh                            6000        7000
Vanshika                           7000        7000
Sarah                              7000

10 rows selected.
```

lag

select empname, salary, lag(salary, 1) over (order by salary asc) as previous_salary from employee;

```
SQL> set pagesize 2000;
SQL> set linesize 1000;
SQL> select empname, salary, lag(salary, 1) over (order by salary asc) as previous_salary from emplo
yee;

EMPNAME                          SALARY PREVIOUS_SALARY
------------------------------ ---------- ---------------
Akash                              2000
Virendra                           2000            2000
Ziya                               3000            2000
Amey                               3000            3000
Siddharth                          4000            3000
Pratik                             5000            4000
Tanveer                            6000            5000
Sarvesh                            6000            6000
Vanshika                           7000            6000
Sarah                              7000            7000

10 rows selected.
```

Window

```
CREATE TABLE Employee1(
   e_id number(4),
  first_name varchar2(15),
  last_name varchar2(15),
  dept_id number(3),
  hire_date date,
  salary number(10,2)
   );
```

```
Connected to:
Oracle Database 10g Enterprise Edition Release 10.2.0.1.0 - Production
With the Partitioning, OLAP and Data Mining options

SQL> CREATE TABLE Employee1(
  2  e_id number(4),
  3  first_name varchar2(15),
  4  last_name varchar2(15),
  5  dept_id number(3),
  6  hire_date date,
  7  salary number(10,2)
  8  );

Table created.
```

```
INSERT INTO Employee1 values(101, 'Steven', 'Patil', 10, '22-Jan-2001',
20000);

INSERT INTO Employee1 values(107, 'Michael', 'Singh', 10, '19-Jan-2001',
30000);

INSERT INTO Employee1 values(106, 'Peter', 'Patil', 10, '15-Feb-2001', 50000);


INSERT INTO Employee1 values(102, 'James', 'Patil', 10, '10-Mar-2001',
200000);

INSERT INTO Employee1 values(103, 'Aarav', 'Patil', 10, '03-Mar-2001',
70000);

INSERT INTO Employee1 values(104, 'David', 'Patil', 10, '23-Jan-2001', 40000);
```

```
INSERT INTO Employee1 values(115, 'Prasanth', 'Patil', 20, '16-Jul-2001',
6000);


INSERT INTO Employee1 values(126, 'Nataraj', 'Patil', 40, '14-Jun-2001',
16000);
INSERT INTO Employee1 values(116, 'Naresh', 'Patil', 20, '29-Jan-2001',
25000);
INSERT INTO Employee1 values(118, 'Sreenivas', 'Patil', 29, '29-Jan-2001',
20000);
INSERT INTO Employee1 values(108, 'Mohammed', 'Singh', 50, '19-Jan-2001',
30000);
INSERT INTO Employee1 values(110, 'Regina', 'Patil', 30, '16-Feb-2001',
25000);
INSERT INTO Employee1 values(113, 'Chandan', 'Patil', 30, '10-Mar-2001',
29000);
COMMIT;
```

```
SQL>
SQL>
SQL>  INSERT INTO Employee1 values(101, 'Steven', 'Patil', 10, '22-Jan-2001', 20000);

1 row created.

SQL>
SQL> INSERT INTO Employee1 values(107, 'Michael', 'Singh', 10, '19-Jan-2001', 30000);

1 row created.

SQL>
SQL>  INSERT INTO Employee1 values(106, 'Peter', 'Patil', 10, '15-Feb-2001', 50000);

1 row created.

SQL>
SQL>
SQL> INSERT INTO Employee1 values(102, 'James', 'Patil', 10, '10-Mar-2001', 200000);

1 row created.

SQL>
SQL>  INSERT INTO Employee1 values(103, 'Aarav', 'Patil', 10, '03-Mar-2001', 70000);

1 row created.

SQL>
SQL>  INSERT INTO Employee1 values(104, 'David', 'Patil', 10, '23-Jan-2001', 40000);

1 row created.

SQL>
SQL>
SQL>  INSERT INTO Employee1 values(115, 'Prasanth', 'Patil', 20, '16-Jul-2001', 6000);

1 row created.

SQL>
SQL>
SQL>  INSERT INTO Employee1 values(126, 'Nataraj', 'Patil', 40, '14-Jun-2001', 16000);

1 row created.

SQL>  INSERT INTO Employee1 values(116, 'Naresh', 'Patil', 20, '29-Jan-2001', 25000);

1 row created.

SQL>  INSERT INTO Employee1 values(116, 'Naresh', 'Patil', 20, '29-Jan-2001', 25000);

1 row created.

SQL> INSERT INTO Employee1 values(118, 'Sreenivas', 'Patil', 29, '29-Jan-2001', 20000);

1 row created.

SQL>  INSERT INTO Employee1 values(108, 'Mohammed', 'Singh', 50, '19-Jan-2001', 30000);

1 row created.

SQL>  INSERT INTO Employee1 values(110, 'Regina', 'Patil', 30, '16-Feb-2001', 25000);

1 row created.

SQL>  INSERT INTO Employee1 values(113, 'Chandan', 'Patil', 30, '10-Mar-2001', 29000);

1 row created.

SQL>  COMMIT;
```

SELECT e_id,first_name,dept_id,salary,SUM(salary)

OVER (order by e_id)"Cumulative_Sal"
FROM Employee1;

```
SQL> set pagesize 1000
SQL> set linesize 1000
SQL>  SELECT e_id,first_name,dept_id,salary,SUM(salary)
  2     OVER (order by e_id)"Cumulative_Sal"
  3     FROM Employee1;

      E_ID FIRST_NAME         DEPT_ID     SALARY Cumulative_Sal
---------- --------------- ---------- ---------- --------------
       101 Steven                  10      20000          20000
       102 James                   10     200000         220000
       103 Aarav                   10      70000         290000
       104 David                   10      40000         330000
       106 Peter                   10      50000         380000
       107 Michael                 10      30000         410000
       108 Mohammed                50      30000         440000
       110 Regina                  30      25000         465000
       113 Chandan                 30      29000         494000
       115 Prasanth                20       6000         500000
       116 Naresh                  20      25000         525000
       118 Sreenivas               29      20000         545000
       126 Nataraj                 40      16000         561000

13 rows selected.
```

SELECT e_id, first_name, dept_id, salary, SUM(salary)
OVER (partition by dept_id order by salary Rows unbounded preceding) "Cumulative"
FROM Employee1;

```
SQL>  SELECT e_id, first_name, dept_id, salary, SUM(salary)
  2             OVER (partition by dept_id order by salary Rows unbounded preceding) "Cumulative"
  3         FROM Employee1;

      E_ID FIRST_NAME         DEPT_ID     SALARY Cumulative
---------- --------------- ---------- ---------- ----------
       101 Steven                  10      20000      20000
       107 Michael                 10      30000      50000
       104 David                   10      40000      90000
       106 Peter                   10      50000     140000
       103 Aarav                   10      70000     210000
       102 James                   10     200000     410000
       115 Prasanth                20       6000       6000
       116 Naresh                  20      25000      31000
       118 Sreenivas               29      20000      20000
       110 Regina                  30      25000      25000
       113 Chandan                 30      29000      54000
       126 Nataraj                 40      16000      16000
       108 Mohammed                50      30000      30000

13 rows selected.
```

SELECT e_id, first_name, dept_id, salary,
SUM(salary) OVER (ORDER BY e_id ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) "Cumulative_Sal"
FROM Employee1;

```
SQL> SQL> SELECT e_id, first_name, dept_id, salary,
  2    SUM(salary) OVER (ORDER BY e_id ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) "Cumulative_Sal"
  3     FROM Employee1;

      E_ID FIRST_NAME         DEPT_ID     SALARY Cumulative_Sal
---------- --------------- ---------- ---------- --------------
       101 Steven                  10      20000          20000
       102 James                   10     200000         220000
       103 Aarav                   10      70000         290000
       104 David                   10      40000         310000
       106 Peter                   10      50000         160000
       107 Michael                 10      30000         120000
       108 Mohammed                50      30000         110000
       110 Regina                  30      25000          85000
       113 Chandan                 30      29000          84000
       115 Prasanth                20       6000          60000
       116 Naresh                  20      25000          60000
       118 Sreenivas               29      20000          51000
       126 Nataraj                 40      16000          61000

13 rows selected.
```

SELECT e_id, first_name, dept_id, salary,
SUM(salary) OVER (ORDER BY dept_id ROWS BETWEEN 3 PRECEDING and 1 following) "Cumulative_Sal"
FROM Employee1;

```
SQL>  SELECT e_id, first_name, dept_id, salary,
  2             SUM(salary) OVER (ORDER BY dept_id ROWS BETWEEN 3 PRECEDING and  1 following) "Cumulative_Sal"
  3           FROM Employee1;

      E_ID FIRST_NAME         DEPT_ID     SALARY Cumulative_Sal
---------- --------------- ---------- ---------- --------------
       101 Steven                  10      20000          50000
       107 Michael                 10      30000         100000
       106 Peter                   10      50000         300000
       102 James                   10     200000         370000
       103 Aarav                   10      70000         390000
       104 David                   10      40000         366000
       115 Prasanth                20       6000         341000
       116 Naresh                  20      25000         161000
       118 Sreenivas               29      20000         116000
       110 Regina                  30      25000         105000
       113 Chandan                 30      29000         115000
       126 Nataraj                 40      16000         120000
       108 Mohammed                50      30000         100000

13 rows selected.
```

SELECT e_id,first_name,dept_id,salary, SUM(salary)
OVER (order by e_id Rows between 2 preceding and 2 following)"Cumulative_Sal"
FROM Employee1;

```
SQL>  SELECT e_id,first_name,dept_id,salary, SUM(salary)
  2      OVER (order by e_id Rows between 2 preceding and 2 following)"Cumulative_Sal"
  3      FROM Employee1;

      E_ID FIRST_NAME         DEPT_ID     SALARY Cumulative_Sal
---------- --------------- ---------- ---------- --------------
       101 Steven                  10      20000         290000
       102 James                   10     200000         330000
       103 Aarav                   10      70000         380000
       104 David                   10      40000         390000
       106 Peter                   10      50000         220000
       107 Michael                 10      30000         175000
       108 Mohammed                50      30000         164000
       110 Regina                  30      25000         120000
       113 Chandan                 30      29000         115000
       115 Prasanth                20       6000         105000
       116 Naresh                  20      25000          96000
       118 Sreenivas               29      20000          67000
       126 Nataraj                 40      16000          61000

13 rows selected.
```

SELECT e_id,first_name,hire_date,count(e_id)
OVER(order by hire_date range interval '15' day preceding)"Days"
FROM Employee1;

```
SQL>  SELECT e_id,first_name,hire_date,count(e_id)
  2     OVER(order by hire_date range interval '15' day preceding)"Days"
  3     FROM Employee1;

      E_ID FIRST_NAME      HIRE_DATE       Days
---------- --------------- --------- ----------
       107 Michael         19-JAN-01          2
       108 Mohammed        19-JAN-01          2
       101 Steven          22-JAN-01          3
       104 David           23-JAN-01          4
       116 Naresh          29-JAN-01          6
       118 Sreenivas       29-JAN-01          6
       106 Peter           15-FEB-01          1
       110 Regina          16-FEB-01          2
       103 Aarav           03-MAR-01          2
       113 Chandan         10-MAR-01          3
       102 James           10-MAR-01          3
       126 Nataraj         14-JUN-01          1
       115 Prasanth        16-JUL-01          1

13 rows selected.
```

SELECT AVG(case when salary<20000 then 20000 else salary end)"Average"
FROM Employee1;

```
SQL>  SELECT AVG(case when salary<20000 then 20000 else salary end)"Average" FROM Employee1;

   Average
----------
44538.4615
```

SELECT first_name,dept_id,AVG(case when salary>20000 then 20000 else 0 end)
OVER(partition by dept_id)"Average" FROM Employee1;

```
SQL>  SELECT first_name,dept_id,AVG(case when salary>20000 then 20000 else 0 end)
  2     OVER(partition by dept_id)"Average" FROM Employee1;

FIRST_NAME         DEPT_ID    Average
--------------- ---------- ----------
Steven                  10 16666.6667
Michael                 10 16666.6667
Peter                   10 16666.6667
James                   10 16666.6667
Aarav                   10 16666.6667
David                   10 16666.6667
Prasanth                20      10000
Naresh                  20      10000
Sreenivas               29          0
Regina                  30      20000
Chandan                 30      20000
Nataraj                 40          0
Mohammed                50      20000

13 rows selected.
```

PRACTICAL NO 3

Aim- Implementation of ORDBMS concepts like ADT (Abstract Data Types), Object table and Inheritance

```
create or replace type ty_address as object
  (street varchar(20),city varchar(20), pincode number(10));
  /
```

```
SQL> create or replace type ty_address as object
  2  (street varchar(20),city varchar(20), pincode number(10));
  3  /

Type created.
```

```
create or replace type ty_name as object
  (fname varchar (20), mname varchar (20), lname varchar (20));
  /
```

```
SQL> create or replace type ty_name as object
  2  (fname varchar (20), mname varchar (20), lname varchar (20));
  3  /

Type created.
```

```
Create table customer_adbt
  ( c_id number(5) primary key, c_name ty_name, c_add ty_address, c_phno
number(10) );
```

```
SQL> Create table customer_adbt
  2  ( c_id number(5) primary key, c_name ty_name, c_add ty_address, c_phno number(10) );

Table created.
```

```
insert into customer_adbt values
(2,ty_name('Akshay','A','Anpat'),ty_address('Marine
  2  Lines','Mumbai',400028),9820173925);

insert into customer_adbt values (1,ty_name('Tushar','D','Kini'),ty_address('IIT
  2  Powai','Mumbai',400076),9892750112);
```

```
insert into customer_adbt values
```

(3,ty_name('Amkit','M','Kansara'),ty_address('Churhgate','Mumbai',400036),9899
162616

```
SQL> insert into customer_adbt values (2,ty_name('Akshay','A','Anpat'),ty_address('Marine
  2  Lines','Mumbai',400028),9820173925);

1 row created.

SQL> insert into customer_adbt values (1,ty_name('Tushar','D','Kini'),ty_address('IIT
  2  Powai','Mumbai',400076),9892750112);

1 row created.

SQL> insert into customer_adbt values
  2  (3,ty_name('Amkit','M','Kansara'),ty_address('Churhgate','Mumbai',400036),9899162616);

1 row created.
```

select c.c_name.fname from customer_adbt c;

```
SQL> select c.c_name.fname from customer_adbt c;

C_NAME.FNAME
--------------------
Akshay
Tushar
Amkit
```

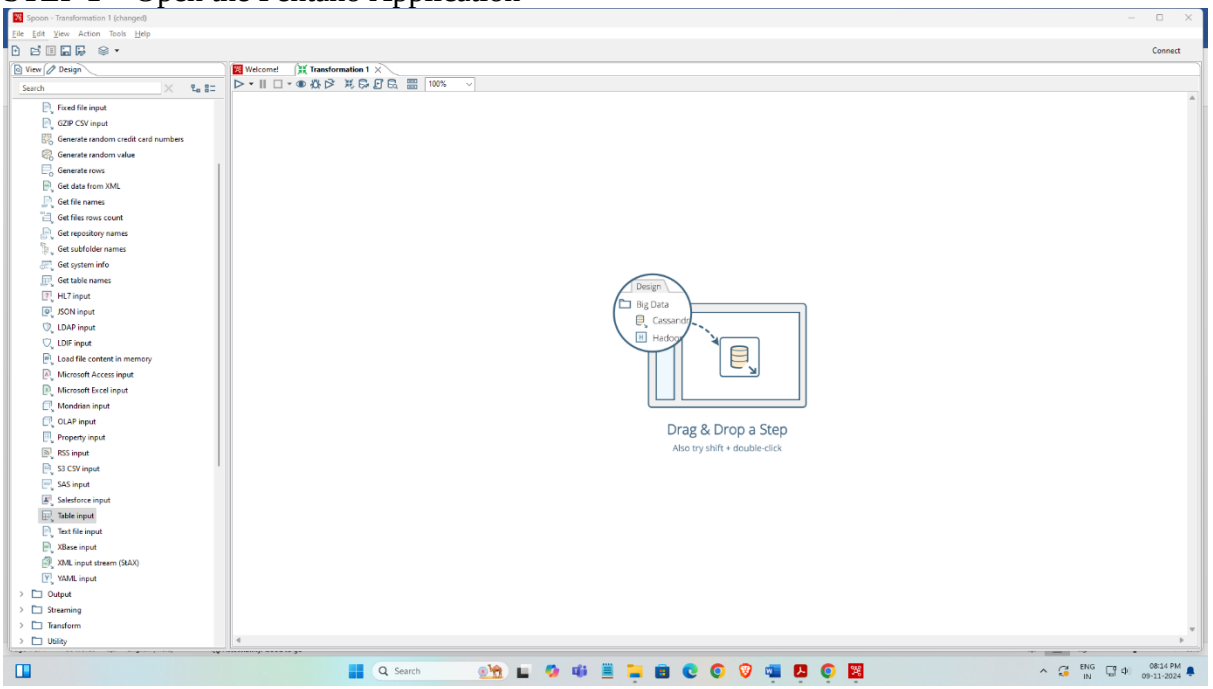
select c.c_name.fname|| ' ' ||c.c_name.mname name from customer_adbt c;

```
SQL> select c.c_name.fname|| ' ' ||c.c_name.mname name from customer_adbt c;

NAME
----------------------------------------
Akshay A
Tushar D
Amkit M
```

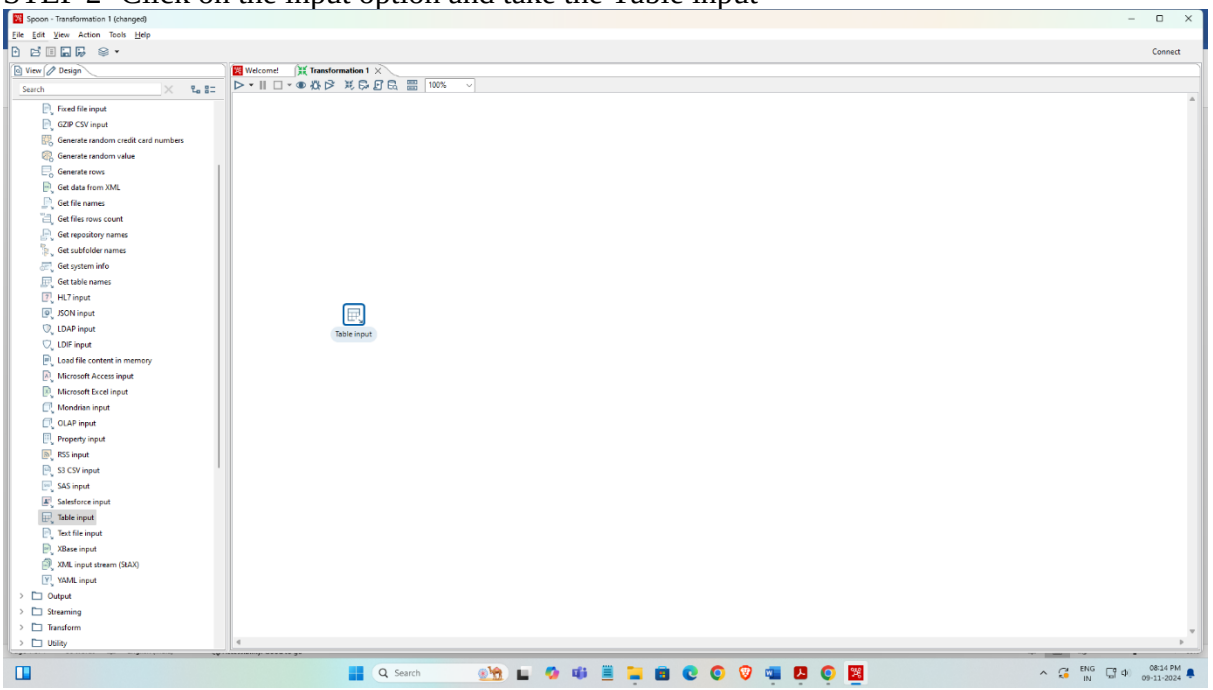
PRACTICAL NO 4

AIM-Implementation of ETL transformation with Pentaho like Copy data from Source (Table/Excel/ Oracle) and store it to Target (Table / Excel / Oracle), Adding sequence, Adding Calculator, Concatenation of two fields, splitting of two fields, Number Range, String Operations, Sorting data, Implement the merge join transformation on tables, Implement data validations on the table data.

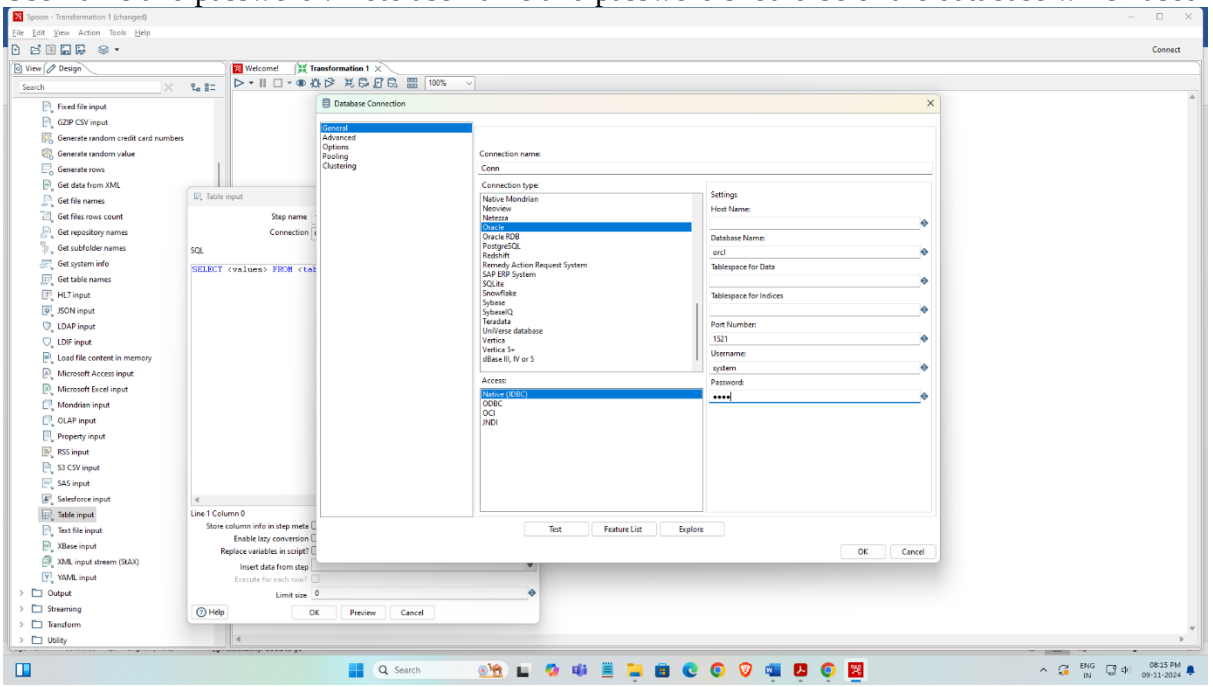
STEP 1 – Open the Pentaho Application



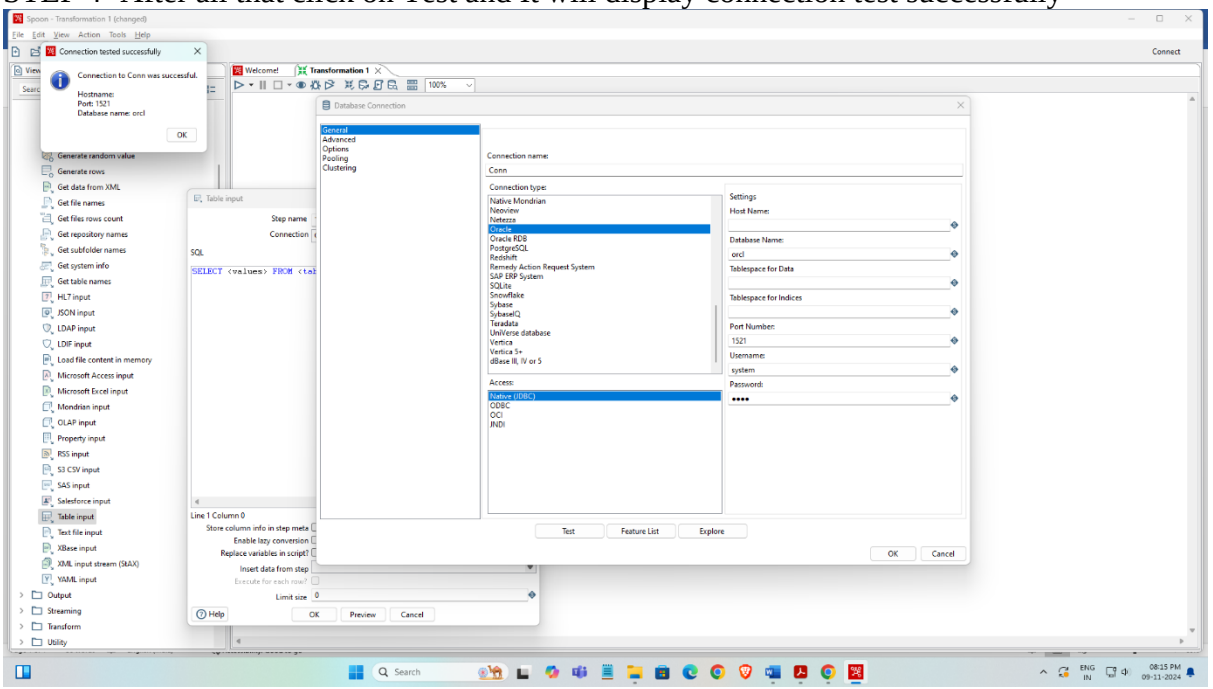
STEP 2- Click on the input option and take the Table input



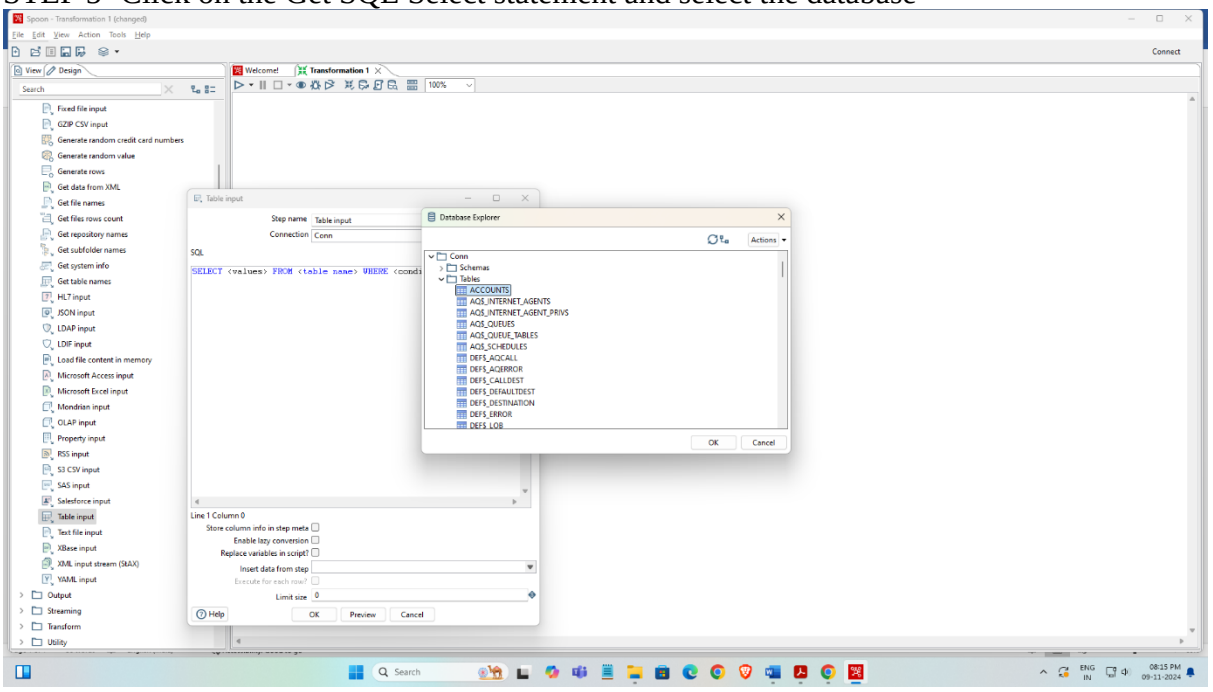
STEP 3- Click on the Table input and file the details like Connection name , Database name , Username and password . Note username and password should be of the database which used



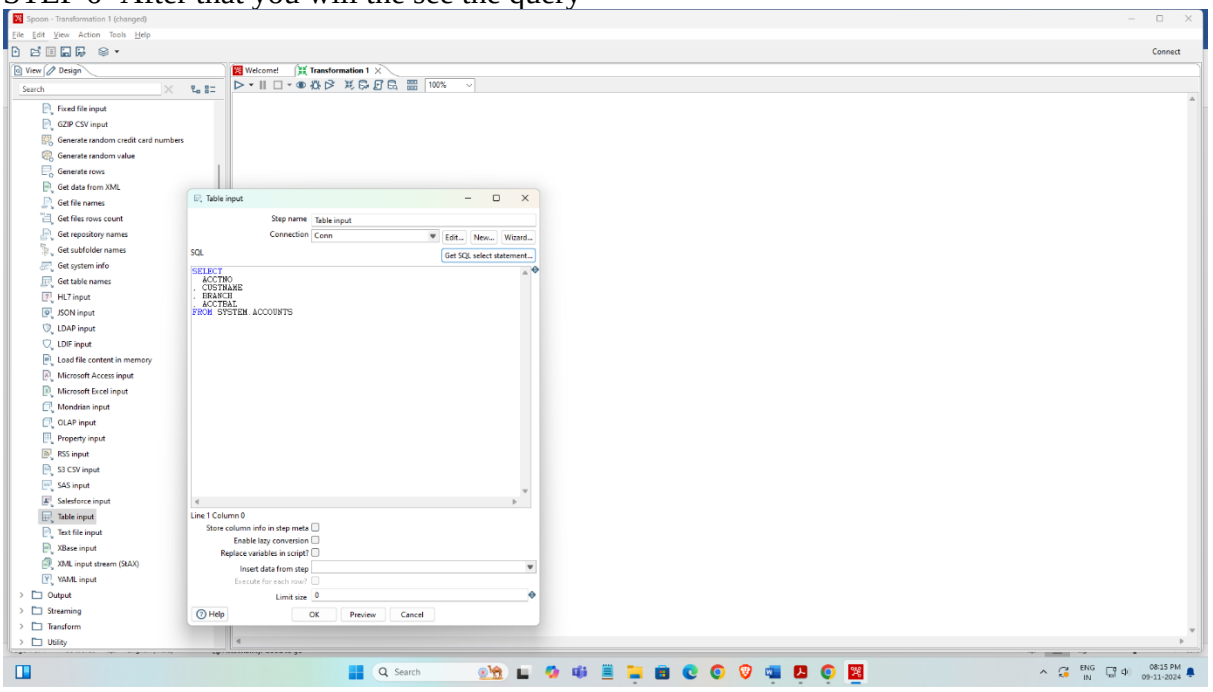
STEP 4- After all that click on Test and It will display connection test successfully



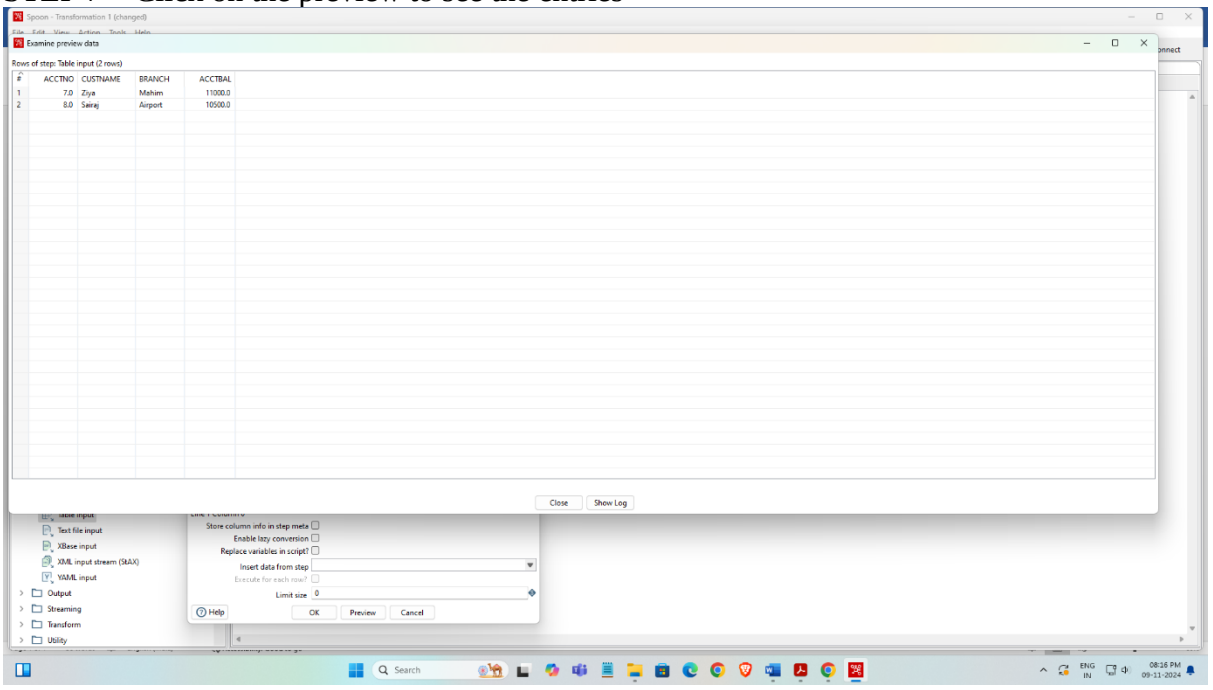
STEP 5- Click on the Get SQL Select statement and select the database



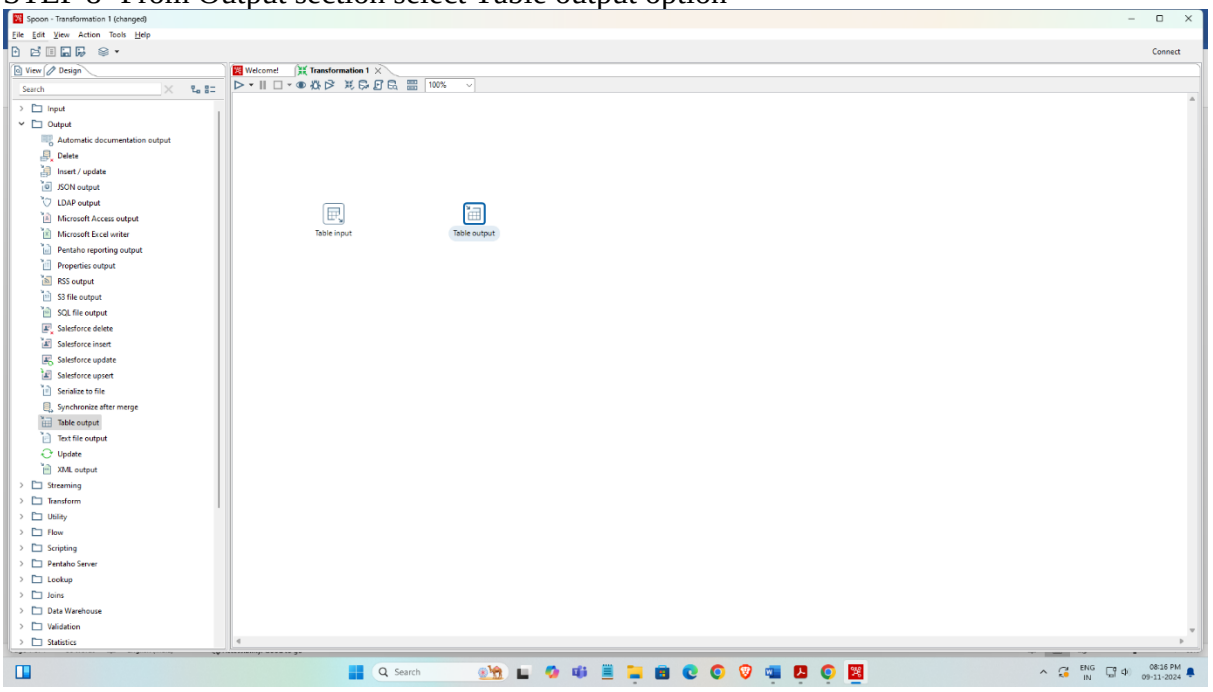
STEP 6- After that you will the see the query



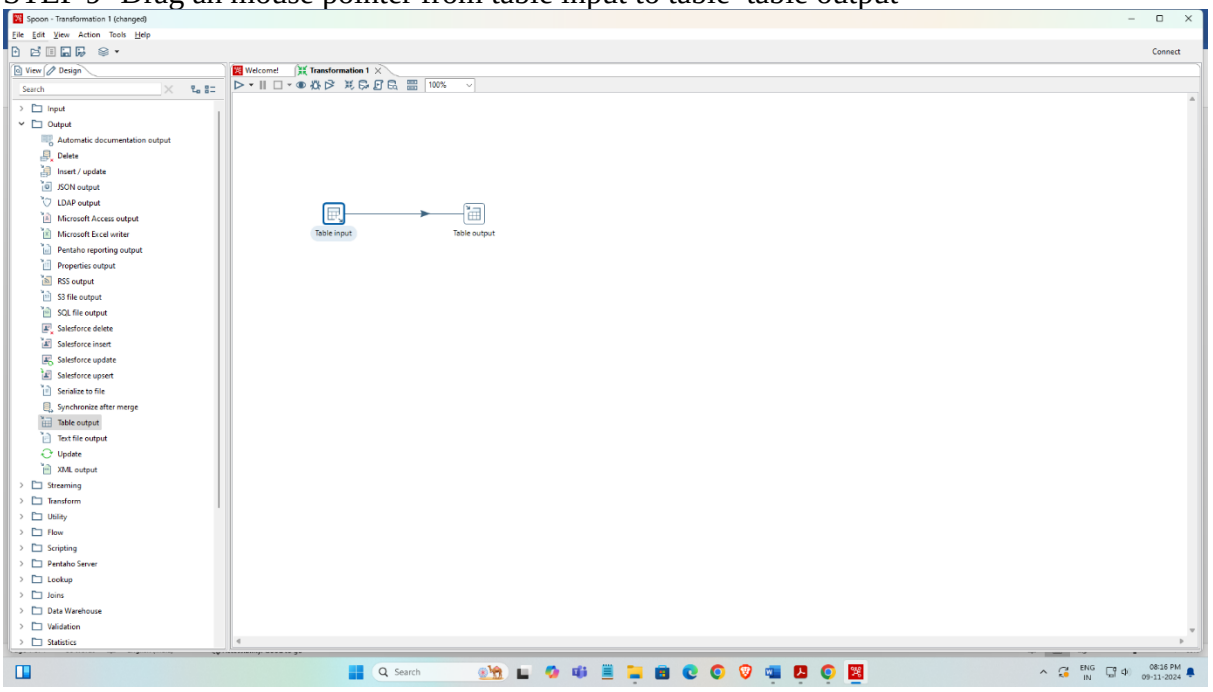
STEP 7 – Click on the preview to see the entries



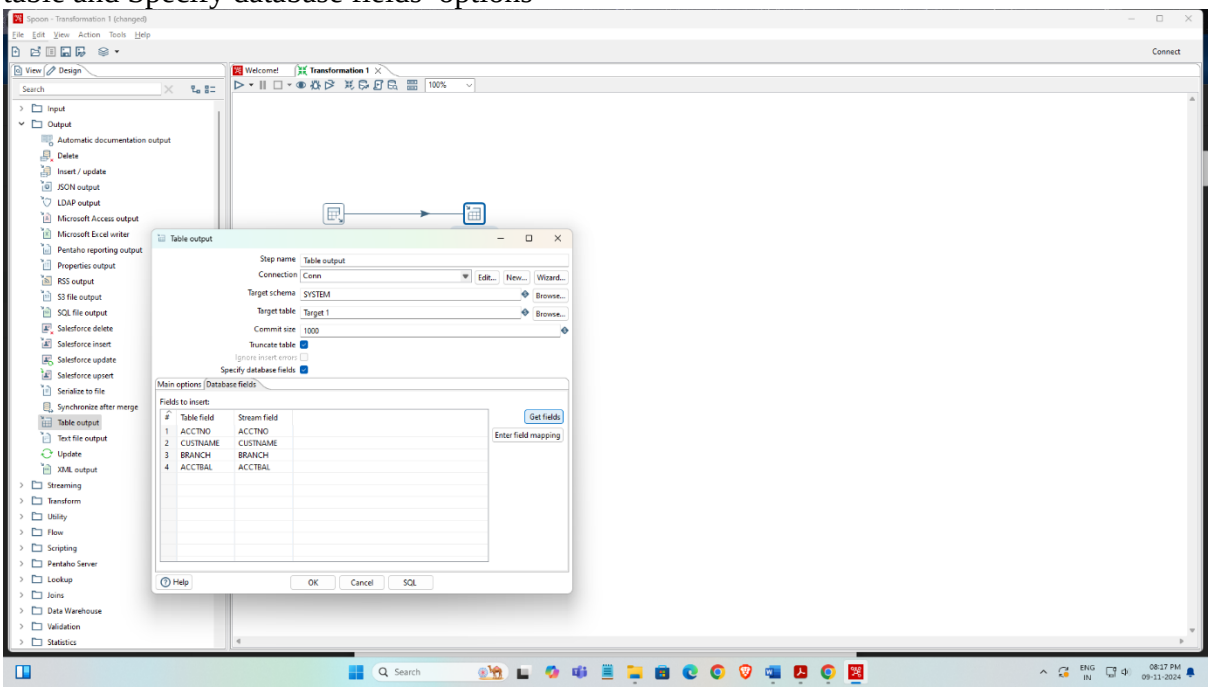
STEP 8- From Output section select Table output option

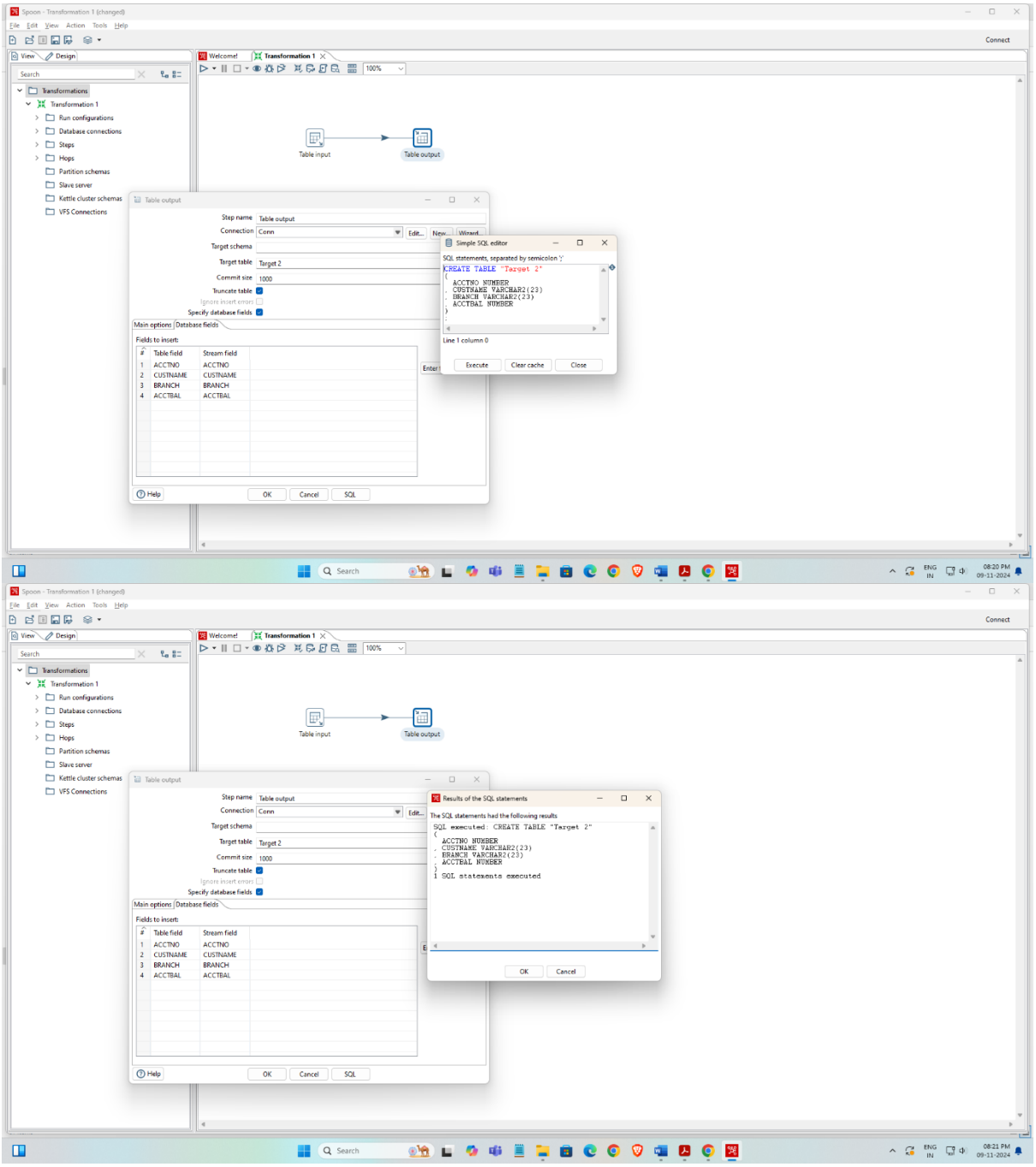


STEP 9- Drag an mouse pointer from table input to table table output

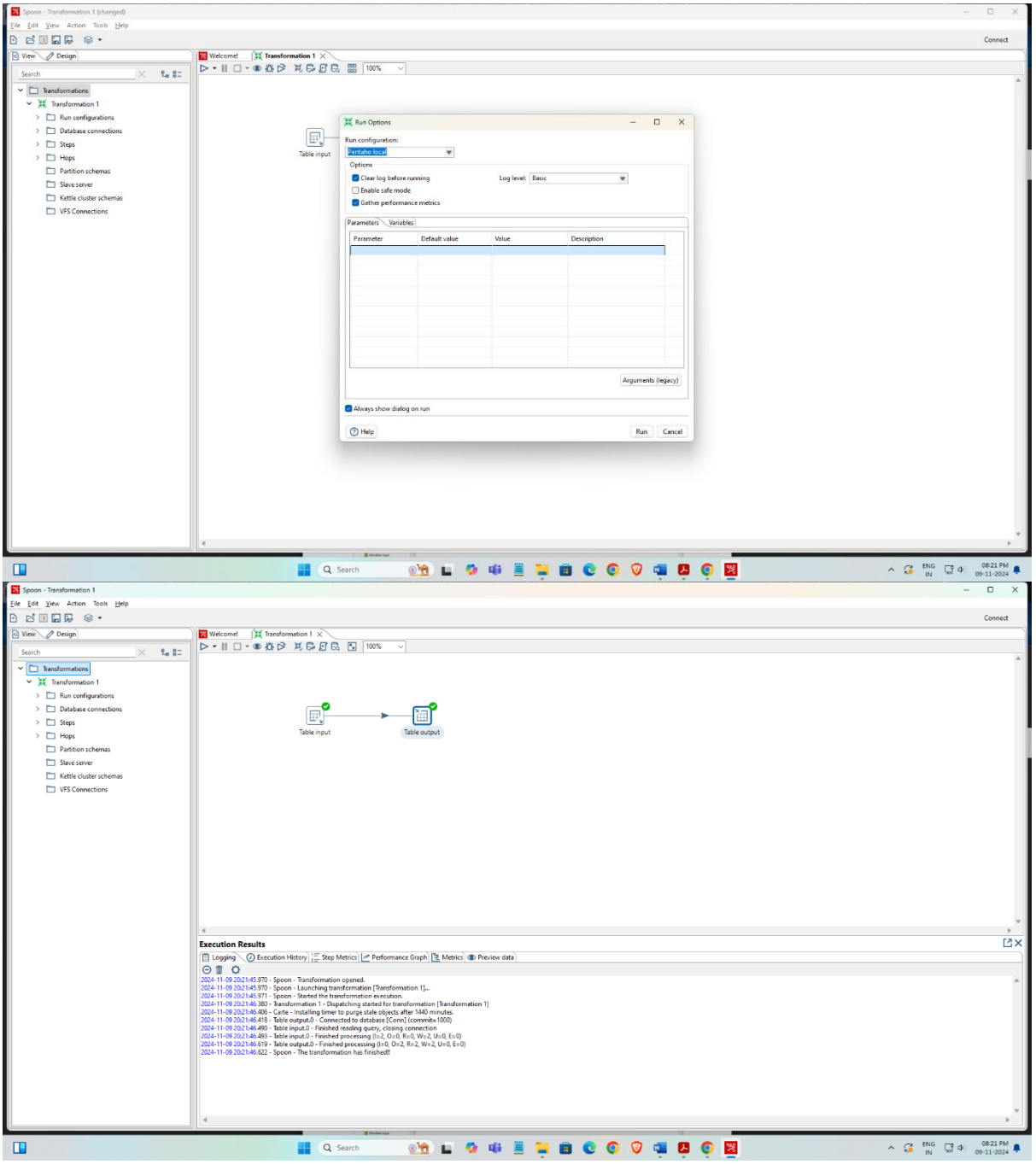


STEP 10- Click the Table output and file the Connection , Target Table, select the truncate table and Specify database fields options




Spoon - Transformation 1 (changed)
Table input
Table output
Simple SQL editor
SQL statements, separated by semicolon ';'
CREATE TABLE "Target 2"
(
ACCTNO NUMBER
, CUSTNAME VARCHAR2(23)
, BRANCH VARCHAR2(23)
, ACCTBAL NUMBER
)
;
Line 1 column 0
Execute
Clear cache
Close
Step name
Table output
Connection
Conn
Target schema
Target table
Target 2
Commit size
1000
Truncate table
Specify database fields
Main options
Database fields
Fields to insert:
Table field
Stream field
ACCTNO
CUSTNAME
BRANCH
ACCTBAL
Help
OK
Cancel
SQL

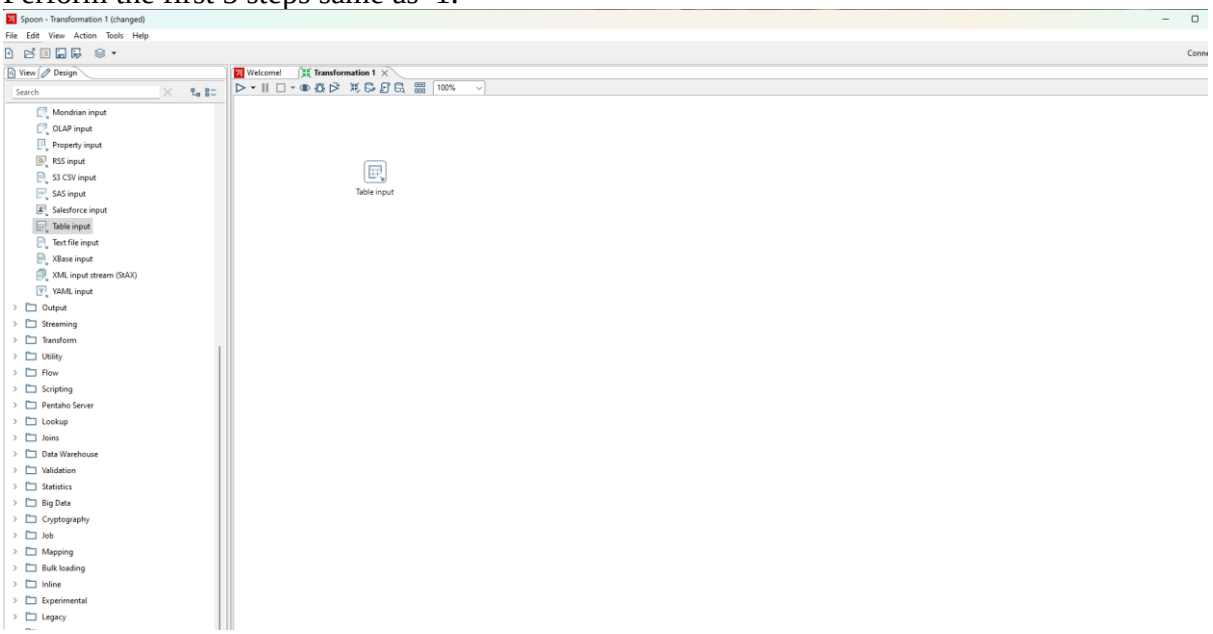
Spoon - Transformation 1 (changed)
Table input
Table output
Results of the SQL statements
The SQL statements had the following results
SQL executed: CREATE TABLE "Target 2"
(
ACCTNO NUMBER
, CUSTNAME VARCHAR2(23)
, BRANCH VARCHAR2(23)
, ACCTBAL NUMBER
)
1 SQL statements executed
OK
Cancel
Step name
Table output
Connection
Conn
Target schema
Target table
Target 2
Commit size
1000
Truncate table
Specify database fields
Main options
Database fields
Fields to insert:
Table field
Stream field
ACCTNO
CUSTNAME
BRANCH
ACCTBAL
Help
OK
Cancel
SQL


Spoon - Transformation 1 (changed)
File Edit View Action Tools Help
Connect
View Design
Search
Transformations
Transformation 1
Run configurations
Database connections
Steps
Hops
Partition schemas
Slave server
Kettle cluster schemas
VFS Connections
Welcome!
Transformation 1
100%
Table input
Run Options
Run configuration:
Pentaho local
Options
Clear log before running
Log level:
Basic
Enable safe mode
Gather performance metrics
Parameters
Variables
Parameter
Default value
Value
Description
Arguments (legacy)
Always show dialog on run
Help
Run
Cancel
Search
ENG
IN
08:21 PM
09-11-2024

Spoon - Transformation 1
File Edit View Action Tools Help
Connect
View Design
Search
Transformations
Transformation 1
Run configurations
Database connections
Steps
Hops
Partition schemas
Slave server
Kettle cluster schemas
VFS Connections
Welcome!
Transformation 1
100%
Table input
Table output
Execution Results
Logging
Execution History
Step Metrics
Performance Graph
Metrics
Preview data
2024-11-09 20:21:45.970 - Spoon - Transformation opened.
2024-11-09 20:21:45.970 - Spoon - Launching transformation [Transformation 1]...
2024-11-09 20:21:45.971 - Spoon - Started the transformation execution.
2024-11-09 20:21:46.380 - Transformation 1 - Dispatching started for transformation [Transformation 1]
2024-11-09 20:21:46.406 - Carte - Installing timer to purge stale objects after 1440 minutes.
2024-11-09 20:21:46.418 - Table output.0 - Connected to database [Conn] (commit=1000)
2024-11-09 20:21:46.490 - Table input.0 - Finished reading query, closing connection
2024-11-09 20:21:46.493 - Table input.0 - Finished processing (I=2, O=0, R=0, W=2, U=0, E=0)
2024-11-09 20:21:46.619 - Table output.0 - Finished processing (I=0, O=2, R=2, W=2, U=0, E=0)
2024-11-09 20:21:46.622 - Spoon - The transformation has finished!
Search
ENG
IN
08:21 PM
09-11-2024

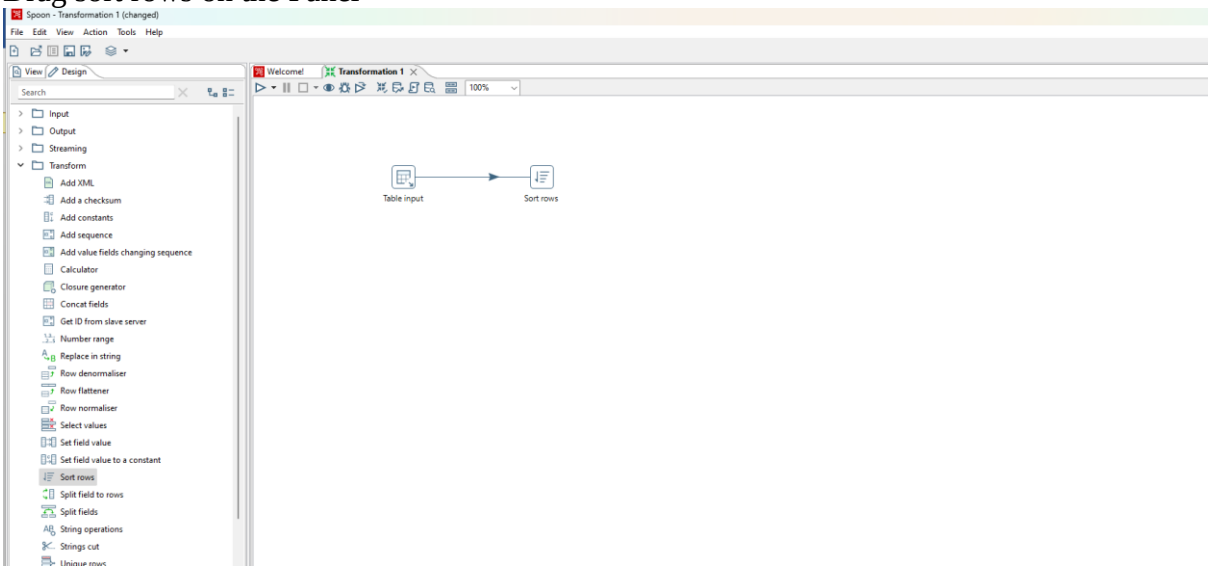
2. Sorting Operation and adding Sequence to Output Table.

Perform the first 5 steps same as 1.

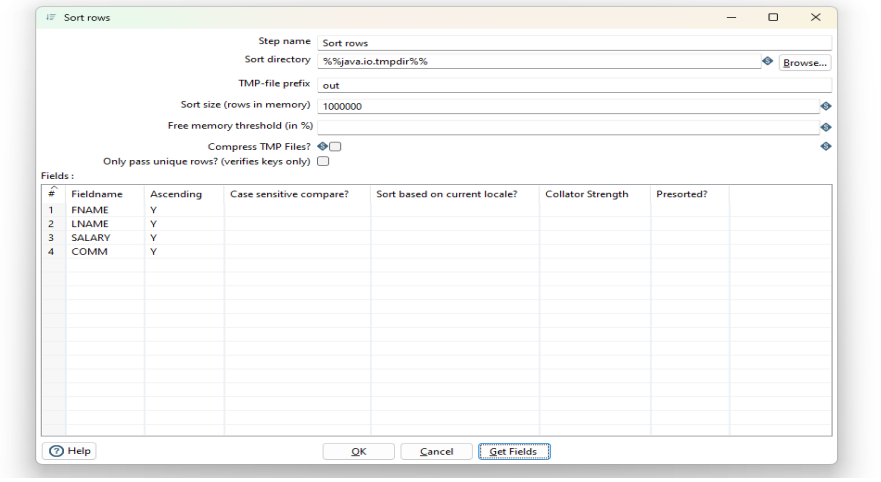


Click on Transform in the left

Drag sort rows on the Panel

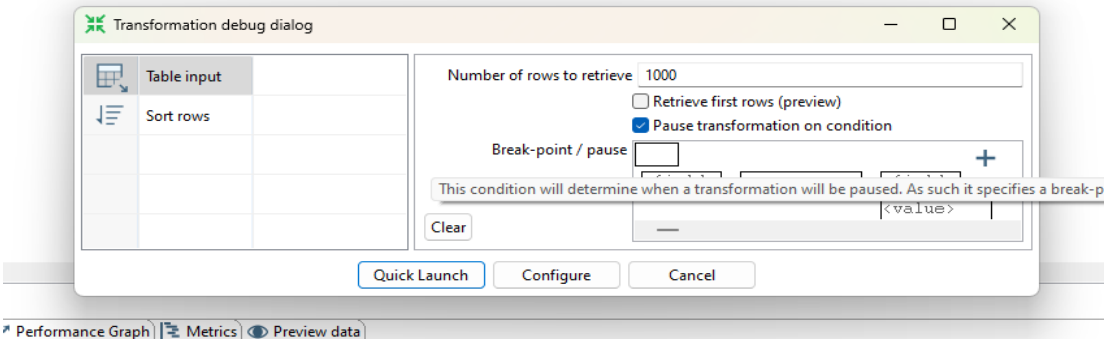


Double click on the sort rows
Edit the fields and press ok

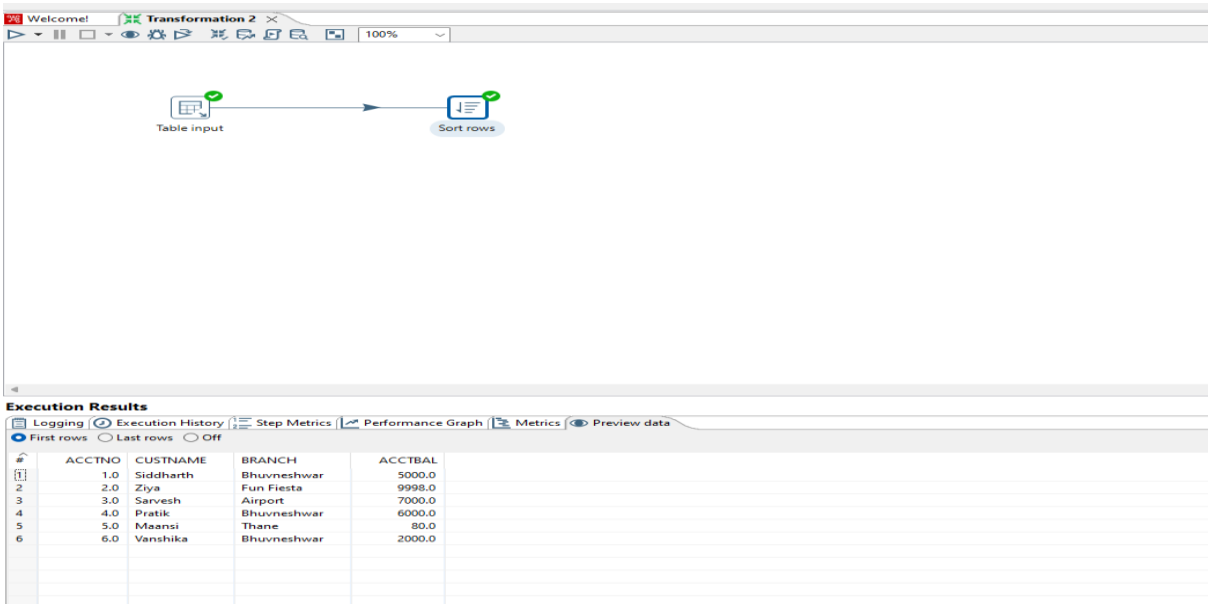


\

Click on Debug the Transformation and Click on Quick Launch.

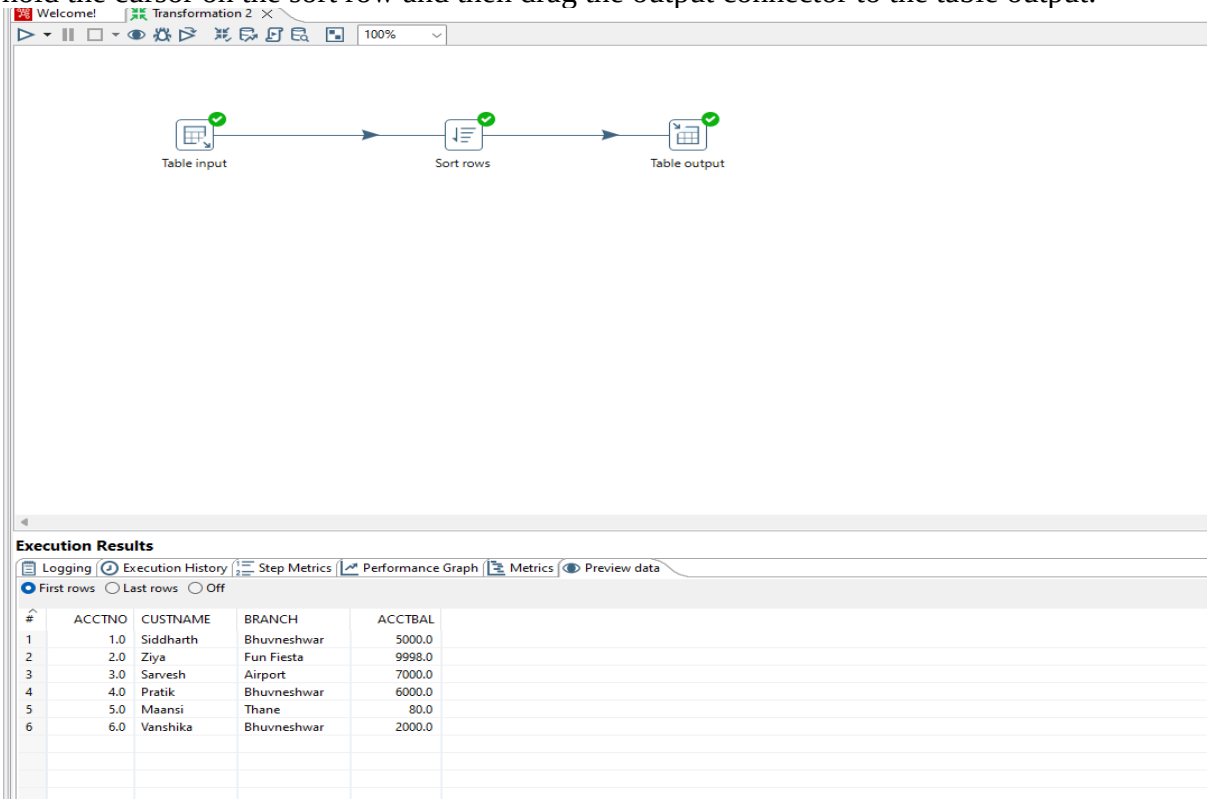


Successful connections(Green Ticks)



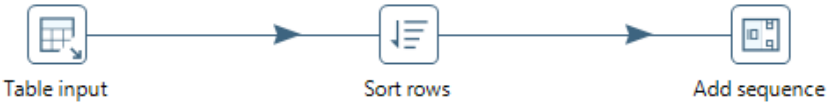
	ACCTNO	CUSTNAME	BRANCH	ACCTBAL
1	1.0	Siddharth	Bhuvneshwar	5000.0
2	2.0	Ziya	Fun Fiesta	9998.0
3	3.0	Sarvesh	Airport	7000.0
4	4.0	Pratik	Bhuvneshwar	6000.0
5	5.0	Maansi	Thane	80.0
6	6.0	Vanshika	Bhuvneshwar	2000.0

hold the cursor on the sort row and then drag the output connector to the table output.

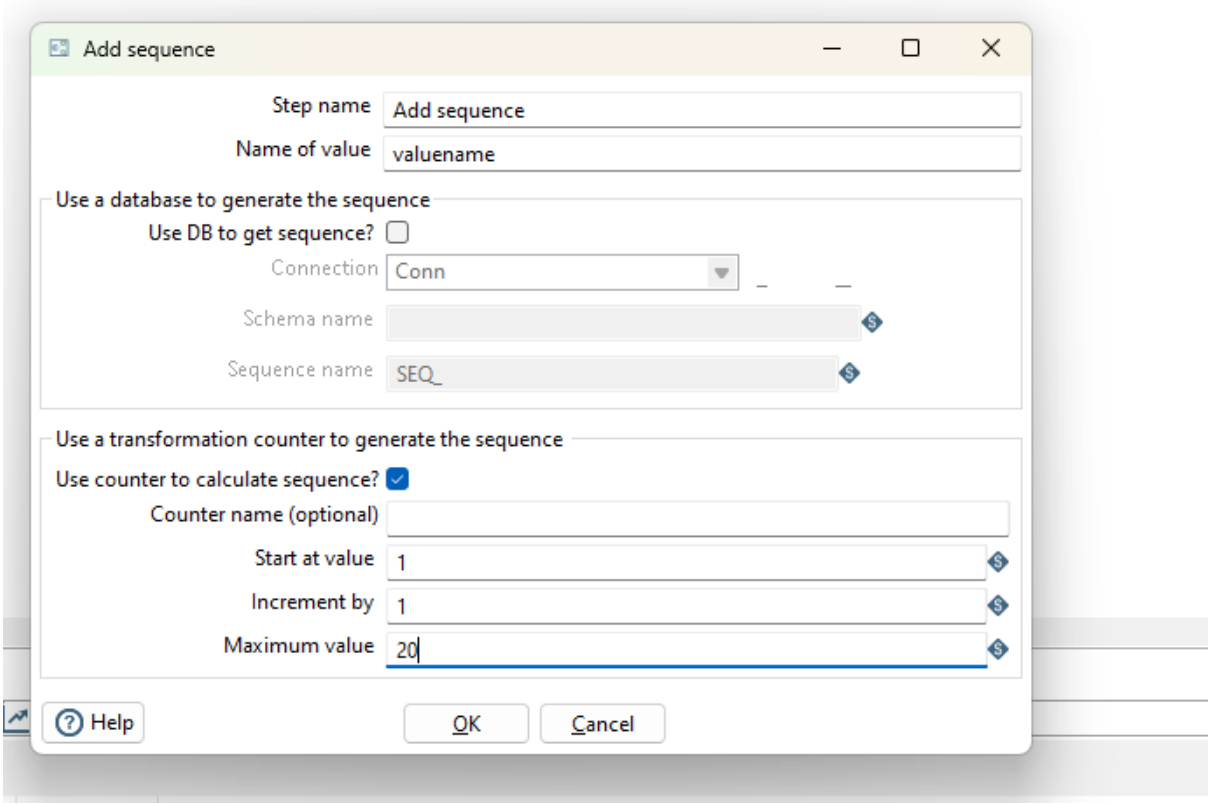


#	ACCTNO	CUSTNAME	BRANCH	ACCTBAL
1	1.0	Siddharth	Bhuvneshwar	5000.0
2	2.0	Ziya	Fun Fiesta	9998.0
3	3.0	Sarvesh	Airport	7000.0
4	4.0	Pratik	Bhuvneshwar	6000.0
5	5.0	Maansi	Thane	80.0
6	6.0	Vanshika	Bhuvneshwar	2000.0

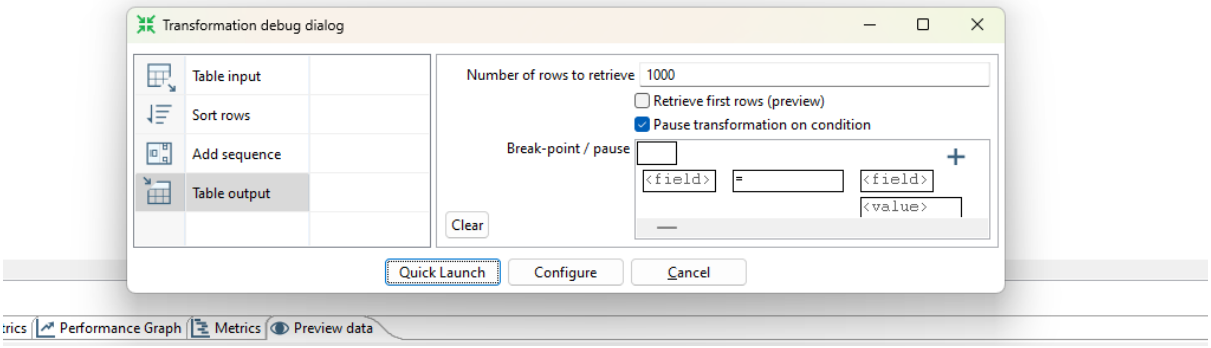
Drag and drop Add sequence on the panel



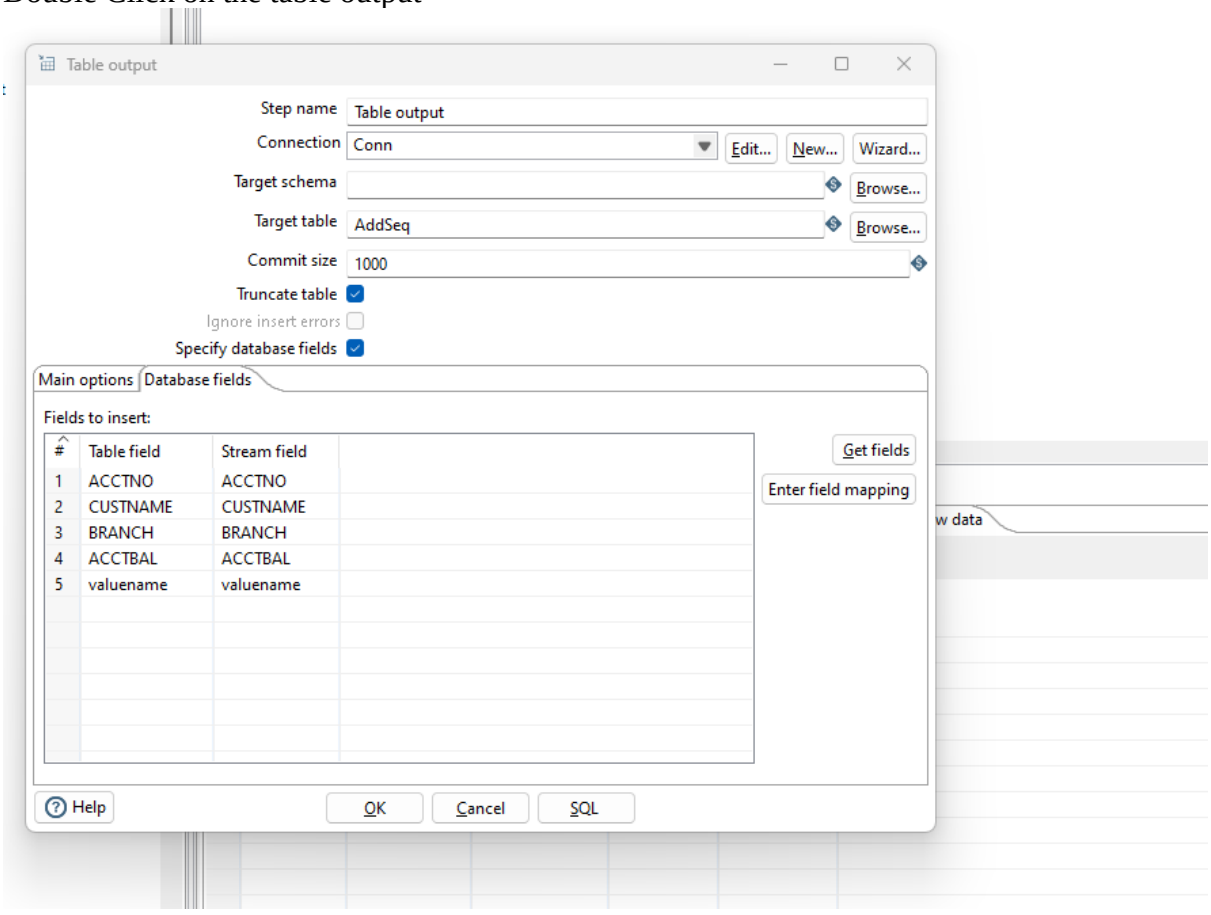
Double Click on Add Sequence Control
Enter the values for Start at Values,Increment by and Maximum Values and press ok.

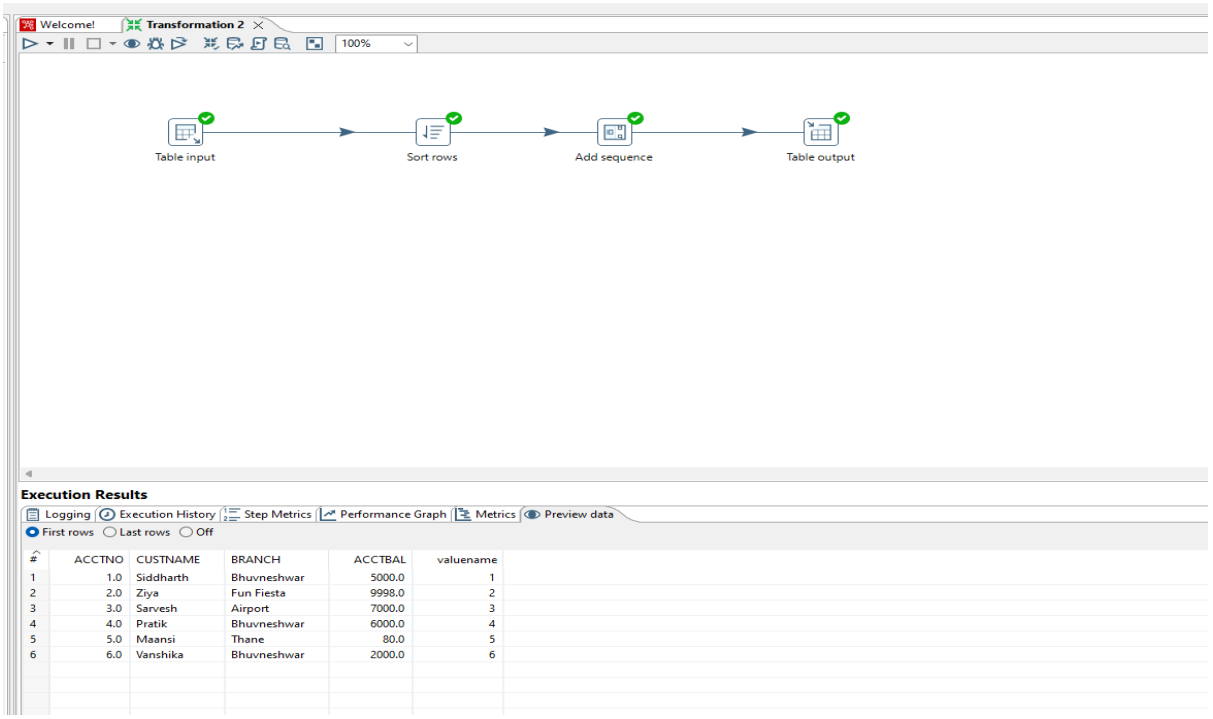


Click on Debug the Transformation and click on Quick Launch



Drag and drop the table output on the panel
Double Click on the table output





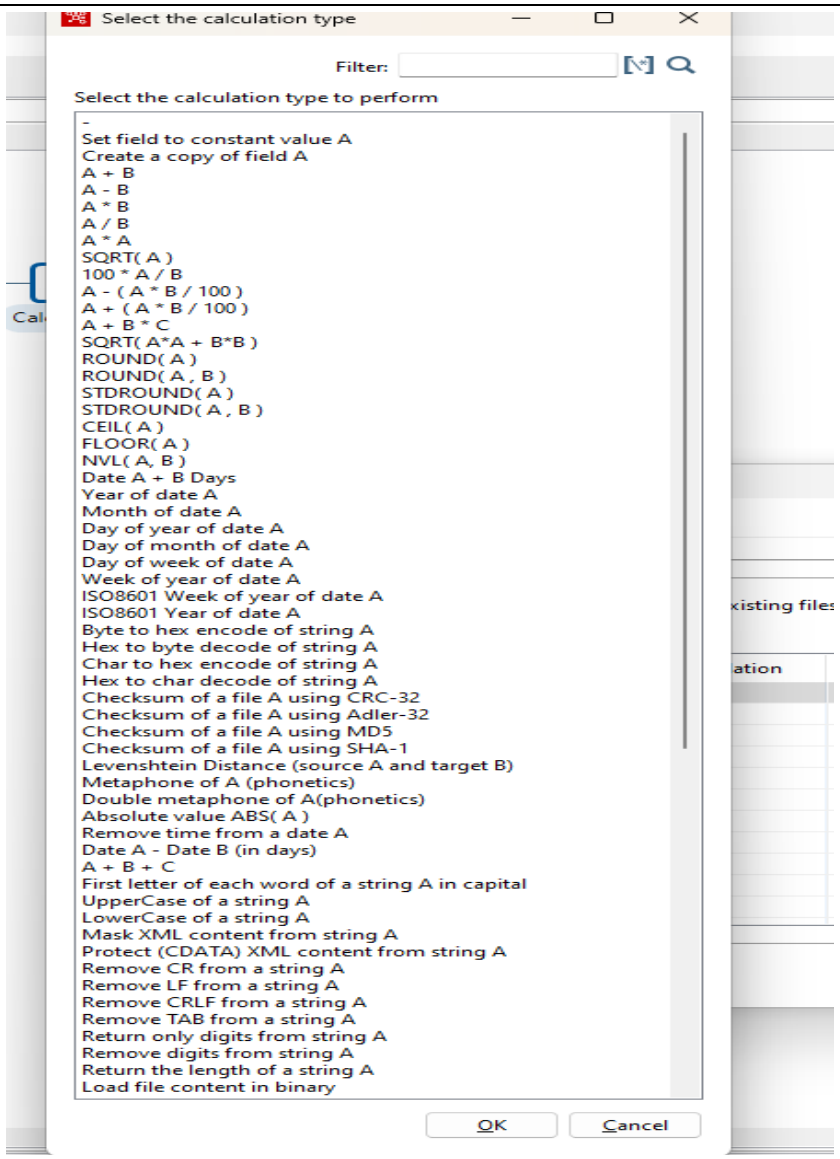

```
SQL> select * from AddSeq;

    ACCTNO CUSTNAME                  BRANCH                    ACCTBAL  VALUENAME
---------- ------------------------- ------------------------- ---------- ----------
         1 Siddharth                 Bhuvneshwar                 5000          1
         2 Ziya                      Fun Fiesta                  9998          2
         3 Sarvesh                   Airport                     7000          3
         4 Pratik                    Bhuvneshwar                 6000          4
         5 Maansi                    Thane                         80          5
         6 Vanshika                  Bhuvneshwar                 2000          6

6 rows selected.

SQL>
```

3. Calculator Operation

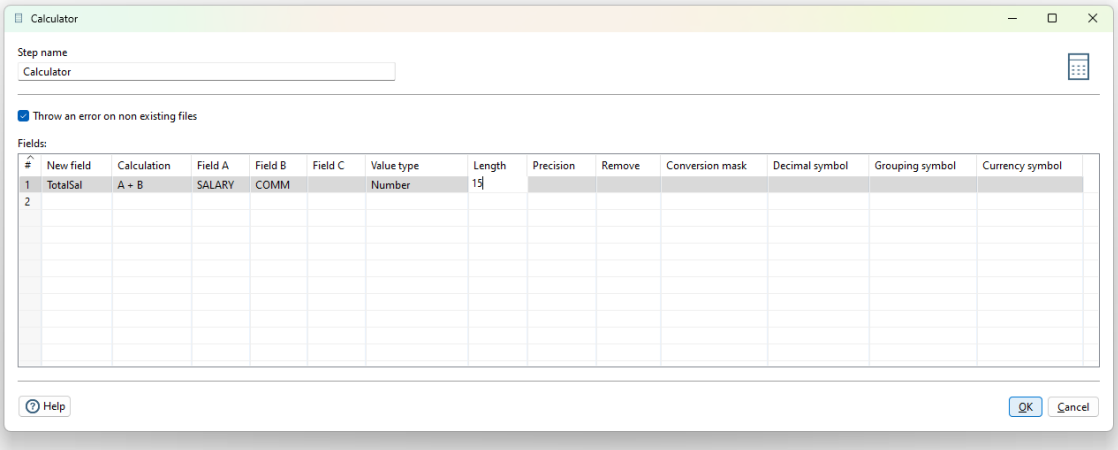
Step 1: Perform first 5 steps same as practical 1.

Drag and Drop Calculator from transformation on the panel.

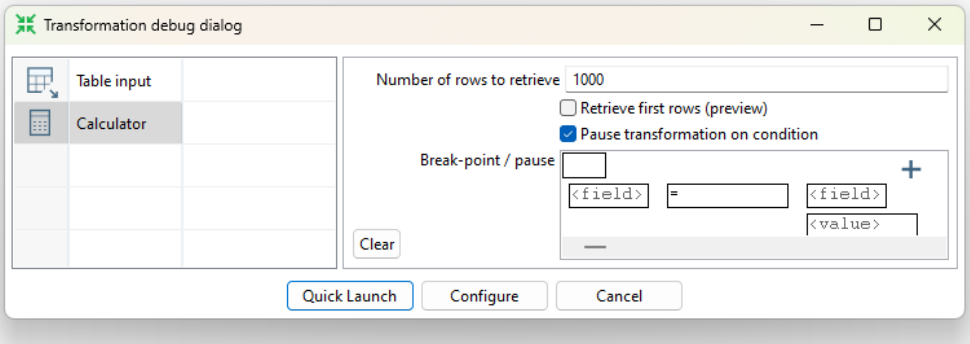
Double Click on the Calculator
a. specify the new field name
b. select the calculation function


Select the calculation type
Filter:
Select the calculation type to perform
-
Set field to constant value A
Create a copy of field A
A + B
A - B
A * B
A / B
A * A
SQRT(A)
100 * A / B
A - (A * B / 100)
A + (A * B / 100)
A + B * C
SQRT(A*A + B*B)
ROUND(A)
ROUND(A , B)
STDROUND(A)
STDROUND(A , B)
CEIL(A)
FLOOR(A)
NVL(A, B)
Date A + B Days
Year of date A
Month of date A
Day of year of date A
Day of month of date A
Day of week of date A
Week of year of date A
ISO8601 Week of year of date A
ISO8601 Year of date A
Byte to hex encode of string A
Hex to byte decode of string A
Char to hex encode of string A
Hex to char decode of string A
Checksum of a file A using CRC-32
Checksum of a file A using Adler-32
Checksum of a file A using MD5
Checksum of a file A using SHA-1
Levenshtein Distance (source A and target B)
Metaphone of A (phonetics)
Double metaphone of A(phonetics)
Absolute value ABS(A)
Remove time from a date A
Date A - Date B (in days)
A + B + C
First letter of each word of a string A in capital
UpperCase of a string A
LowerCase of a string A
Mask XML content from string A
Protect (CDATA) XML content from string A
Remove CR from a string A
Remove LF from a string A
Remove CRLF from a string A
Remove TAB from a string A
Return only digits from string A
Remove digits from string A
Return the length of a string A
Load file content in binary
OK
Cancel

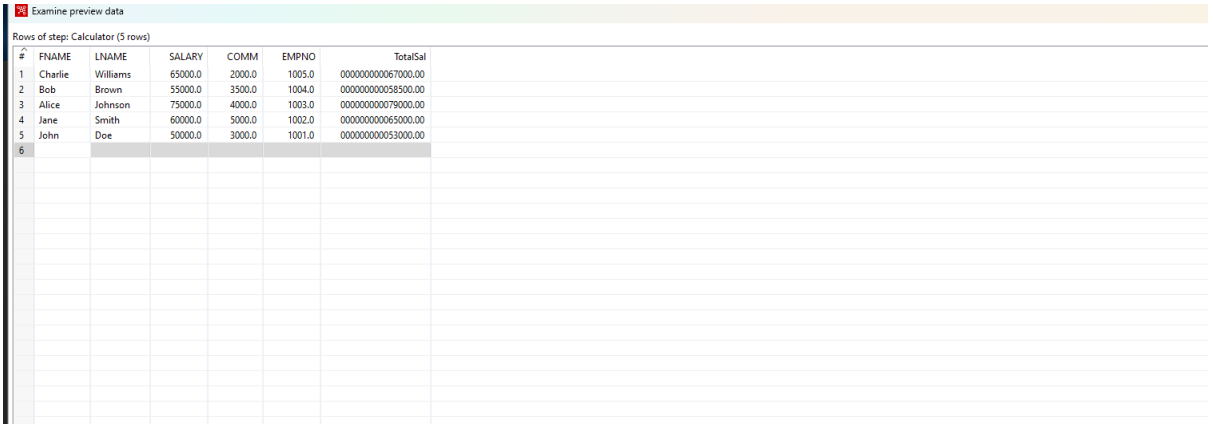
c. Enter Field A, Field B, Value type and Length and press ok.



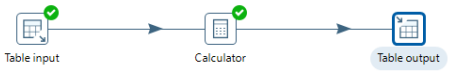
Click on Debug transformation and click on Quick Launch.

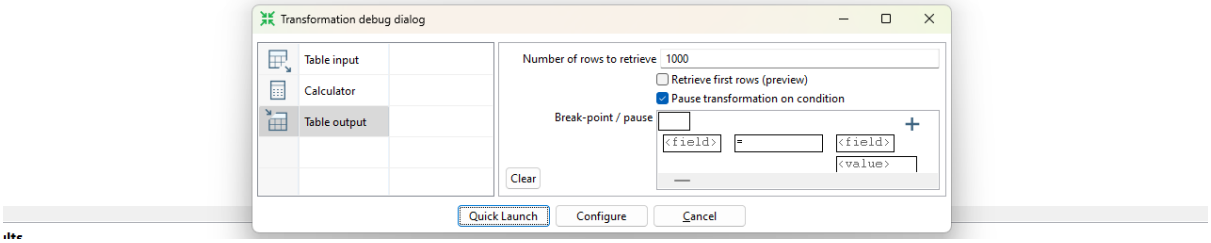


Preview

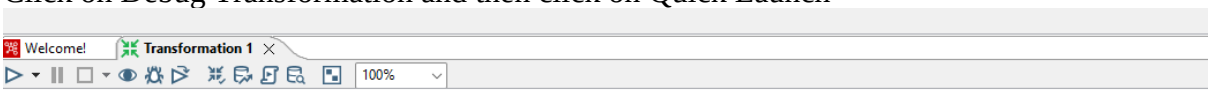


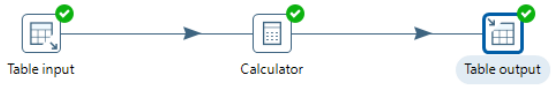
Now Perform the steps 6-10 same as practical 1.





Click on Debug Transformation and then click on Quick Launch





4. Concatenation Operation

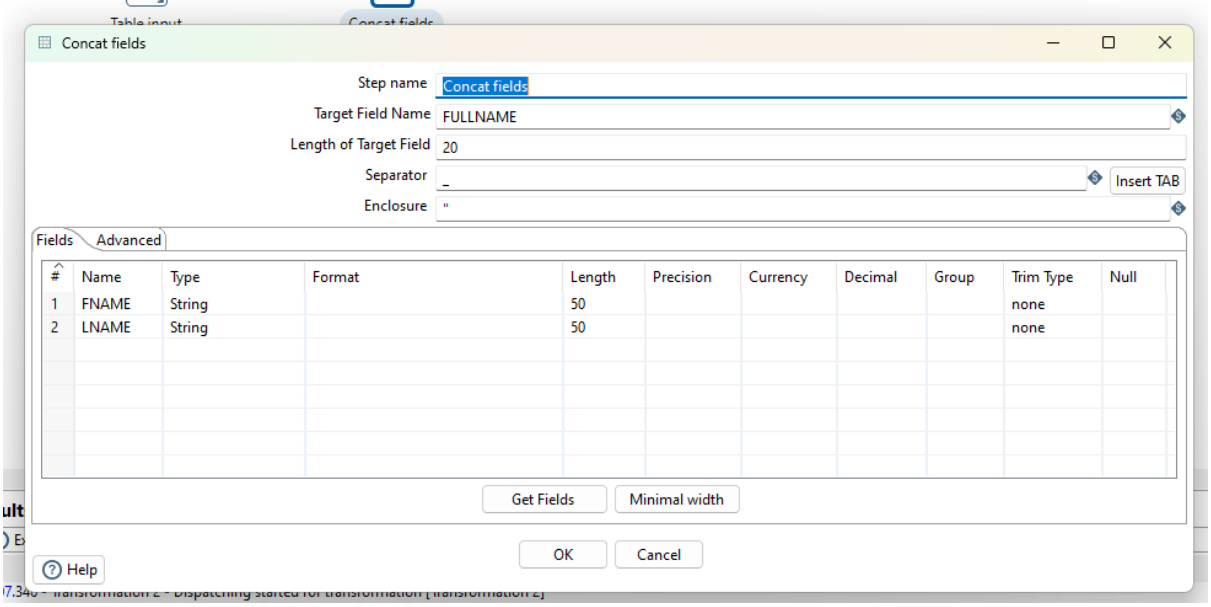
Perform first 6 steps same as practical 1.

Drag and drop Concat Field

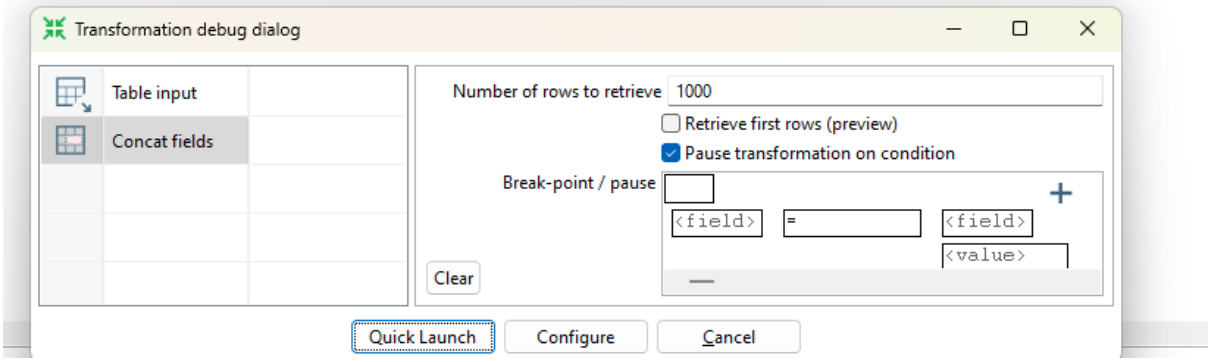
Double Click on Concat Field

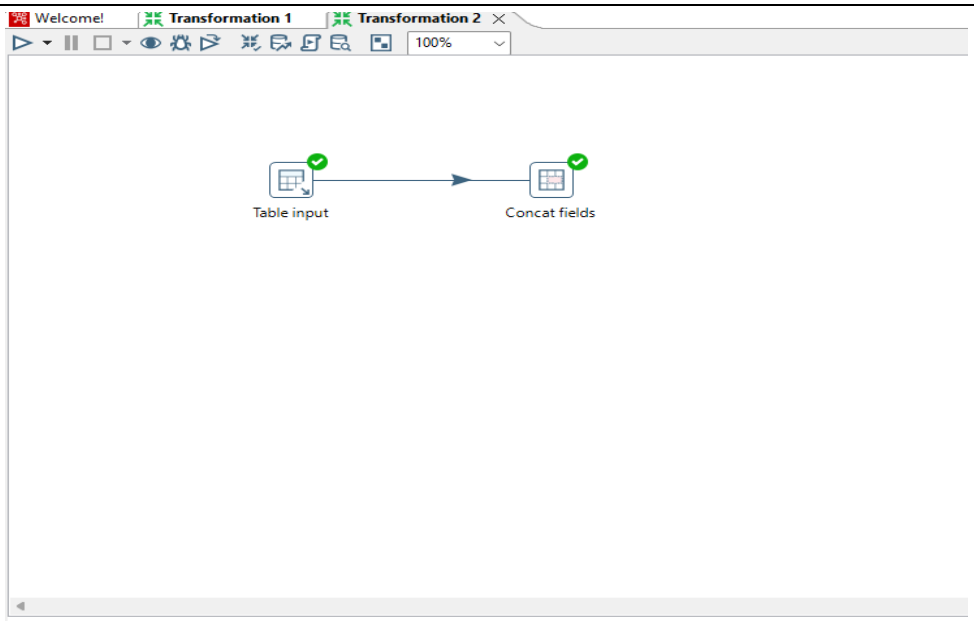
a. Enter the values for Target Field Name, Length of the Target Field and the Seperator and press on Get Fields

b. Select the columns that you want to concat



Click on Debug transformation and click on Quick Launch





Execution Results

Logging | Execution History | Step Metrics | Performance Graph | Metrics | Preview data

First rows | Last rows | Off

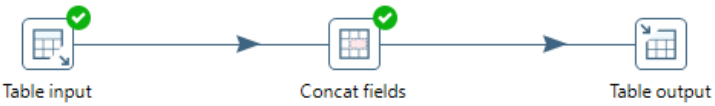
#	FNAME	LNAME	SALARY	COMM	EMPNO	FULLNAME	
1	John	Doe	50000.0	3000.0	1001.0	John	_Doe
2	Jane	Smith	60000.0	5000.0	1002.0	Jane	_Smith
3	Alice	Johnson	75000.0	4000.0	1003.0	Alice	_Johnson
4	Bob	Brown	55000.0	3500.0	1004.0	Bob	_Brown
5	Charlie	Williams	65000.0	2000.0	1005.0	Charlie	_Williams

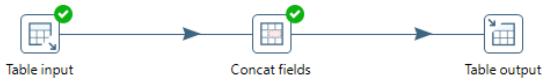
Preview

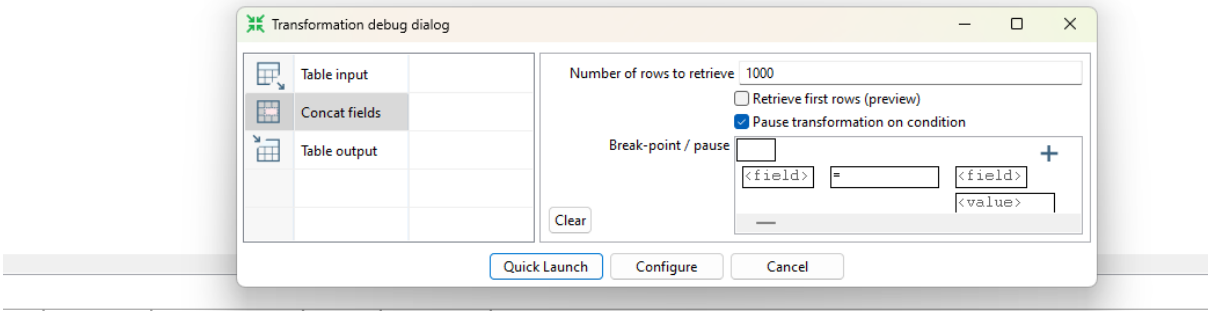
Examine preview data

ows of step: Concat fields (5 rows)

#	FNAME	LNAME	SALARY	COMM	EMPNO	FULLNAME	
1	Charlie	Williams	65000.0	2000.0	1005.0	Charlie	_Williams
2	Bob	Brown	55000.0	3500.0	1004.0	Bob	_Brown
3	Alice	Johnson	75000.0	4000.0	1003.0	Alice	_Johnson
4	Jane	Smith	60000.0	5000.0	1002.0	Jane	_Smith
5	John	Doe	50000.0	3000.0	1001.0	John	_Doe



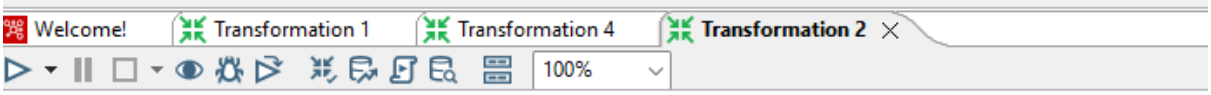




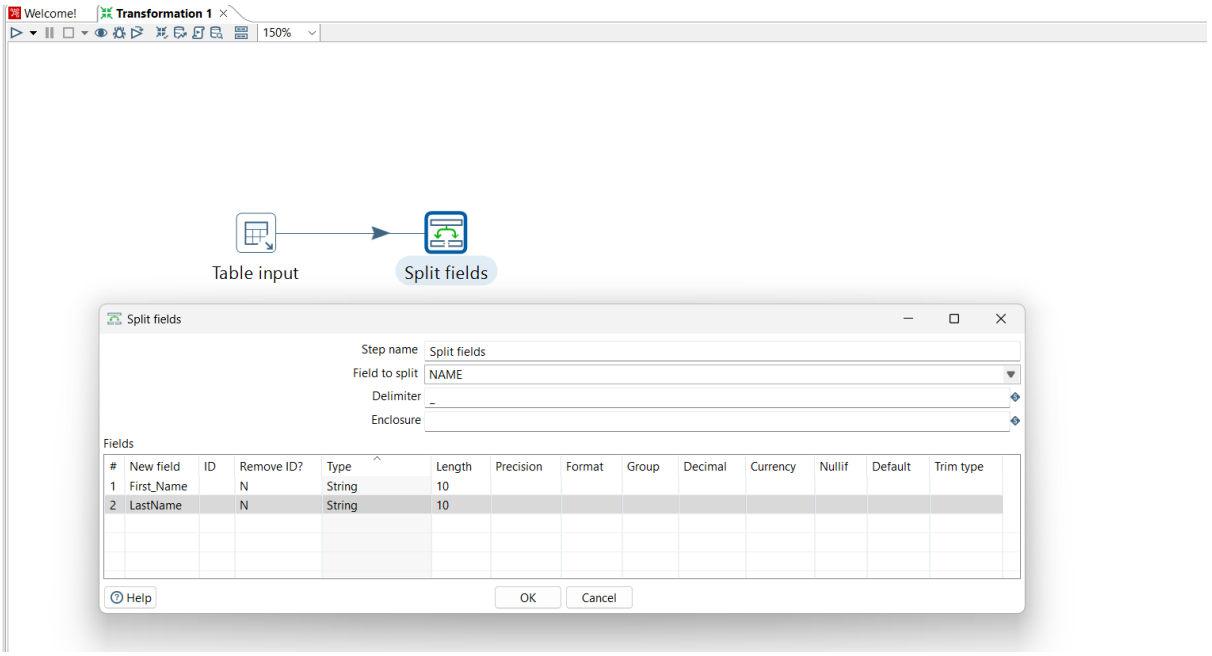
5. Splitting Operation

Perform first 6 steps same as practical 1.

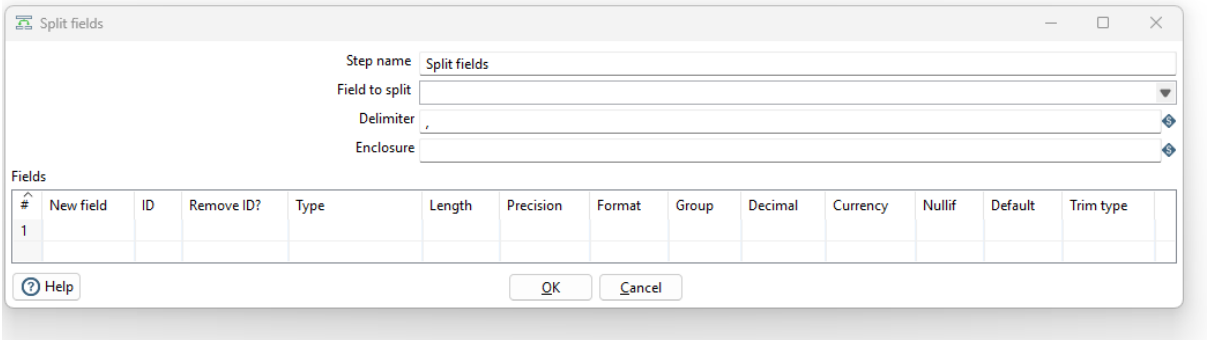
Drag and drop Split Field on the Panel



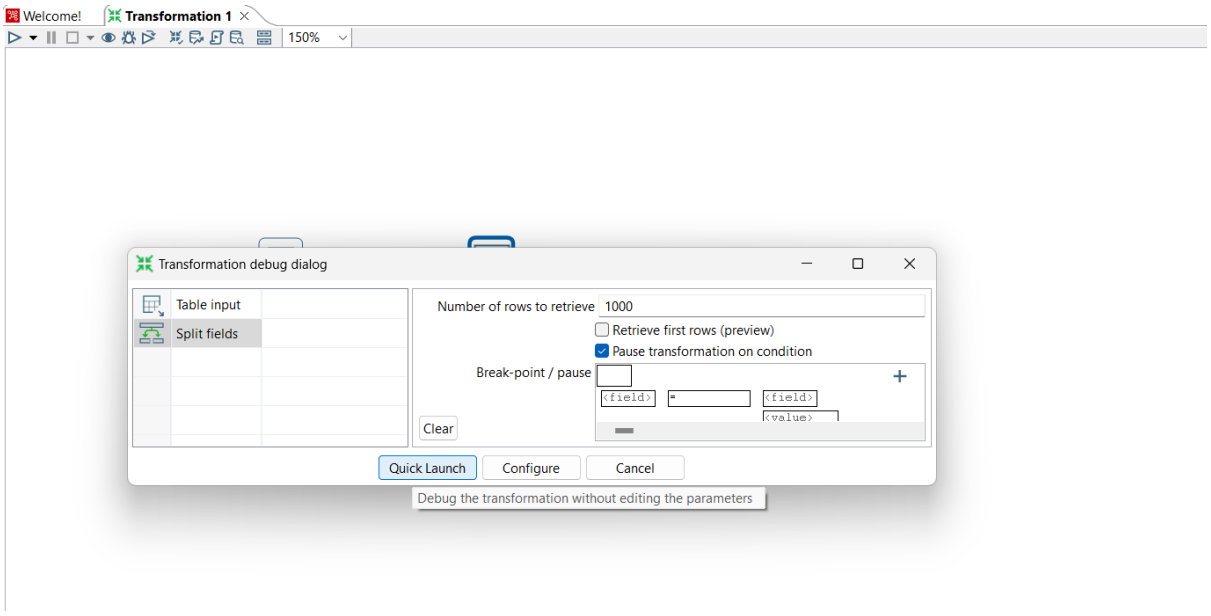
4: Double Click On Split Field.

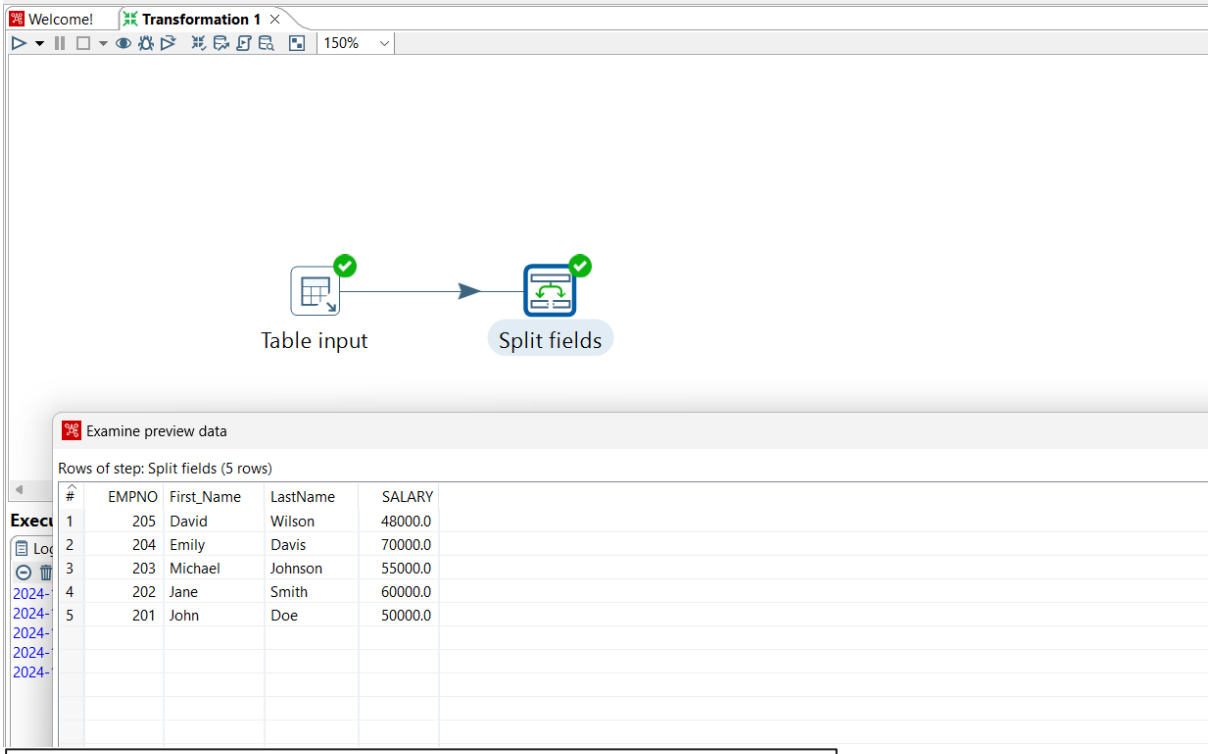


a. Enter the values For Field to Split, Delimiter and the field names and press ok



Click on Debug transformation and Click on Quick Launch.


Welcome!
Transformation 1
150%
Transformation debug dialog
Table input
Split fields
Number of rows to retrieve
1000
Retrieve first rows (preview)
Pause transformation on condition
Break-point / pause
Clear
Quick Launch
Configure
Cancel
Debug the transformation without editing the parameters


Welcome!
Transformation 1
150%
Table input
Split fields
Examine preview data
Rows of step: Split fields (5 rows)
EMPNO First_Name LastName SALARY
1 205 David Wilson 48000.0
2 204 Emily Davis 70000.0
3 203 Michael Johnson 55000.0
4 202 Jane Smith 60000.0
5 201 John Doe 50000.0

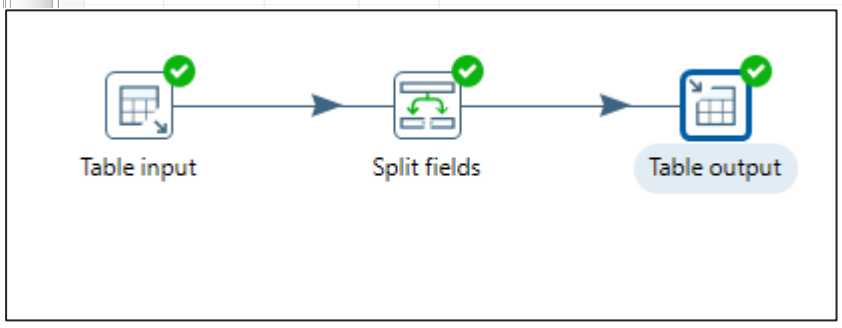


Table input
Split fields
Table output

```
SQL> select * from EMPLOYEE_DETAILS;

     EMPNO NAME                                                                                                SALARY
---------- ---------------------------------------------------------------------------------------------------------
       201 John Doe                                                                                           50000
       202 Jane Smith                                                                                         60000
       203 Michael Johnson                                                                                    55000
       204 Emily Davis                                                                                        70000
       205 David Wilson                                                                                       48000

SQL> select * from SplitTable;

     EMPNO FIRST_NAME LASTNAME       SALARY
---------- ---------- ---------- ----------
       201 John       Doe             50000
       202 Jane       Smith           60000
       203 Michael    Johnson         55000
       204 Emily      Davis           70000
       205 David      Wilson          48000

SQL> |
```

6. Number Range:

Create Student table in SQL

SQL> select * from student ;

ROLL_NO PERCENTAGE

---------- ----------

101 55

102 59

103 68

104 75

105 83

106 35

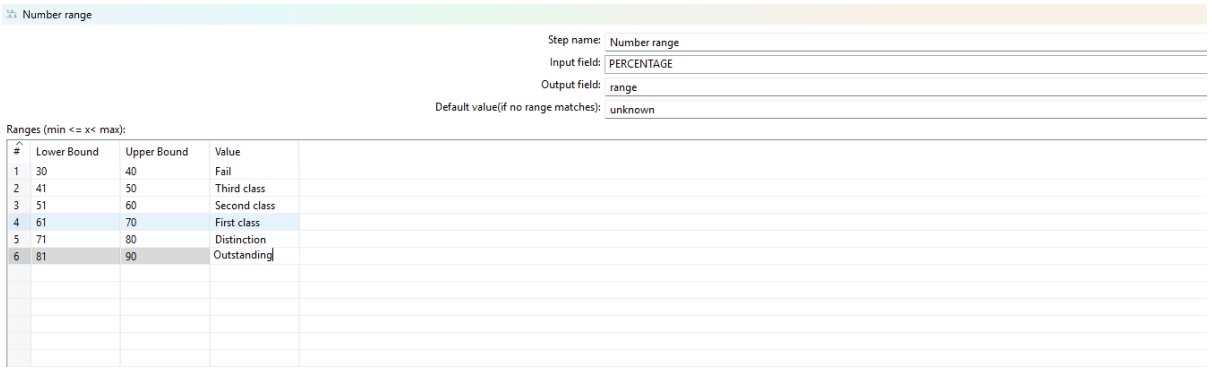
107 88

Perform first 6 steps same as practical 1.

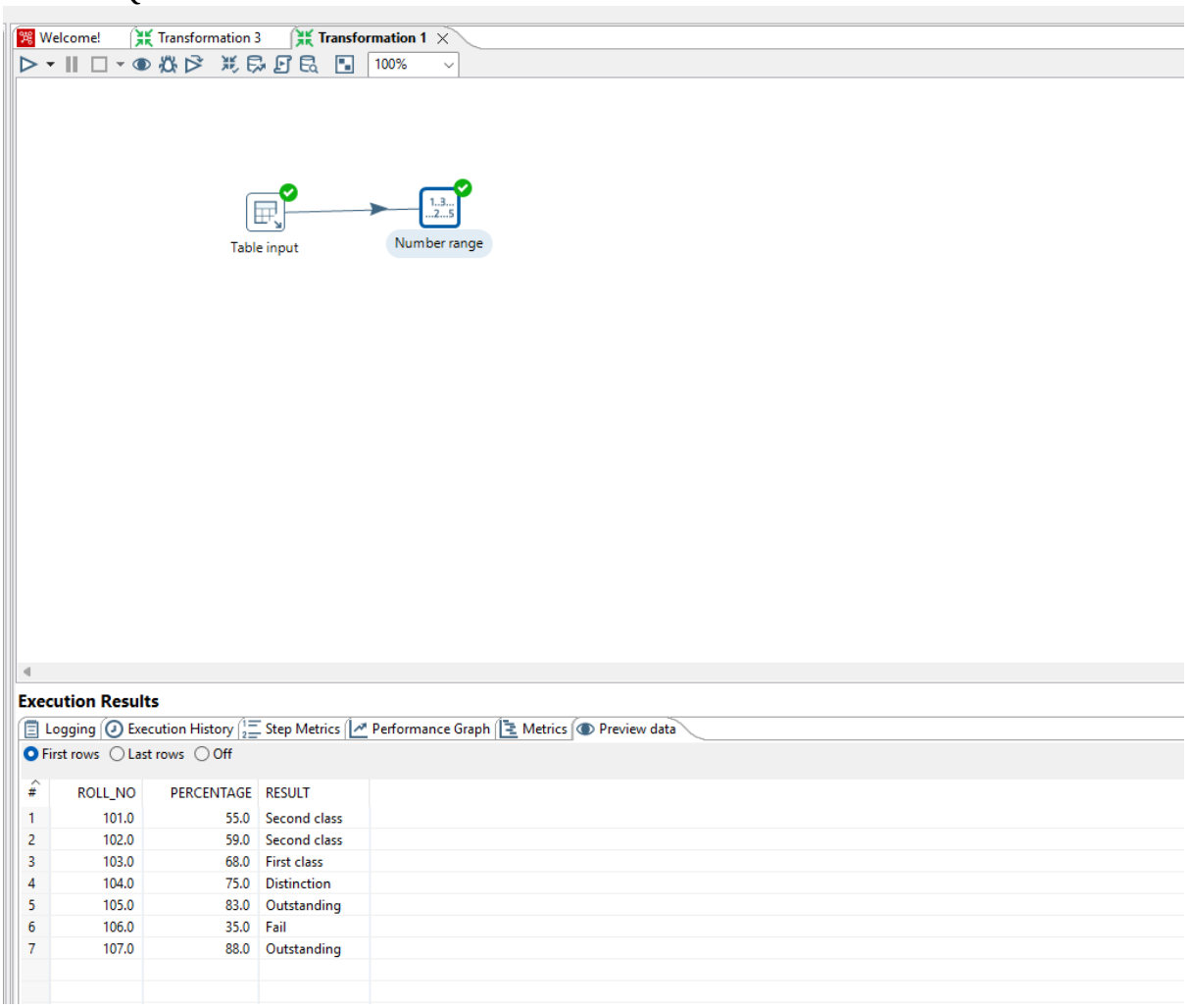
Table input

Drag and drop Number Range transformation and double click on it.

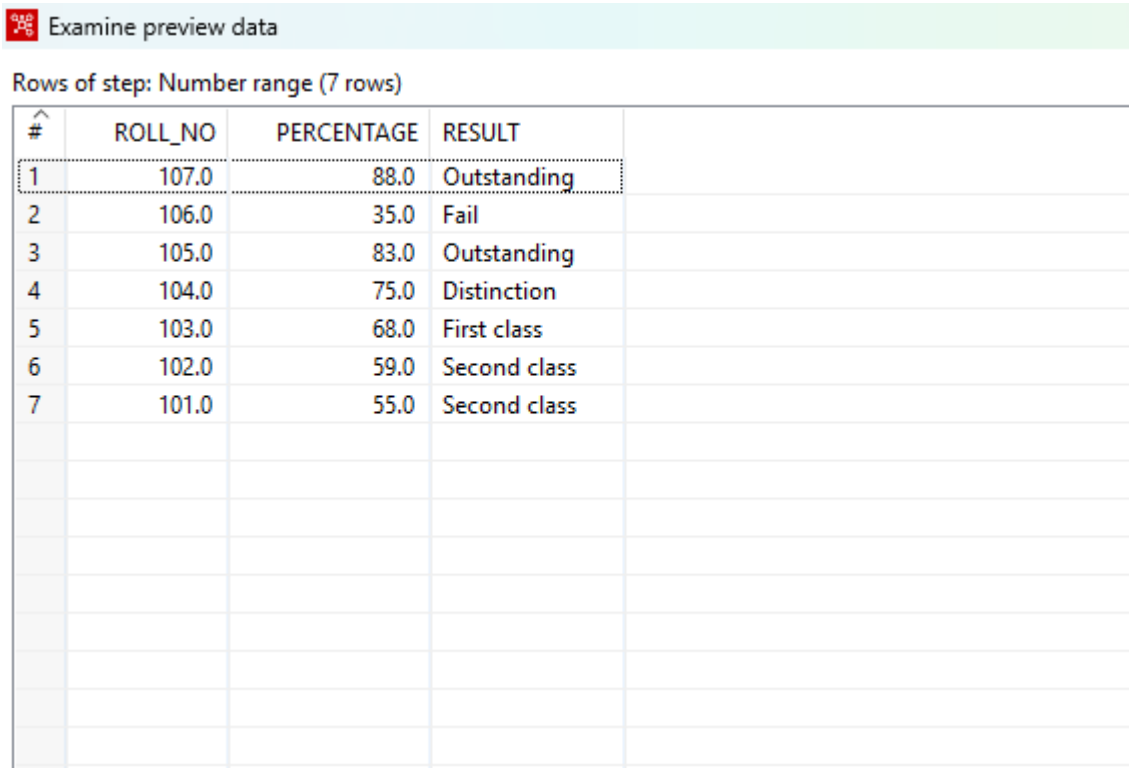
Set values for Input field, Output field, Lower bound, Upper bound and value according to the values specified in percentage column.



Click on Quick launch



Click on Preview data



#	ROLL_NO	PERCENTAGE	RESULT
1	107.0	88.0	Outstanding
2	106.0	35.0	Fail
3	105.0	83.0	Outstanding
4	104.0	75.0	Distinction
5	103.0	68.0	First class
6	102.0	59.0	Second class
7	101.0	55.0	Second class

7.String Operations

Required SQL table:

SQL> select * from newemp;

EMPN ENAME DEPT JOB SALARY

---- -------------------- ---- ---------- ----------

E001 John D002 Manager 50000

E002 Smit D001 Hr 40000

E003 steven_king D002 30000

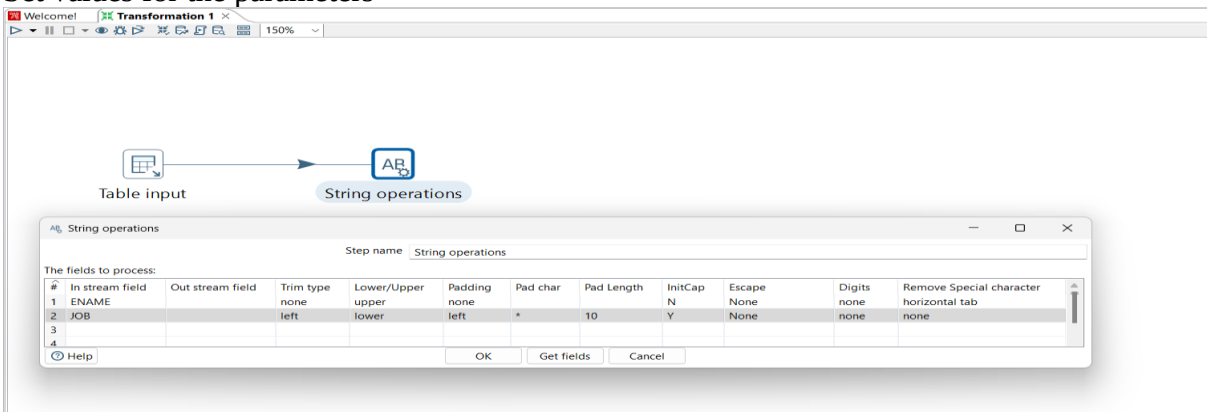
E004 Lex Hean D003 Manager 14000

E005 Bruce D003 Manager 17000

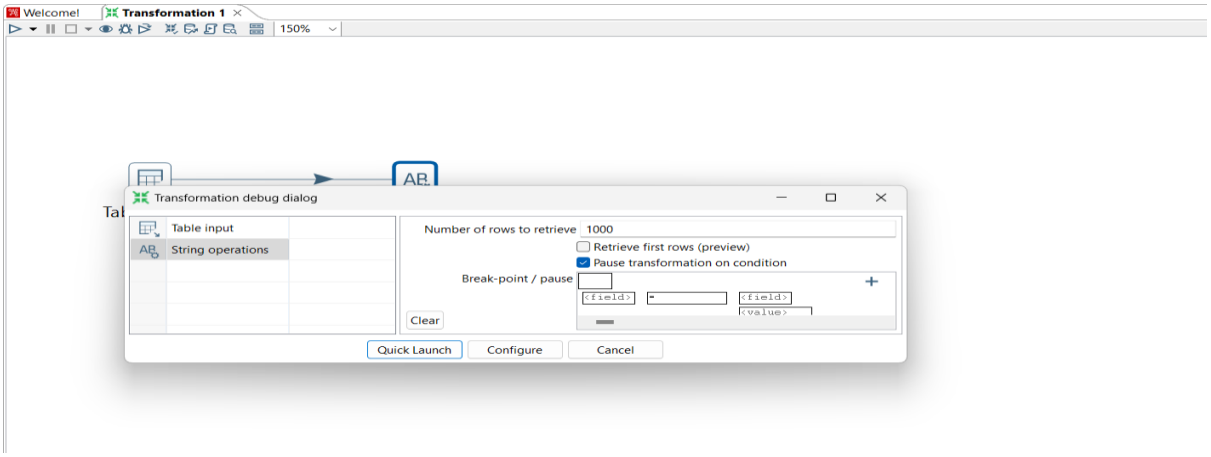
E006 Kevin#Nash D002 Sales 18000

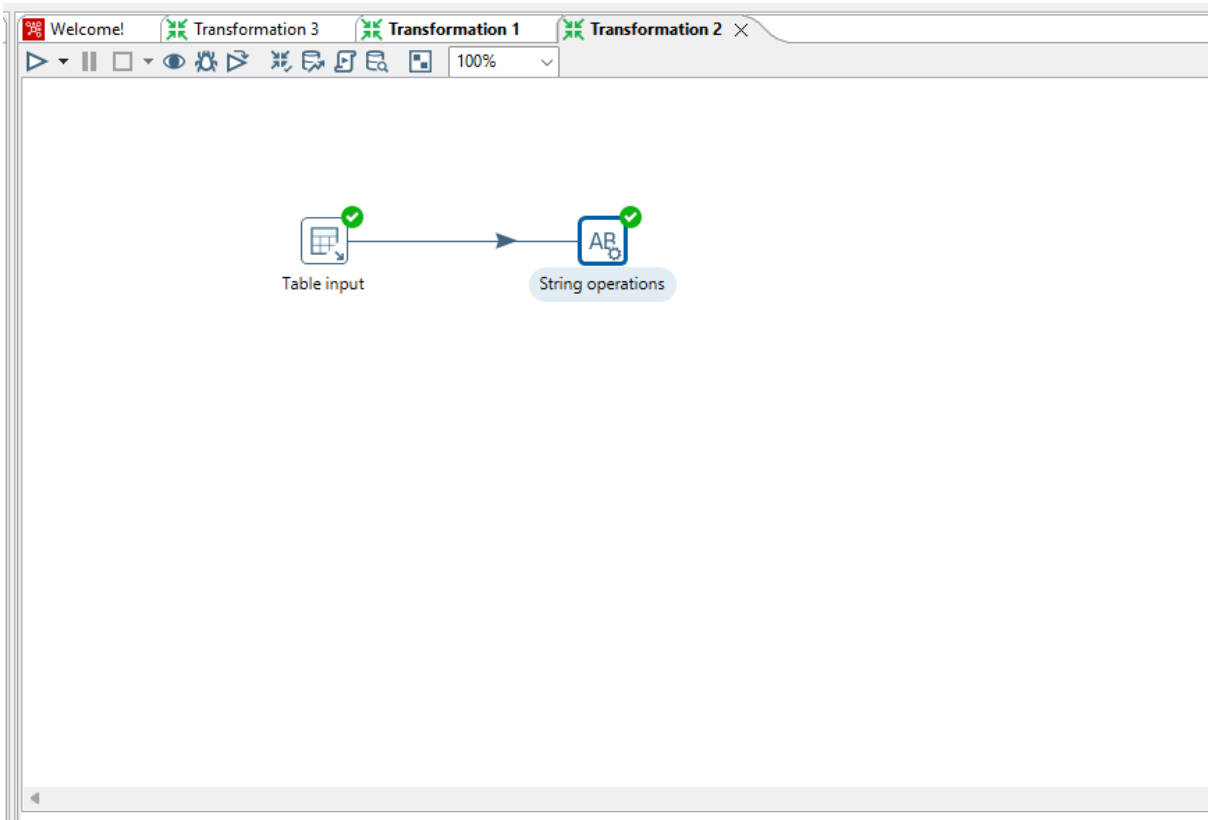
1: Perform first 6 steps same as practical 1.

Drag and drop String operation transformation, make connection between table input and string operation and double click on it.
Set values for the parameters



Launch the transformation





Execution Results

Logging | Execution History | Step Metrics | Performance Graph | Metrics | Preview data

First rows | Last rows | Off

#	EMPN	ENAME	DEPT	JOB	SALARY
1	E001	John	D002	Manager	50000.0
2	E002	Smit	D001	Hr	40000.0
3	E003	steven_king	D002	Sales	30000.0
4	E004	Lex Hean	D003	Manager	14000.0
5	E005	Bruce	D003	Manager	17000.0
6	E006	Kevin#Nash	D002	Sales	18000.0

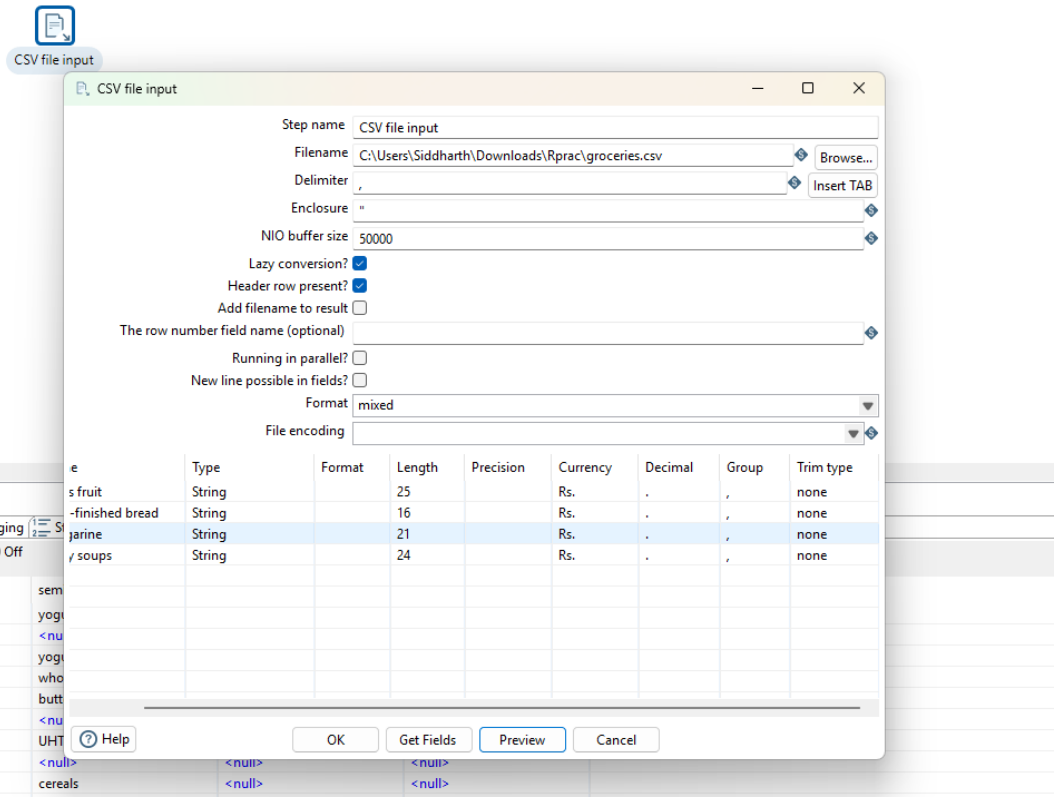
Examine preview data

Rows of step: String operations (6 rows)

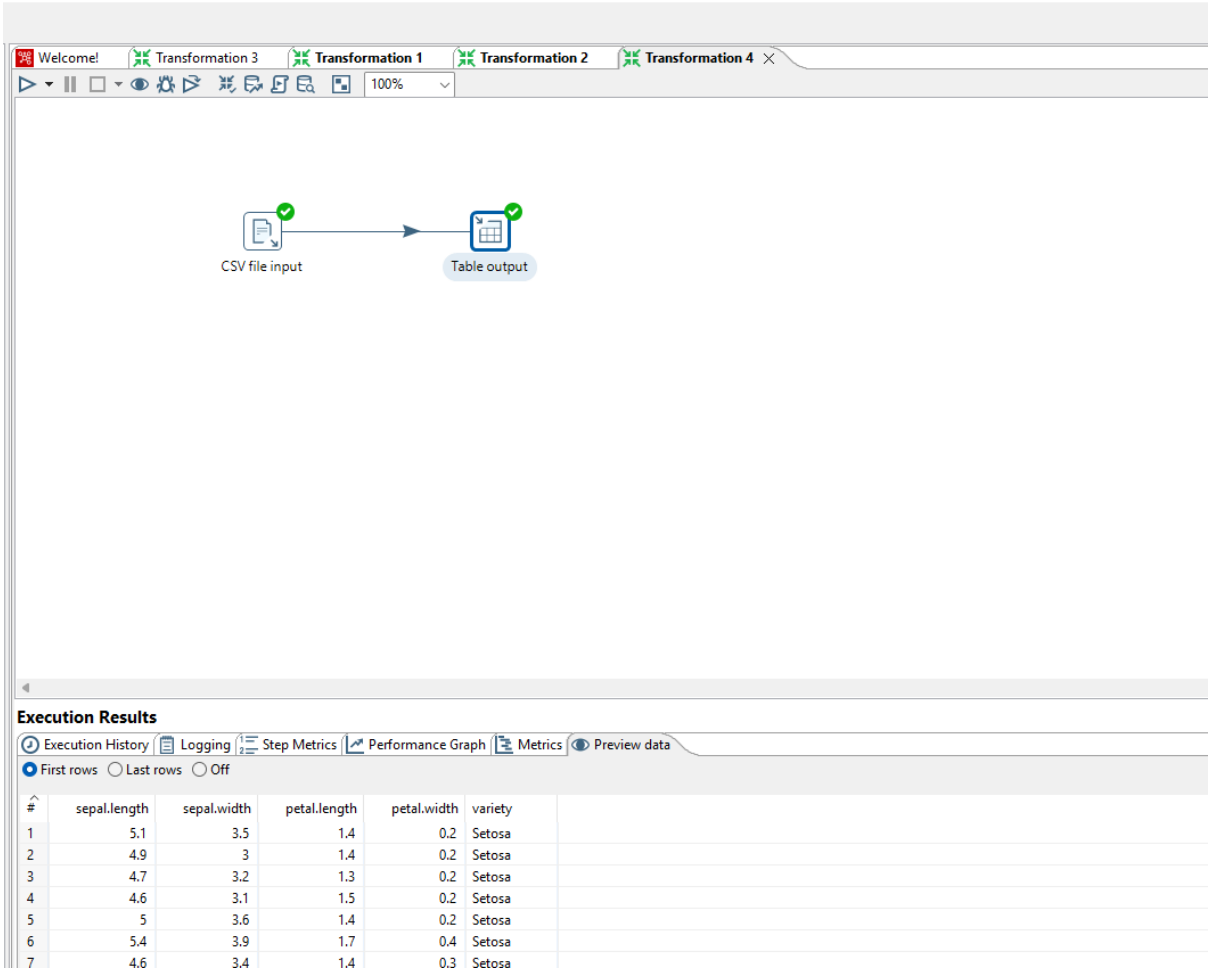
#	EMPN	ENAME	DEPT	JOB	SALARY
1	E006	Kevin#Nash	D002	Sales	18000.0
2	E005	Bruce	D003	Manager	17000.0
3	E004	Lex Hean	D003	Manager	14000.0
4	E003	steven_king	D002	Sales	30000.0
5	E002	Smit	D001	Hr	40000.0
6	E001	John	D002	Manager	50000.0

8. Copy contents of.CSV file to the target table
Drag and drop CSV file input on a panel

Double click on it and open respective CSV file, Lazy conversation and Header row present? Options should be checked, click on Getfield and then ok.



Drag and drop Table output and connect it with CSV file.



#	sepal.length	sepal.width	petal.length	petal.width	variety
1	5.1	3.5	1.4	0.2	Setosa
2	4.9	3	1.4	0.2	Setosa
3	4.7	3.2	1.3	0.2	Setosa
4	4.6	3.1	1.5	0.2	Setosa
5	5	3.6	1.4	0.2	Setosa
6	5.4	3.9	1.7	0.4	Setosa
7	4.6	3.4	1.4	0.3	Setosa

Examine preview data

Rows of step: Table output (150 rows)

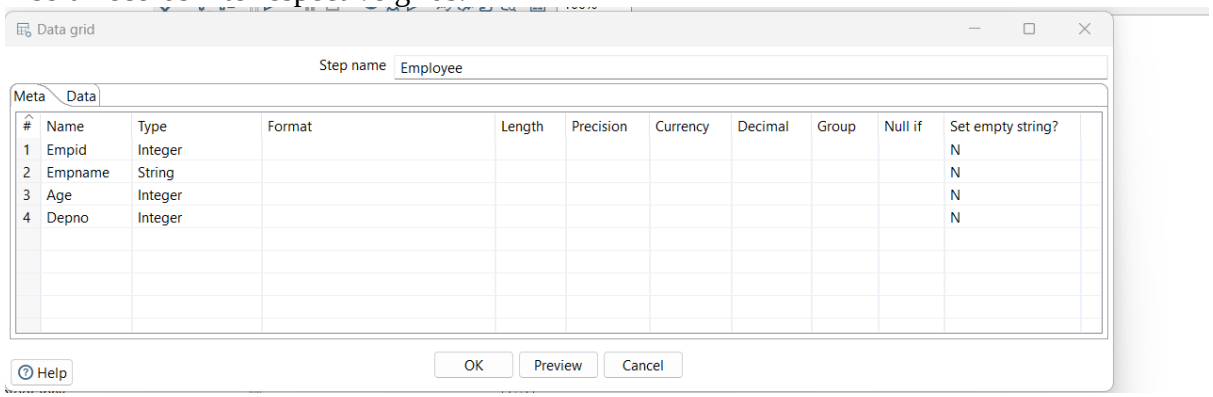
#	sepal.length	sepal.width	petal.length	petal.width	variety
1	5.9	3	5.1	1.8	Virginica
2	6.2	3.4	5.4	2.3	Virginica
3	6.5	3	5.2	2	Virginica
4	6.3	2.5	5	1.9	Virginica
5	6.7	3	5.2	2.3	Virginica
6	6.7	3.3	5.7	2.5	Virginica
7	6.8	3.2	5.9	2.3	Virginica
8	5.8	2.7	5.1	1.9	Virginica
9	6.9	3.1	5.1	2.3	Virginica
10	6.7	3.1	5.6	2.4	Virginica
11	6.9	3.1	5.4	2.1	Virginica
12	6	3	4.8	1.8	Virginica
13	6.4	3.1	5.5	1.8	Virginica
14	6.3	3.4	5.6	2.4	Virginica
15	7.7	3	6.1	2.3	Virginica
16	6.1	2.6	5.6	1.4	Virginica
17	6.3	2.8	5.1	1.5	Virginica
18	6.4	2.8	5.6	2.2	Virginica
19	7.9	3.8	6.4	2	Virginica
20	7.4	2.8	6.1	1.9	Virginica
21	7.2	3	5.8	1.6	Virginica
22	6.4	2.8	5.6	2.1	Virginica
23	6.1	3	4.9	1.8	Virginica
24	6.2	2.8	4.8	1.8	Virginica
25	7.2	3.2	6	1.8	Virginica
26	6.7	3.3	5.7	2.1	Virginica
27	6.3	2.7	4.9	1.8	Virginica
28	7.7	2.8	6.7	2	Virginica
29	5.6	2.8	4.9	2	Virginica
30	6.9	3.2	5.7	2.3	Virginica
31	6	2.2	5	1.5	Virginica
32	7.7	2.6	6.9	2.3	Virginica
33	7.7	3.8	6.7	2.2	Virginica
34	6.5	3	5.5	1.8	Virginica
35	6.4	3.2	5.3	2.3	Virginica
36	5.8	2.8	5.1	2.4	Virginica
37	5.7	2.5	5	2	Virginica
38	6.8	3	5.5	2.1	Virginica
39	6.4	2.7	5.3	1.9	Virginica
40	6.5	3.2	5.1	2	Virginica
41	7.2	3.6	6.1	2.5	Virginica
42	6.7	2.5	5.8	1.8	Virginica
43	7.3	2.9	6.3	1.8	Virginica
44	4.9	2.5	4.5	1.7	Virginica
45	7.6	3	6.6	2.1	Virginica

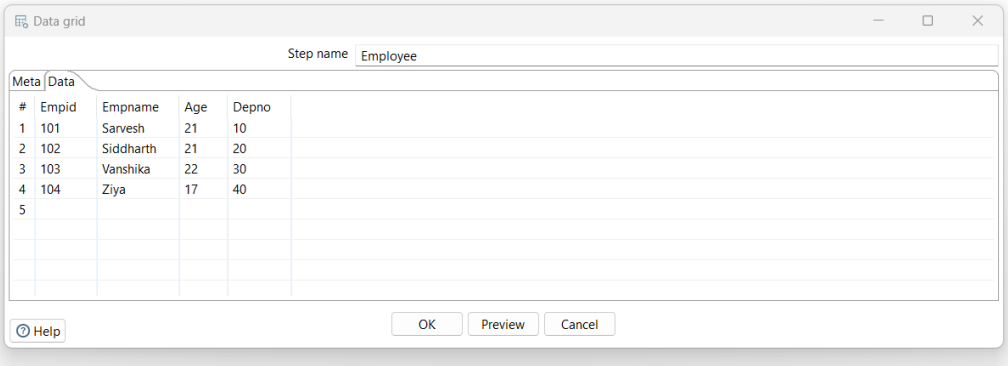
```
SQL> select * from Buy;

sepal.length sepal.width petal.length petal.width VARIETY
------------ ----------- ------------ ----------- ----------
         5.1         3.5          1.4          .2 Setosa
         4.9           3          1.4          .2 Setosa
         4.7         3.2          1.3          .2 Setosa
         4.6         3.1          1.5          .2 Setosa
           5         3.6          1.4          .2 Setosa
         5.4         3.9          1.7          .4 Setosa
         4.6         3.4          1.4          .3 Setosa
           5         3.4          1.5          .2 Setosa
         4.4         2.9          1.4          .2 Setosa
         4.9         3.1          1.5          .1 Setosa
         5.4         3.7          1.5          .2 Setosa
```

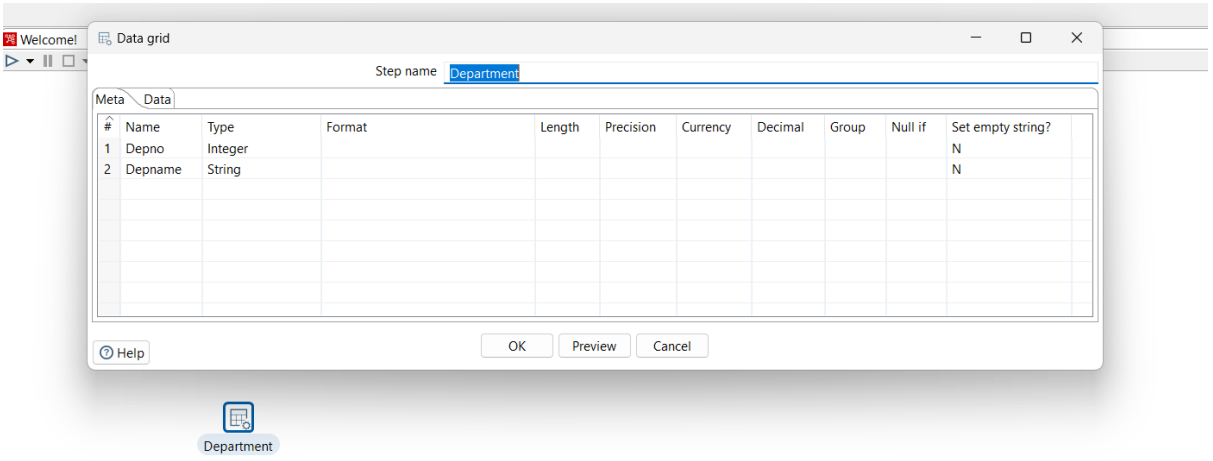
9. Implement the merge join transformation on tables

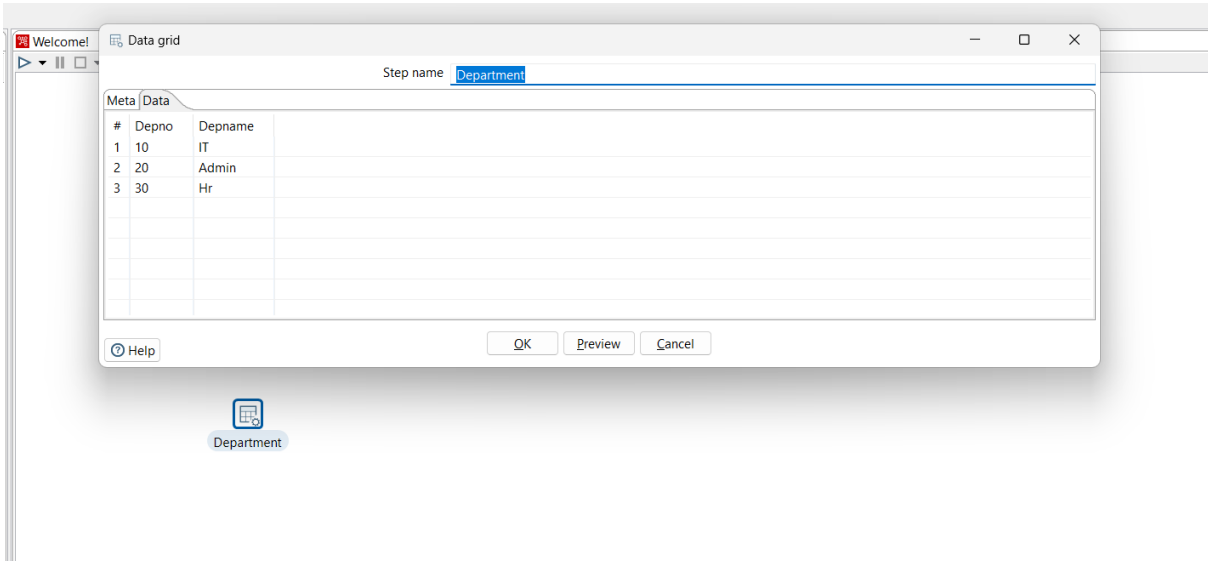
Drag two Data grid on panel rename them with Employee and Department respectively.

Insert Records into respective grids.

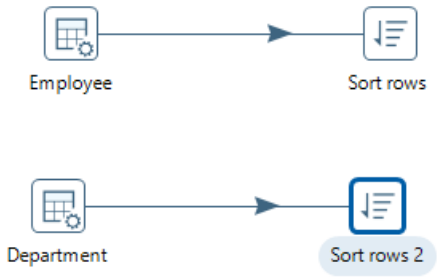


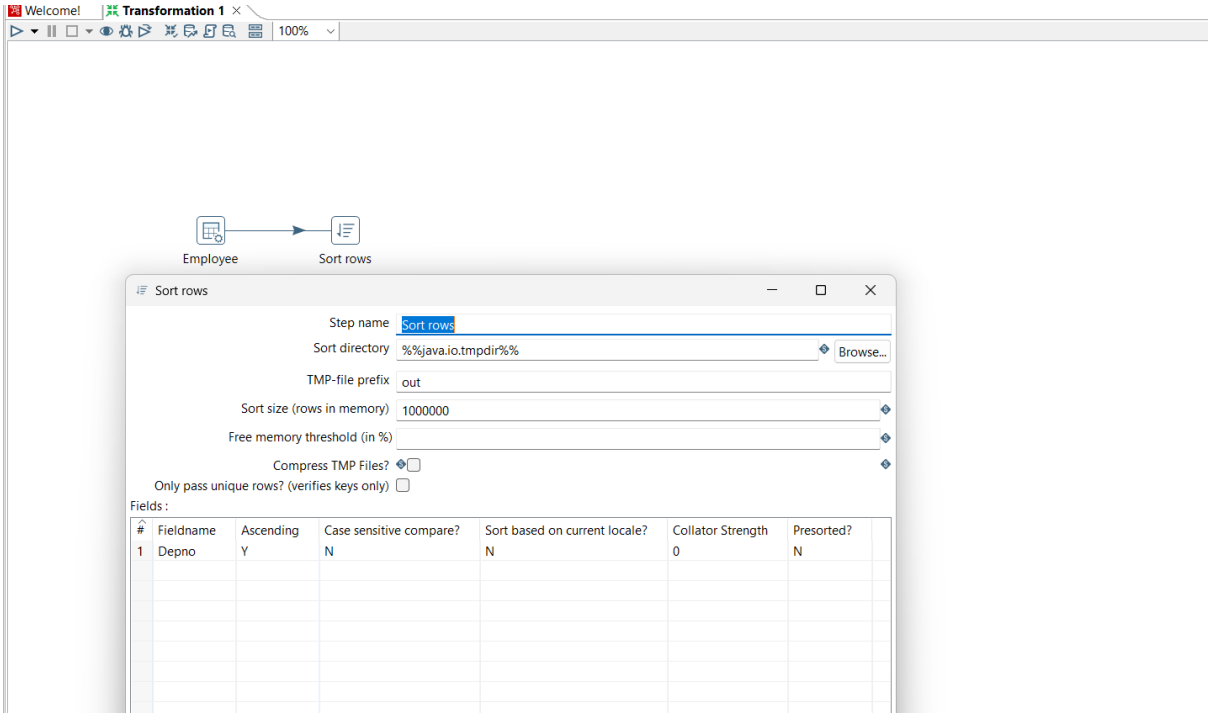


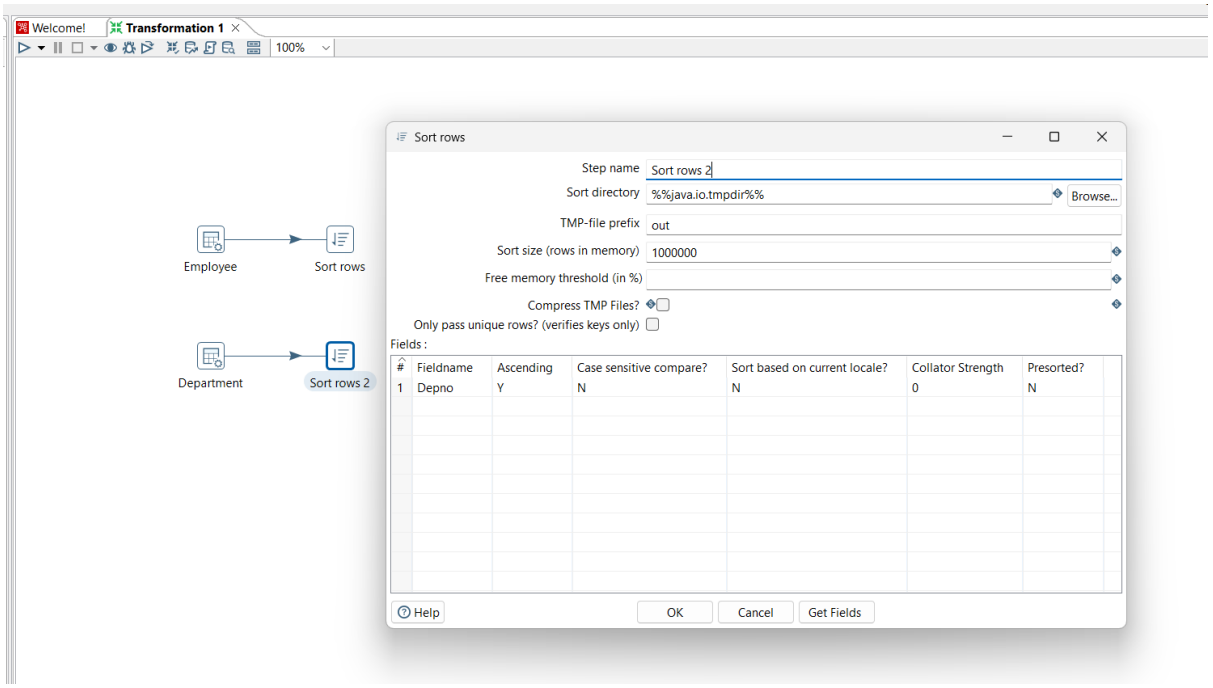




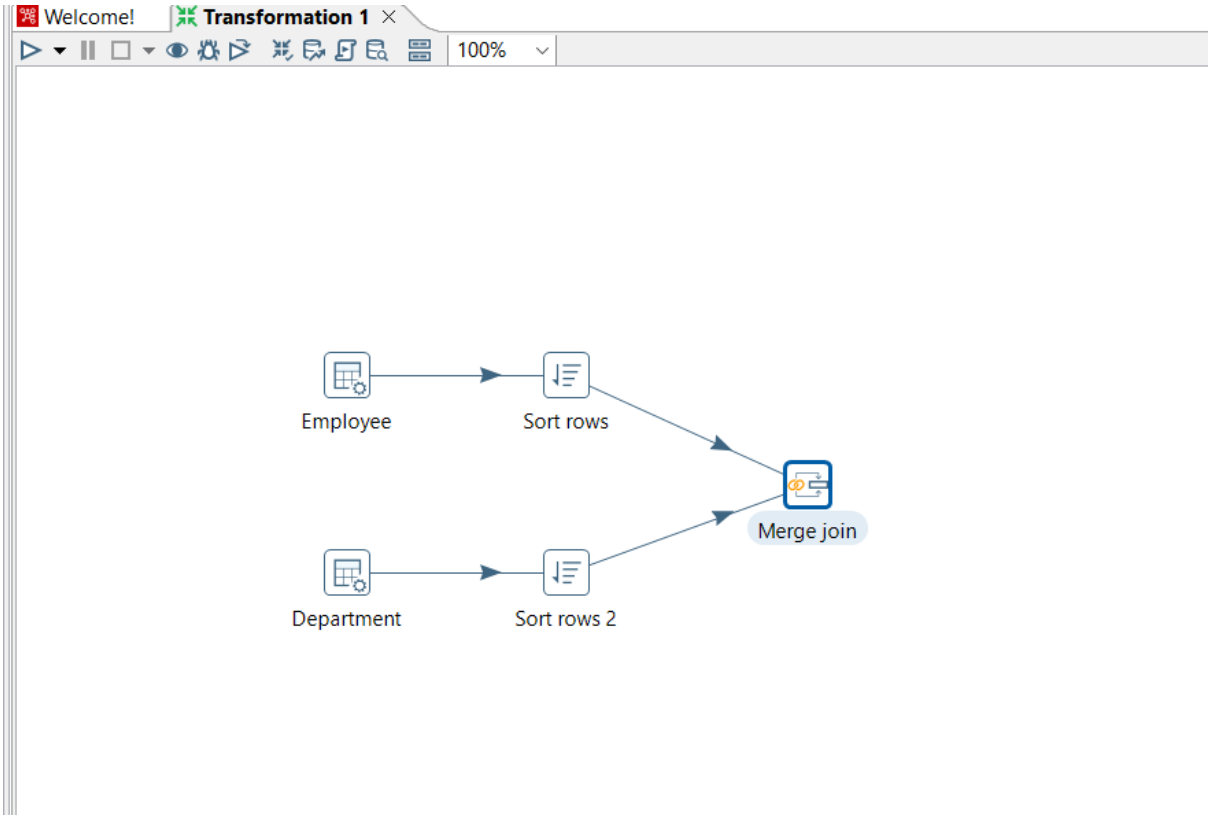
Insert sort rows transformation(on Empid) for both Employee and Department.



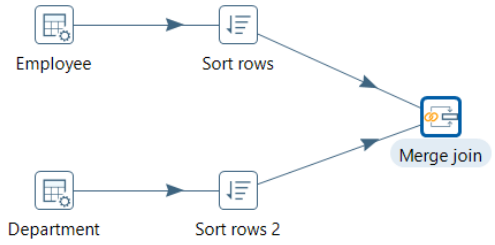


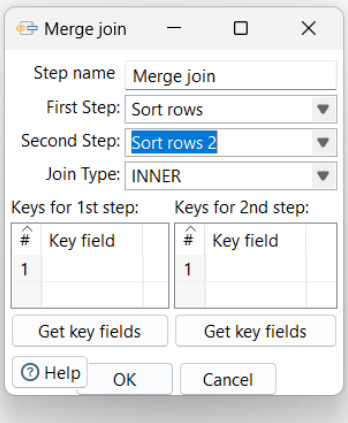


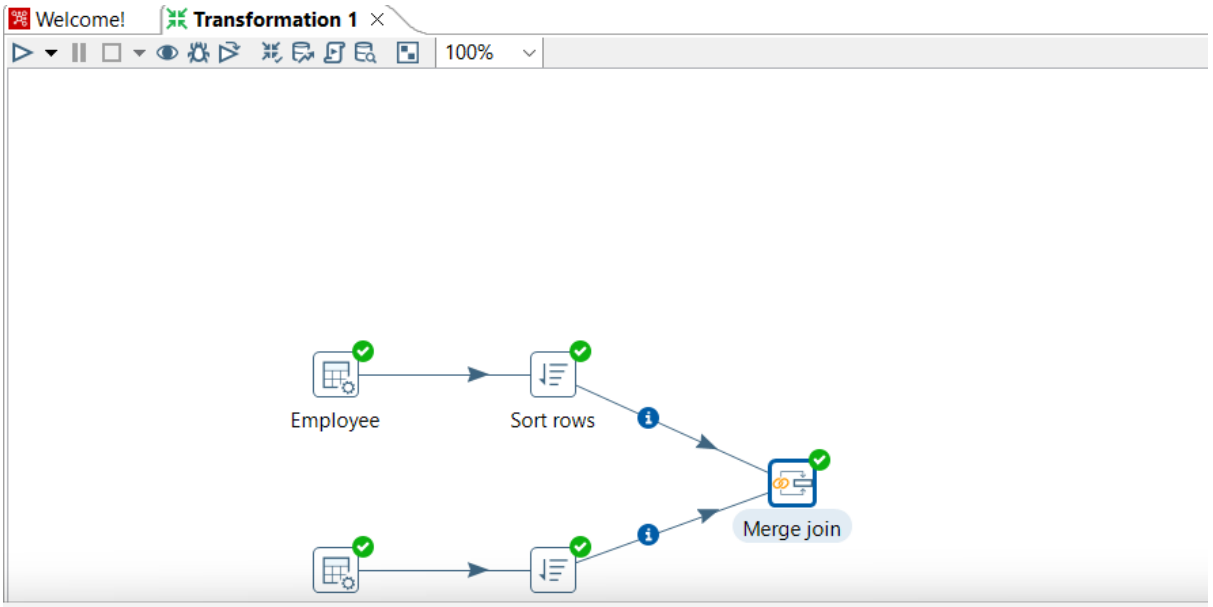
Add Merge join From Joins and connect it with sort rows.



Double click on Merge join and made necessary settings First select First_step, then Second_step, select Join type INNER, and from get fields select Depno only, after that quick launch the transformation.





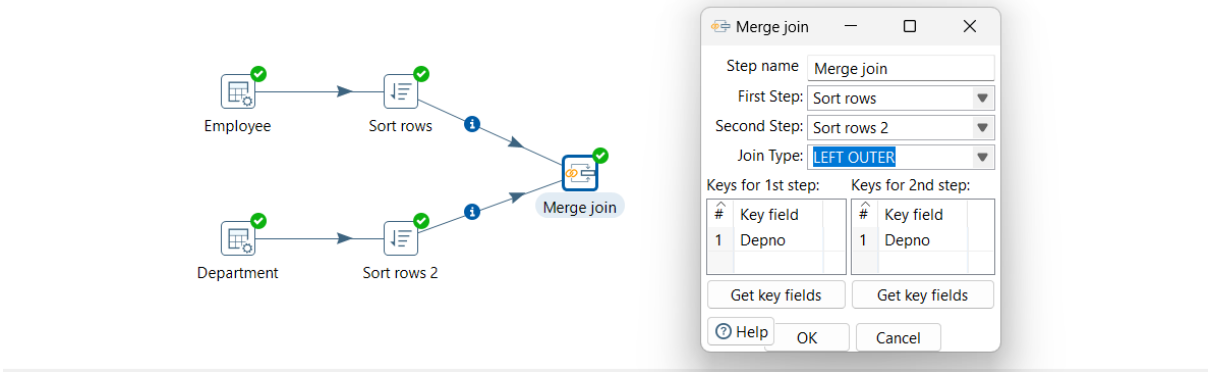


Examine preview data

ows of step: Merge join (3 rows)

#	Empid	Empname	Age	Depno	Depno_1	Depname
1	103	Vanshika	22	30	30	Hr
2	102	Siddharth	21	20	20	Admin
3	101	Sarvesh	21	10	10	IT

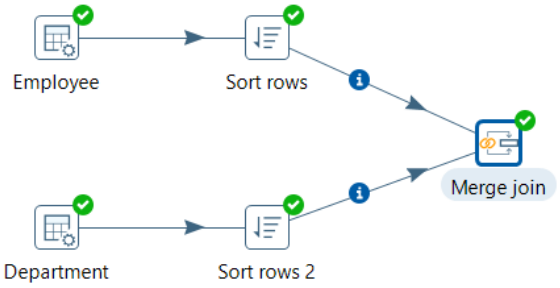
Select Join Type Left Outer, then Right Outer and finally Full Outer join

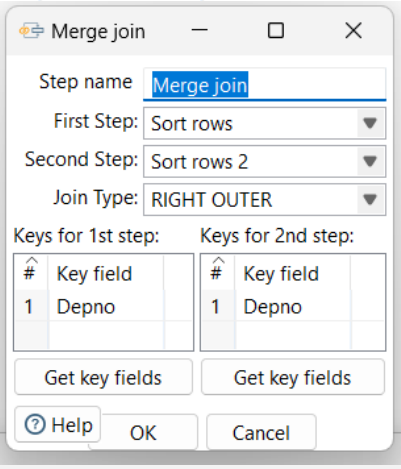


Examine preview data

Rows of step: Merge join (5 rows)

#	Empid	Empname	Age	Depno	Depno_1	Depname
1	104	Ziya	17	40	<null>	<null>
2	103	Vanshika	22	30	30	Hr
3	102	Siddharth	21	20	20	Admin
4	101	Sarvesh	21	10	10	IT
5	<null>	<null>	<null>	<null>	<null>	<null>

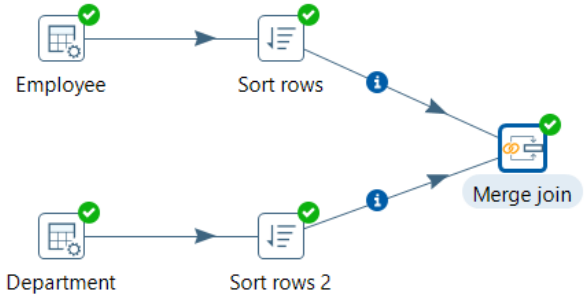


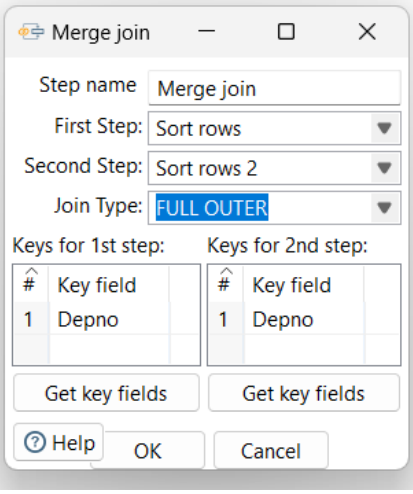


Examine preview data

Rows of step: Merge join (3 rows)

#	Empid	Empname	Age	Depno	Depno_1	Depname
1	103	Vanshika	22	30	30	Hr
2	102	Siddharth	21	20	20	Admin
3	101	Sarvesh	21	10	10	IT



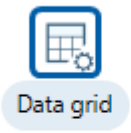


Examine preview data

Rows of step: Merge join (5 rows)

#	Empid	Empname	Age	Depno	Depno_1	Depname
1	104	Ziya	17	40	<null>	<null>
2	103	Vanshika	22	30	30	Hr
3	102	Siddharth	21	20	20	Admin
4	101	Sarvesh	21	10	10	IT
5	<null>	<null>	<null>	<null>	<null>	<null>

10. Implement different data validations on table.



Double click on Data grid and create product table in Meta tab and enter records into it using Data tab.

Data grid

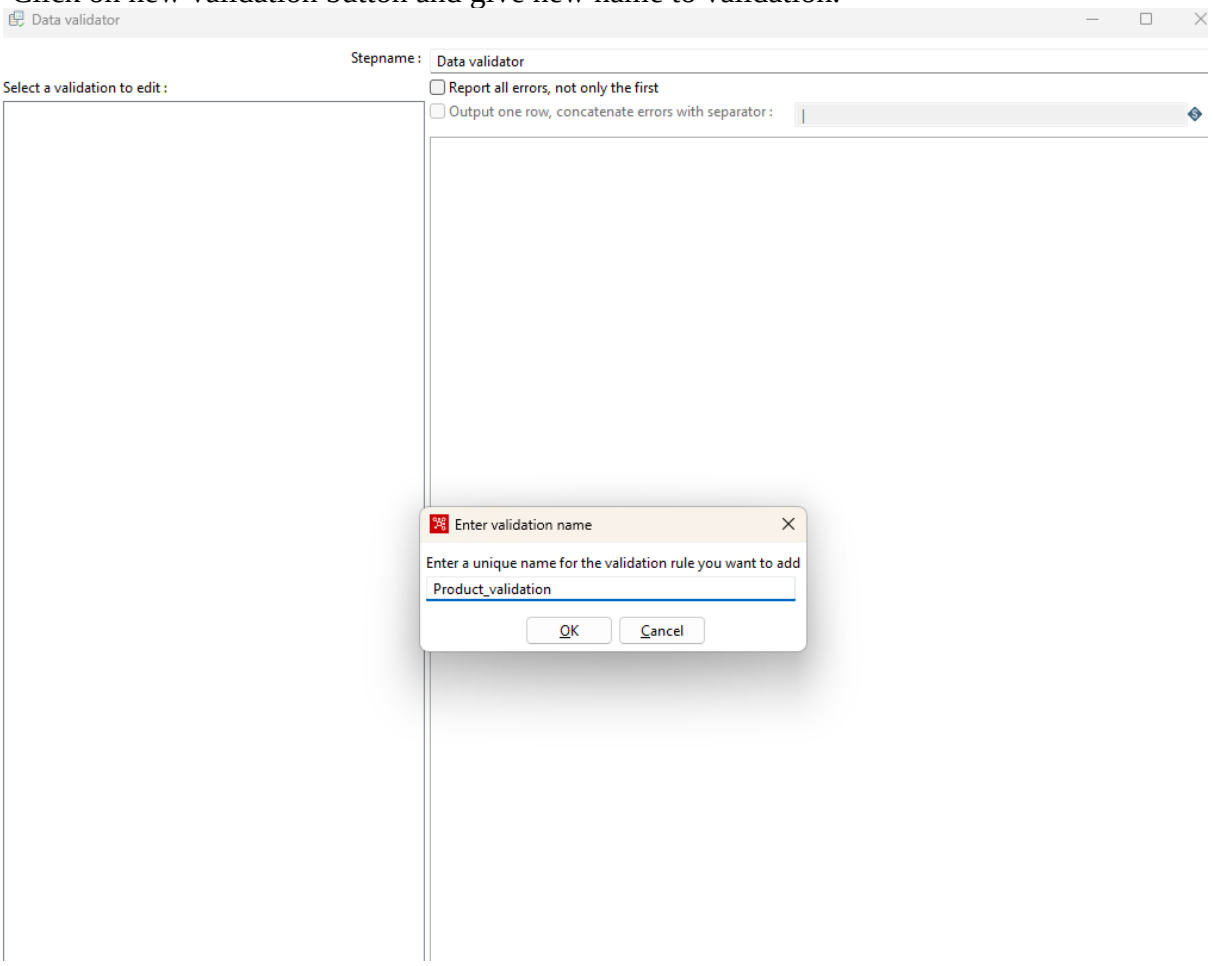
Step name Product

Meta Data

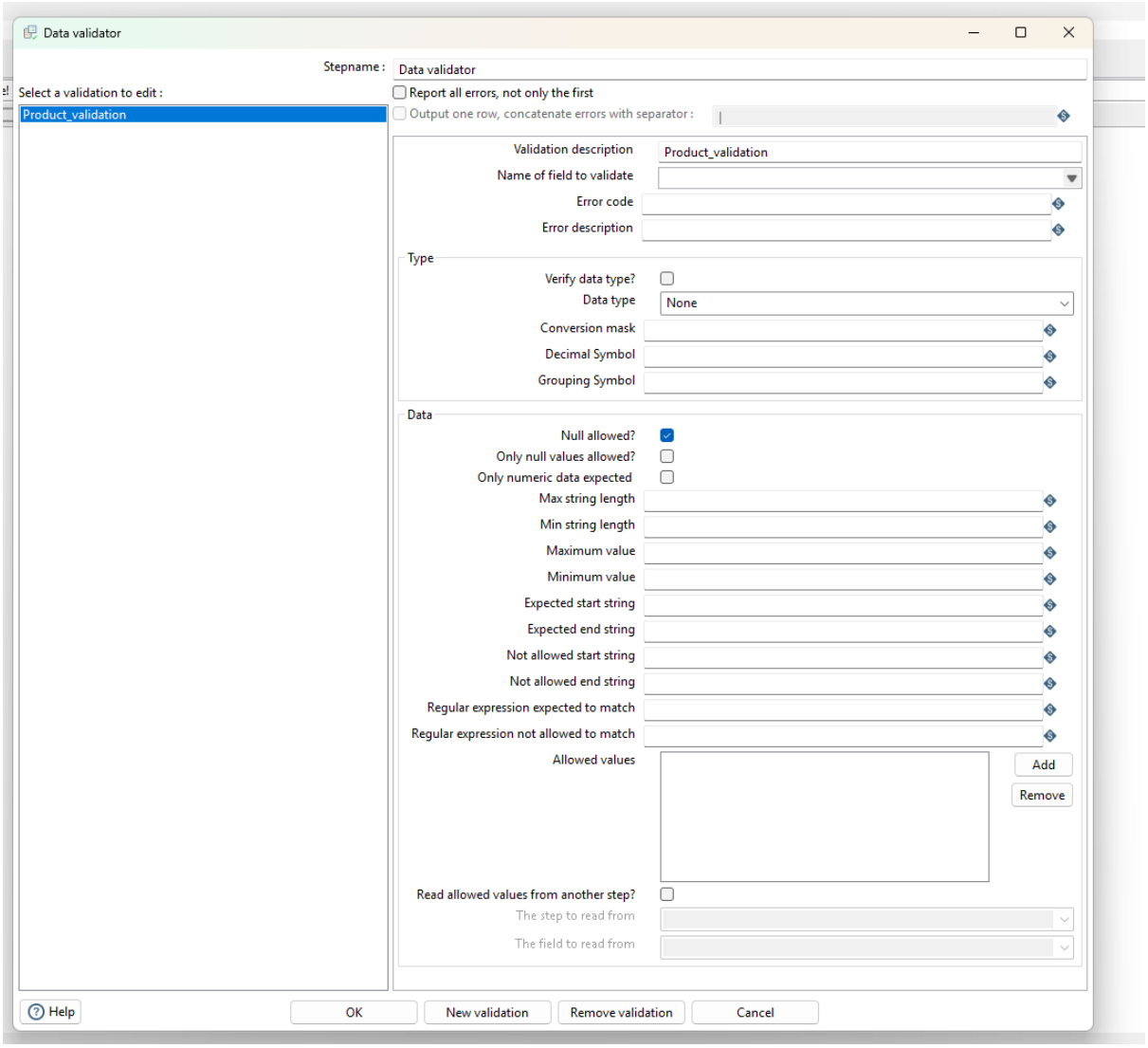
#	Name	Type	Format	Length	Precision	Cu
1	ProName	String				
2	ProPrice	integer				
3	Prold	integer				
4	Check	String				

Go to Validation steps and select, Drag and Drop Data validator on panel and double click on it.

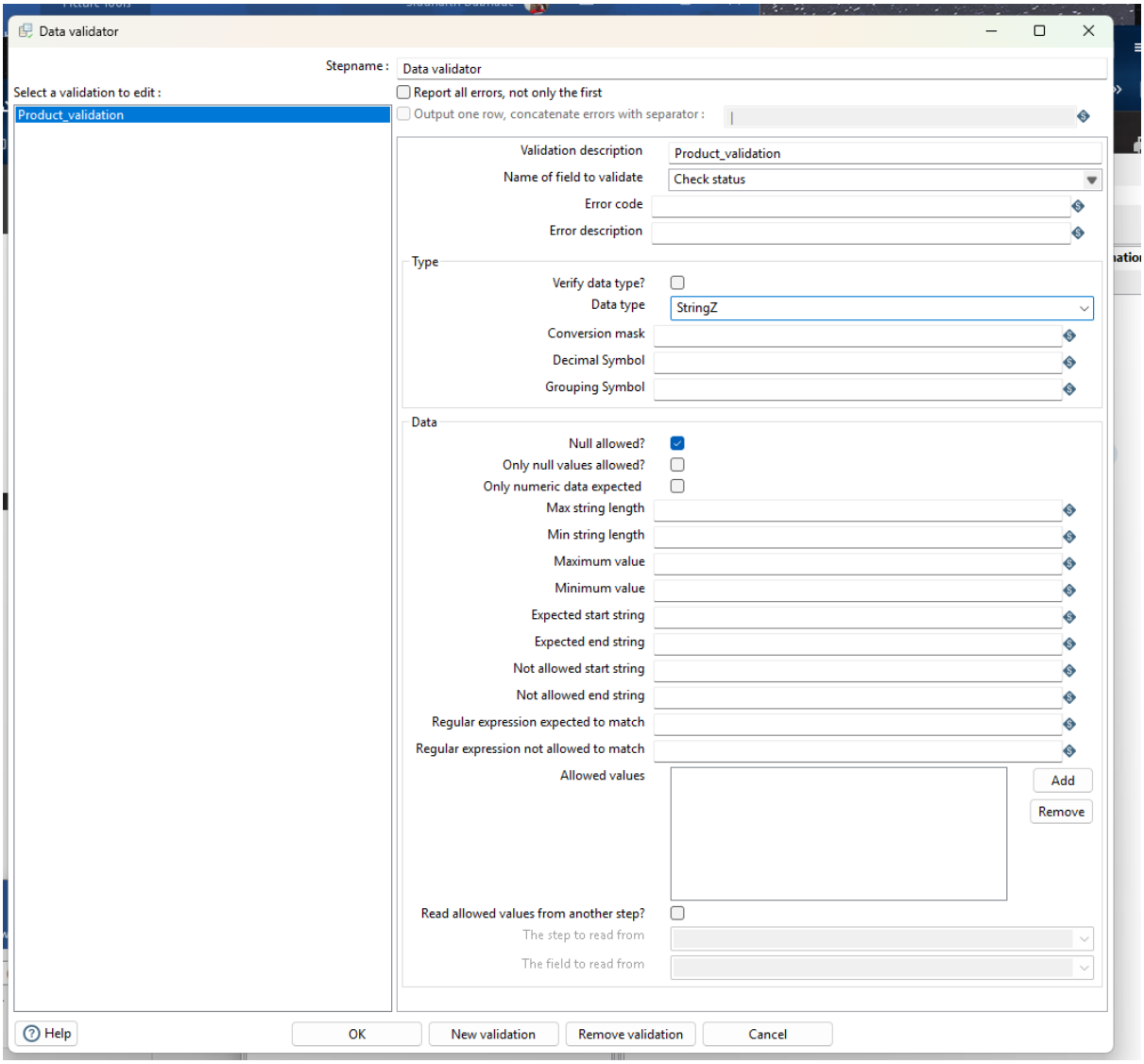
Click on new validation button and give new name to validation.



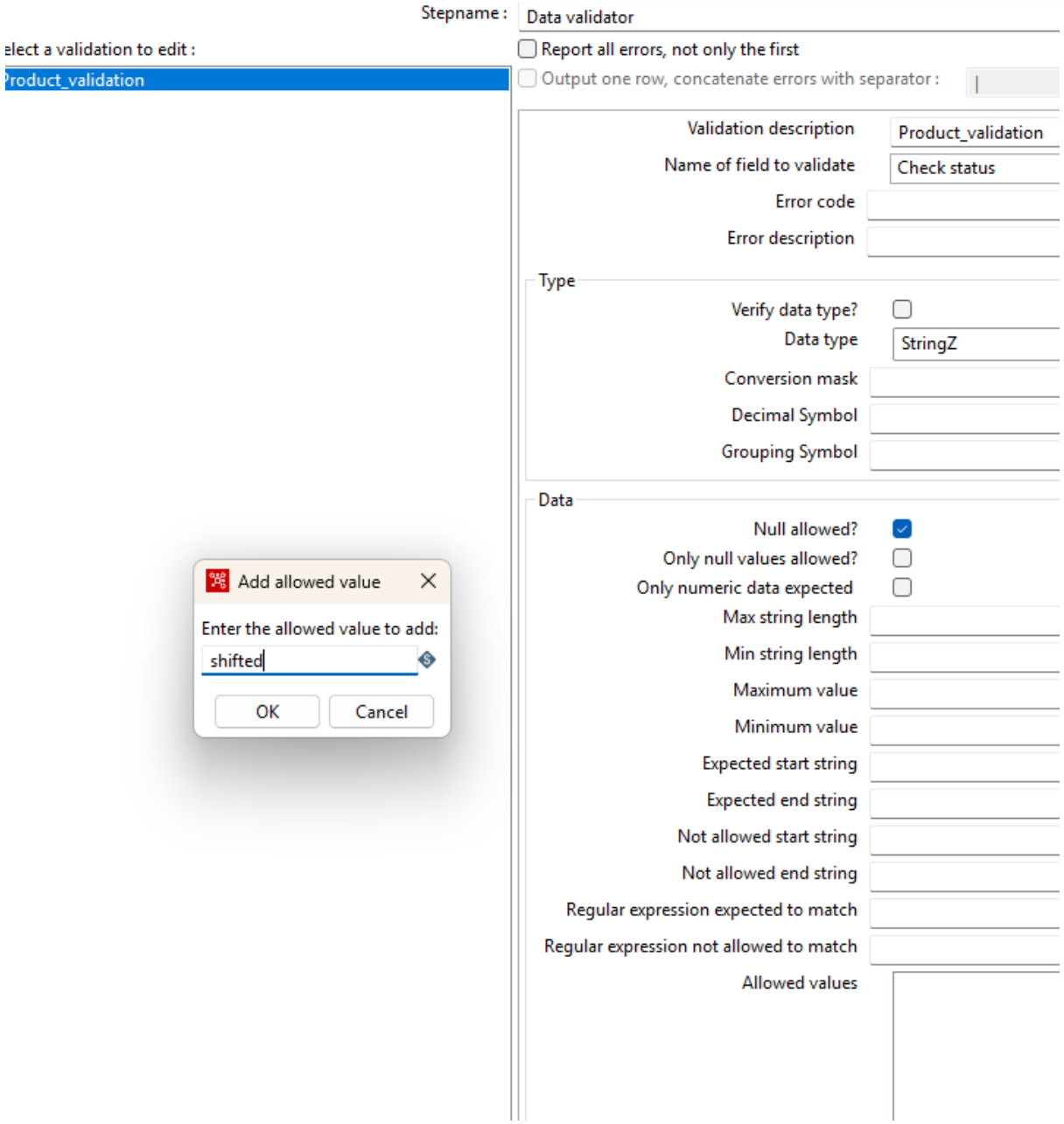
Once new validation is created it will appear on left panel, double click on it to set data validation



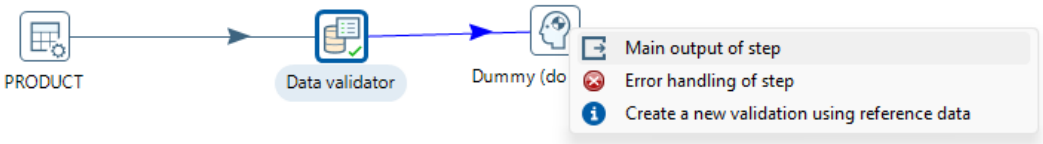
Set values for Name of field to validate=check status, and Data type=string



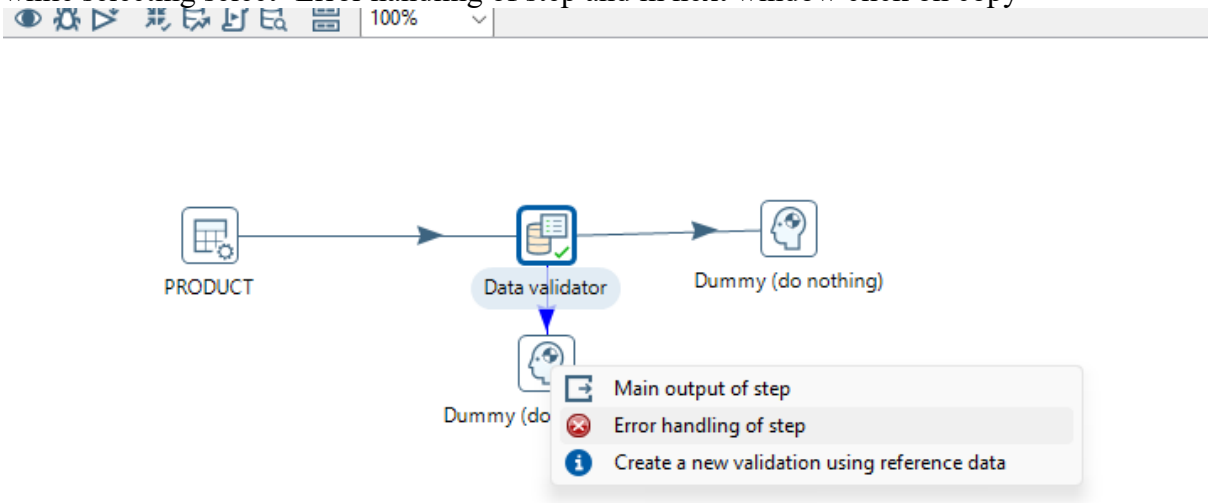
Click on add to set validation, set it to Shifted and press Enter and click on ok.



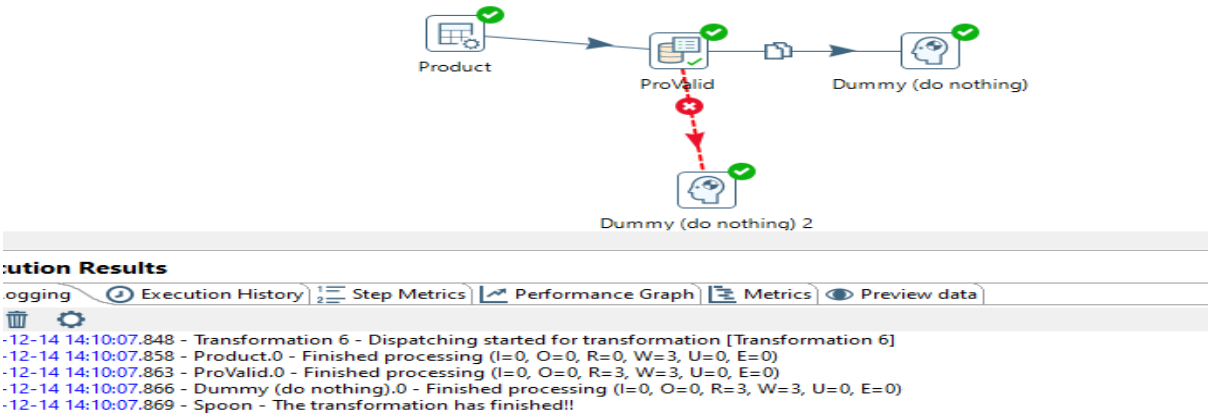
Select Dummy file from Flow to hold output and connect it with Data validation, while connecting select option " Main Output of step"



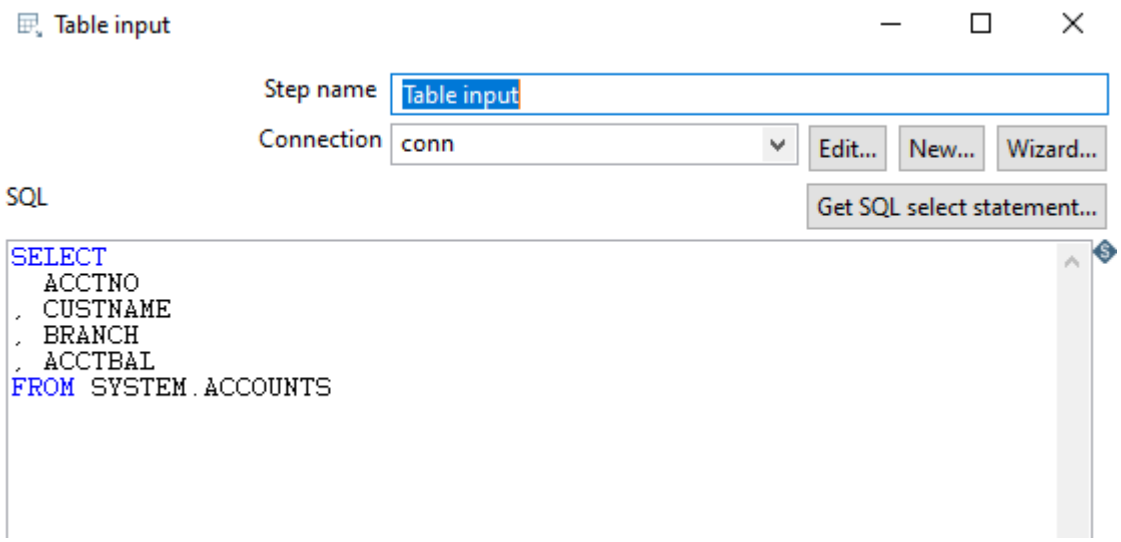
Select another dummy and connect it to ‘Validation for product’ and
while selecting select ‘Error handling of step and in next window click on copy

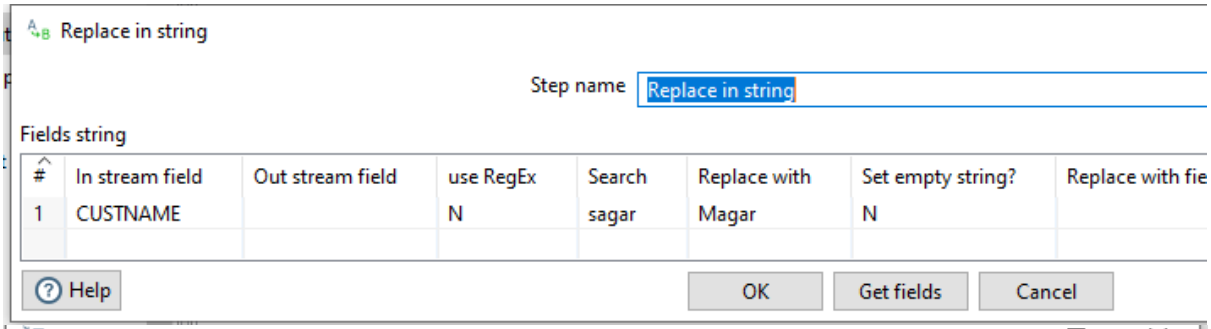


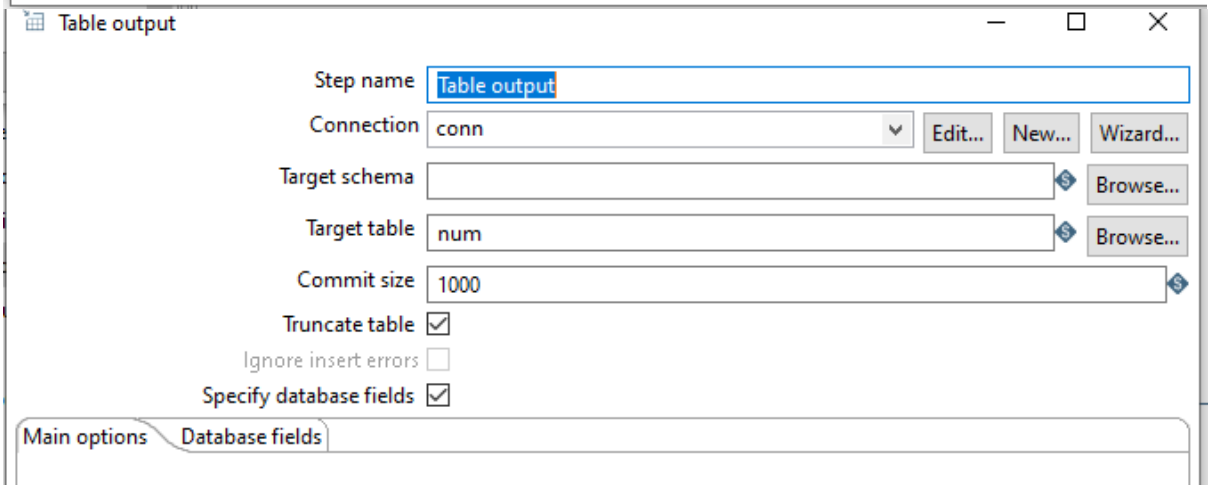
launch transformation by selecting one dummy file each.

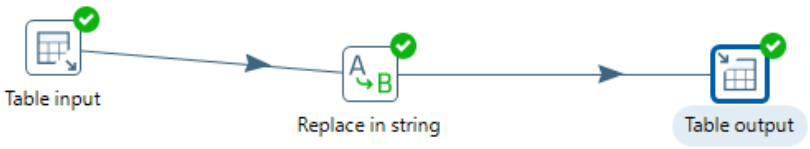


11.Replace Strings









Execution Results

Logging | Execution History | Step Metrics | Performance Graph | Metrics | Preview data

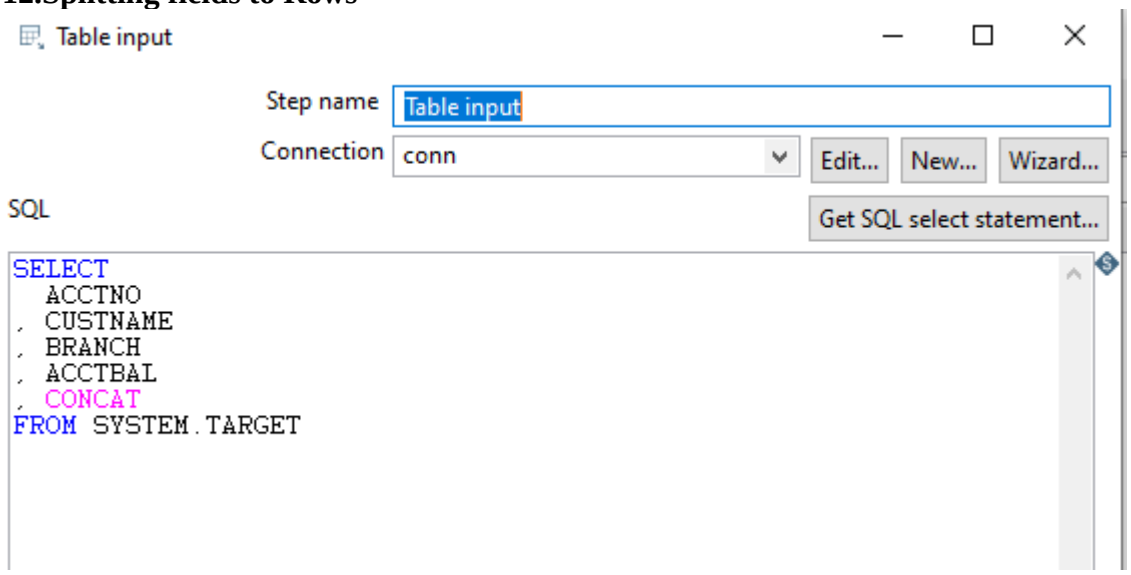
```
2024-12-14 14:15:59.335 - Transformation 7 - Dispatching started for transformation [Transformation 7]
2024-12-14 14:15:59.336 - Table output.0 - Connected to database [conn] (commit=1000)
2024-12-14 14:15:59.368 - Table input.0 - Finished reading query, closing connection
2024-12-14 14:15:59.369 - Table input.0 - Finished processing (I=7, O=0, R=0, W=7, U=0, E=0)
2024-12-14 14:15:59.372 - Replace in string.0 - Finished processing (I=0, O=0, R=7, W=7, U=0, E=0)
2024-12-14 14:15:59.382 - Table output.0 - Finished processing (I=0, O=7, R=7, W=7, U=0, E=0)
2024-12-14 14:15:59.383 - Spoon - The transformation has finished!!
```

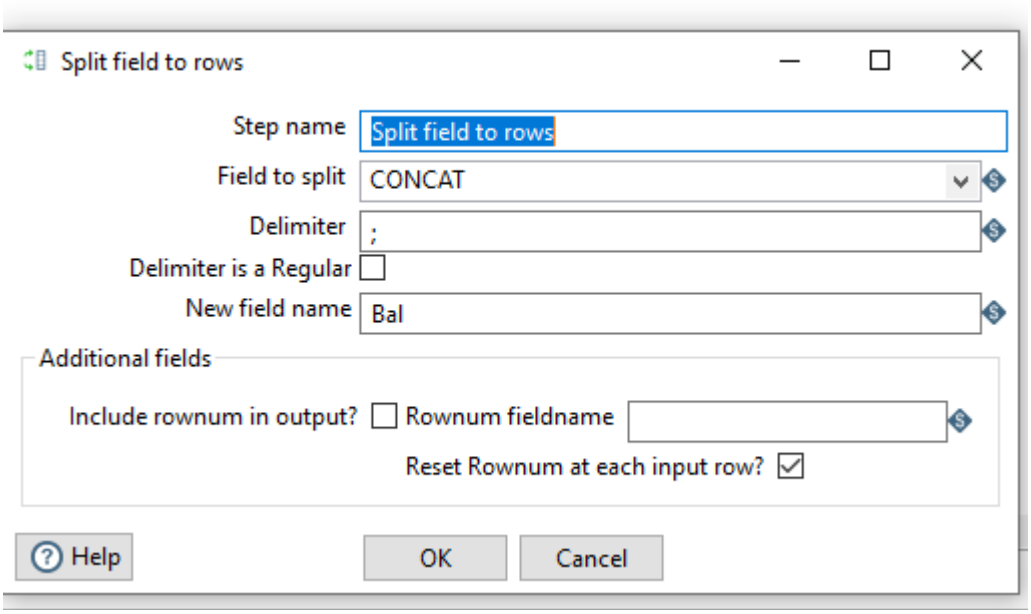
Examine preview data

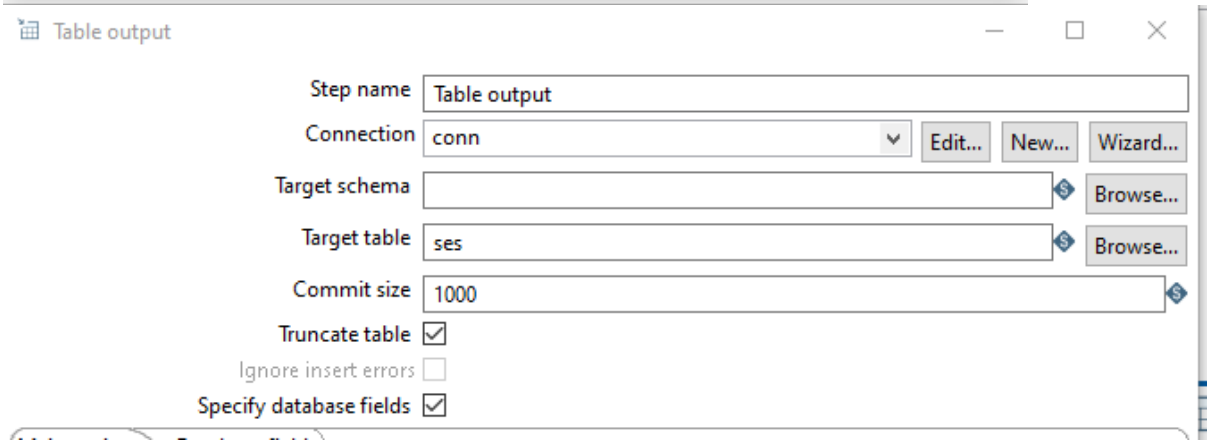
Rows of step: Replace in string (7 rows)

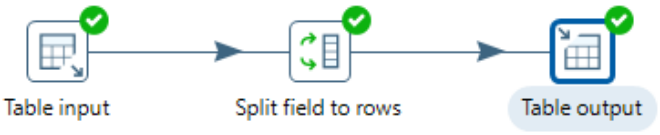
#	ACCTNO	CUSTNAME	BRANCH	ACCTBAL
7	1.0	Magar	mumbai	5000.0
6	2.0	arman	airport	9998.0
5	3.0	shashi	mumbai	7000.0
4	4.0	chutki	Fun-Fiesta	6000.0
3	7.0	sanju	ranchi	6500.0
2	8.0	monu	thane	11000.0
1	9.0	sonu	thane	11500.0

12.Splitting fields to Rows









Execution Results

Logging | Execution History | Step Metrics | Performance Graph | Metrics | Preview data

```
2024-12-14 14:21:47.376 - Transformation 8 - Dispatching started for transformation [Transformation 8]
2024-12-14 14:21:47.378 - Table output.0 - Connected to database [conn] (commit=1000)
2024-12-14 14:21:47.411 - Table input.0 - Finished reading query, closing connection
2024-12-14 14:21:47.412 - Table input.0 - Finished processing (I=42, O=0, R=0, W=42, U=0, E=0)
2024-12-14 14:21:47.415 - Split field to rows.0 - Finished processing (I=0, O=0, R=42, W=49, U=0, E=0)
2024-12-14 14:21:47.425 - Table output.0 - Finished processing (I=0, O=49, R=49, W=49, U=0, E=0)
2024-12-14 14:21:47.427 - Spoon - The transformation has finished!!
```

Examine preview data

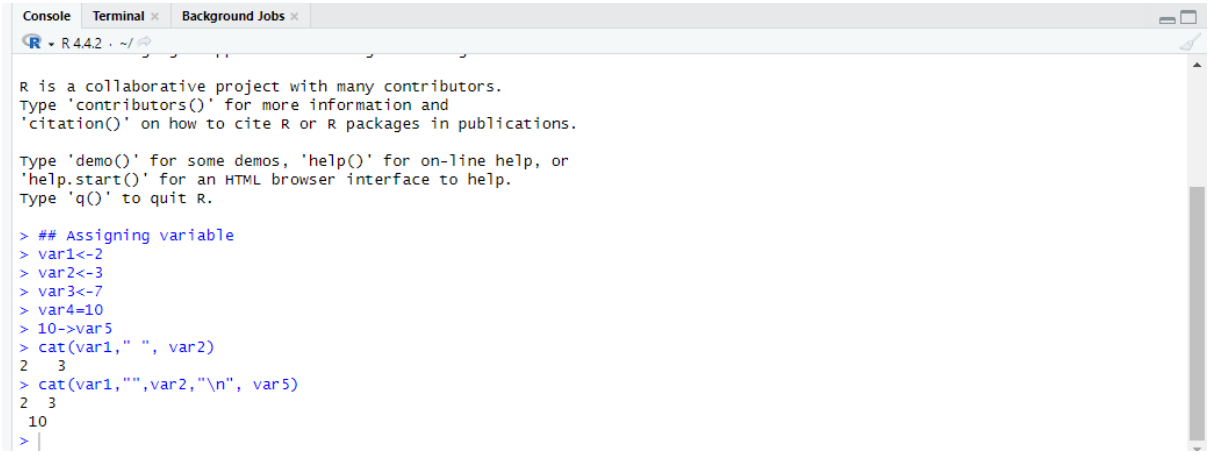
Rows of step: Split field to rows (49 rows)

#	ACCTNO	CUSTNAME	BRANCH	ACCTBAL	CONCAT	Bal
1	9.0	sonu	thane	11500.0	9.0-11500.0	9.0-11500.0
2	8.0	monu	thane	11000.0	8.0-11000.0	8.0-11000.0
3	7.0	sanju	ranchi	6500.0	7.0-6500.0	7.0-6500.0
4	4.0	chutki	Fun-Fiesta	6000.0	4.0-6000.0	4.0-6000.0
5	3.0	shashi	mumbai	7000.0	3.0-7000.0	3.0-7000.0
6	2.0	arman	airport	9998.0	2.0-9998.0	2.0-9998.0
7	1.0	sagar	mumbai	5000.0	1.0-5000.0	1.0-5000.0
8	9.0	sonu	thane	11500.0	9.0-11500.0	9.0-11500.0
9	8.0	monu	thane	11000.0	8.0-11000.0	8.0-11000.0
10	7.0	sanju	ranchi	6500.0	7.0-6500.0	7.0-6500.0
11	4.0	chutki	Fun-Fiesta	6000.0	4.0-6000.0	4.0-6000.0
12	3.0	shashi	mumbai	7000.0	3.0-7000.0	3.0-7000.0
13	2.0	arman	airport	9998.0	2.0-9998.0	2.0-9998.0
14	1.0	sagar	mumbai	5000.0	1.0-5000.0	1.0-5000.0
15	9.0	sonu	thane	11500.0	9.0;11500.0	11500.0
16	9.0	sonu	thane	11500.0	9.0;11500.0	9.0
17	8.0	monu	thane	11000.0	8.0;11000.0	11000.0
18	8.0	monu	thane	11000.0	8.0;11000.0	8.0
19	7.0	sanju	ranchi	6500.0	7.0;6500.0	6500.0
20	7.0	sanju	ranchi	6500.0	7.0;6500.0	7.0

PRACTICAL NO 5

Introduction to basics of R programming: - Install packages, loading packages Data types, checking type of variable, printing variable and objects (Vector, Matrix, List, Factor, Data frame, Table) cbind-ing and rbind-ing, Reading and Writing data. setwd (), getwd (), data (), rm (), Attaching and Detaching data. Reading data from the consol. Loading data from different data sources. (CSV, Excel)

```
## Assigning variable
var1<-2
var2<-3
var3<-7
var4=10
10->var5
cat(var1," ", var2)
cat(var1,"",var2,"\n", var5)
```

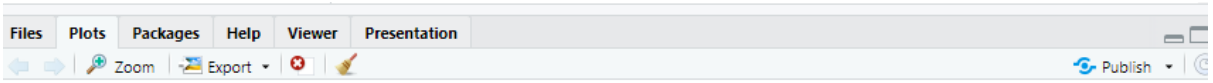


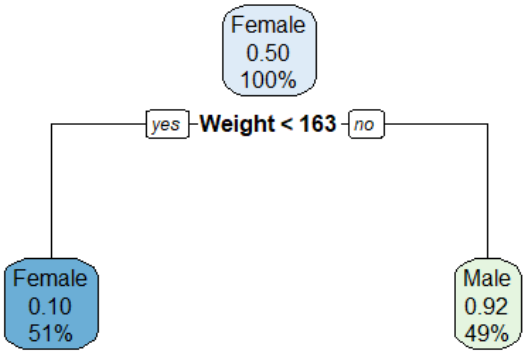

```
## Data types in R
## logical, numeric,integer,complex,logical,character
## num<- 12,12.35,-34.45,11.67
## integer<-36L
## complex<-5+2i
## logical= TRUE,FALSE ( not 0 and 1)
## character= 'a',"Hello R", "FALSE",'26.3454'
num<- 12
class(num)
intl<-15
class(intl)
intl<-as.integer(intl)
class(intl)
int2<-45L
class(int2)
```


```
# R Matrix
## Matrix is two dimensional data
##matrix(data,nrow,ncol,byrow,dim_name)
##mat<-matrix(c(2:13),nrow=4,byrow = TRUE)
##mat<-matrix(c(2,3,11,5,6,7),nrow= 3, ncol= 2, byrow= TRUE)
##mat
##mat<-matrix(c(2,3,11,5,6,7),nrow= 2, ncol= 3, byrow= TRUE)
##mat
x<-matrix(c(5:16), nrow=4, byrow= TRUE)
y<-matrix(c(7:18),nrow=4, byrow= FALSE)
x
y
## Naming Matrix
row_name<-c("r1","r2","r3","r4")
col_names<-c("c1","c2","c3")
z<- matrix(c(7:18), nrow=4, byrow= TRUE , dimnames= list(row_name,col_names))
z
## Accessing elements
print(z[3,1])
print(z[,1])
print(z[2,])
## updating elements
z[4,3]<-0
z[z==11]<-0
z[z>15]<-0
#cbind() and rbind() are used to add col n row resp
rbind(z,c(2,3,4))
cbind(z,c(8,5,2,0))
z
## Transpose i.e row into col n col into row
t(z)
a1<-matrix(c(5:16), nrow=4, ncol=3)
a2<-matrix(c(1:12), nrow=4, ncol=3)
sum<-a1+a2
```

```
> # R Matrix
> ## Matrix is two dimensional data
> ##matrix(data,nrow,ncol,byrow,dim_name)
> ##mat<-matrix(c(2:13),nrow=4,byrow = TRUE)
> ##mat<-matrix(c(2,3,11,5,6,7),nrow= 3, ncol= 2, byrow= TRUE)
> ##mat
> ##mat<-matrix(c(2,3,11,5,6,7),nrow= 2, ncol= 3, byrow= TRUE)
> ##mat
> x<-matrix(c(5:16), nrow=4, byrow= TRUE)
> y<-matrix(c(7:18),nrow=4, byrow= FALSE)
> x
     [,1] [,2] [,3]
[1,]    5    6    7
[2,]    8    9   10
[3,]   11   12   13
[4,]   14   15   16
> y
     [,1] [,2] [,3]
[1,]    7   11   15
[2,]    8   12   16
[3,]    9   13   17
[4,]   10   14   18
> ## Naming Matrix
> row_name<-c("r1","r2","r3","r4")
> col_names<-c("c1","c2","c3")
> z<- matrix(c(7:18), nrow=4, byrow= TRUE , dimnames= list(row_name,col_names))
> z
   c1 c2 c3
r1  7  8  9
r2 10 11 12
r3 13 14 15
r4 16 17 18
> ## Accessing elements
> print(z[3,1])
[1] 13
> print(z[,1])
r1 r2 r3 r4
 7 10 13 16
> print(z[2,])
c1 c2 c3
10 11 12
> ## updating elements
> z[4,3]<-0
> z[z==11]<-0
> z[z>15]<-0
> #cbind() and rbind() are used to add col n row resp
> rbind(z,c(2,3,4))
   c1 c2 c3
r1  7  8  9
r2 10  0 12
r3 13 14 15
r4  0  0  0
    2  3  4
> cbind(z,c(8,5,2,0))
   c1 c2 c3
r1  7  8  9 8
r2 10  0 12 5
r3 13 14 15 2
r4  0  0  0 0
> z
   c1 c2 c3
r1  7  8  9
r2 10  0 12
r3 13 14 15
r4  0  0  0
> ## Transpose i.e row into col n col into row
> t(z)
   r1 r2 r3 r4
c1  7 10 13  0
c2  8  0 14  0
c3  9 12 15  0
> a1<-matrix(c(5:16), nrow=4, ncol=3)
> a2<-matrix(c(1:12), nrow=4, ncol=3)
>
```

```
install.packages("rpart.plot")
library(rpart)
library(rpart.plot)
setwd("C:/Users/Siddharth/Downloads/Rprac")
data<- read.csv("weight-height.csv")
#tree<- rpart(Height ~ Gender+ Weight,data)
#a<- data.frame(Gender= c("Male"),Weight = c(85))
#result<- predict(tree,a)
#print(result)
#rpart.plot(tree)

tree <- rpart(Gender ~ Height + Weight , data)
a<-data.frame(Height=c(190), Weight=c(85))
result <- predict(tree,a)
rpart.plot(tree)
```





PRACTICAL NO 6

Data preprocessing techniques in R Naming and Renaming variables Adding a new variables Dealing with missing value Dealing with categorical data Data reduction using subsetting

Naming and Renaming Variables

Code-

```
# Create a simple dataset
data <- data.frame(
  Height = c(5.1, 5.5, 6.2, 5.9, 5.4),
  Weight = c(150, 160, 180, 175, 165),
  Age = c(23, 25, 22, 28, 24)
)

# View the dataset
print(data)

# Rename columns
colnames(data) <- c("Person_Height", "Person_Weight", "Person_Age")

# View the updated dataset
print(data)
```

```
Console   Terminal   Background Jobs
R 4.4.2 · C:/Users/Siddharth/Downloads/Rprac/
4          4.6         3.1          1.5         0.2  setosa       1
5          5.0         3.6          1.4         0.2  setosa       1
6          5.4         3.9          1.7         0.4  setosa       1
> # Step 7: Visualize the Clusters
> # Scatter plot using ggplot2 to visualize the clusters based on Sepal.Length and Sepal.Width
> ggplot(iris, aes(x = Sepal.Length, y = Sepal.Width, color = Cluster)) +
+   geom_point(size = 3) +
+   labs(title = "Agglomerative Clustering - Clustered Data",
+        x = "Sepal Length",
+        y = "Sepal Width") +
+   theme_minimal()
> source("C:/Users/Siddharth/Downloads/Rprac/Naive Bayes.R", echo=TRUE)

> # Create a simple dataset
> data <- data.frame(
+   Height = c(5.1, 5.5, 6.2, 5.9, 5.4),
+   Weight = c(150, 160, 180, 175, 165),
+   Age = c(23, 25 .... [TRUNCATED]

> # View the dataset
> print(data)
  Height Weight Age
1    5.1    150  23
2    5.5    160  25
3    6.2    180  22
4    5.9    175  28
5    5.4    165  24

> # Rename columns
> colnames(data) <- c("Person_Height", "Person_Weight", "Person_Age")

> # View the updated dataset
> print(data)
  Person_Height Person_Weight Person_Age
1           5.1           150         23
2           5.5           160         25
3           6.2           180         22
4           5.9           175         28
5           5.4           165         24
>
```

Adding New Variables

Code-

```
# Add a new variable 'BMI' calculated as Weight / Height^2
data$BMI <- data$Person_Weight / (data$Person_Height ^ 2)

# View the updated dataset with the new column
print(data)
```

Output-

```
> # Add a new variable 'BMI' calculated as Weight / Height^2
> data$BMI <- data$Person_Weight / (data$Person_Height ^ 2)
> # View the updated dataset with the new column
> print(data)
  Person_Height Person_Weight Person_Age      BMI
1           5.1           150         23 5.767013
2           5.5           160         25 5.289256
3           6.2           180         22 4.682622
4           5.9           175         28 5.027291
5           5.4           165         24 5.658436
>
```

Dealing with Missing Values

Code-

```
# Introduce missing values in the Age column
data$Person_Age[2] <- NA
data$Person_Weight[4] <- NA

# View the dataset with missing values
print(data)
```

Output-

```
> # Introduce missing values in the Age column
> data$Person_Age[2] <- NA
> data$Person_Weight[4] <- NA
> # View the dataset with missing values
> print(data)
  Person_Height Person_Weight Person_Age      BMI
1           5.1           150         23 5.767013
2           5.5           160         NA 5.289256
3           6.2           180         22 4.682622
4           5.9            NA         28 5.027291
5           5.4           165         24 5.658436
>
```

PRACTICAL NO 7

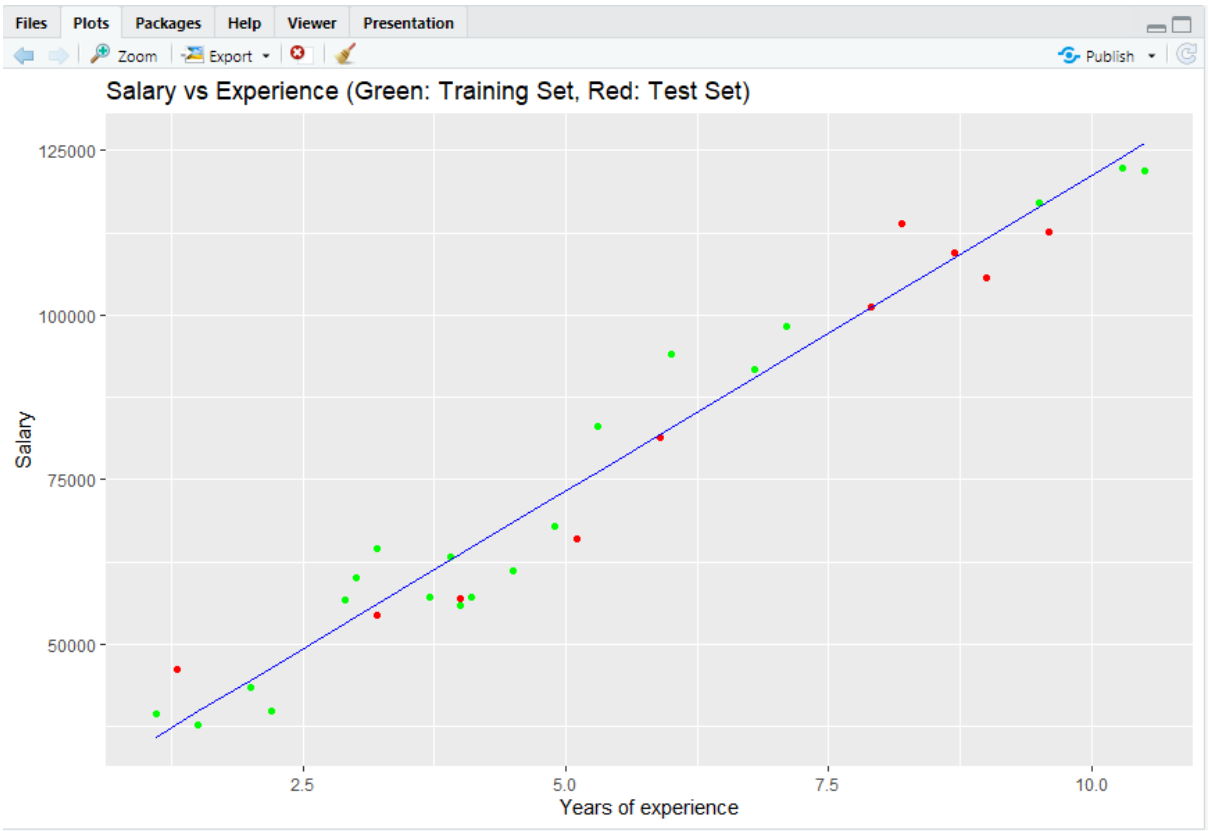
Aim-Implementation and analysis of Linear regression.

Code-

```
# Load the dataset into R
dataset = read.csv("Salary_Data.csv")

# Load the caTools library for splitting the dataset into training and test sets
library(caTools)

# Split the dataset into training and test sets using a 2/3 to 1/3 ratio
split = sample.split(dataset$Salary, SplitRatio = 2/3)

# Create the training set by subsetting the dataset where split is TRUE
training_set = subset(dataset, split == TRUE)

# Create the test set by subsetting the dataset where split is FALSE
test_set = subset(dataset, split == FALSE)

# Fit a linear regression model to the training data
# Salary is the dependent variable, and YearsExperience is the independent variable
regressor = lm(formula = Salary ~ YearsExperience, data = training_set)

# Summarize the fitted linear model to see the coefficients, R-squared value, and other statistics
summary(regressor)

# Predict the salaries in the test set using the fitted linear model
salary_predict = predict(regressor, newdata = test_set)

# Print the actual and predicted salaries in the test set side by side
print(data.frame(Salary = test_set$Salary, Salary_Predict = salary_predict))

# Load the ggplot2 library for data visualization
library(ggplot2)

# Create a scatter plot with the training set points in green and test set points in red
# Add a regression line (fitted line) in blue based on the training set
ggplot() +
  geom_point(aes(x = training_set$YearsExperience, y = training_set$Salary), colour = 'green')
+
  geom_point(aes(x = test_set$YearsExperience, y = test_set$Salary), colour = 'red') +
  geom_line(aes(x = training_set$YearsExperience, y = predict(regressor, newdata =
training_set)), colour = 'blue') +
  ggtitle('Salary vs Experience (Green: Training Set, Red: Test Set)') +
  xlab('Years of experience') +
  ylab('Salary')
```

Output-

```
Call:
lm(formula = Salary ~ YearsExperience, data = training_set)

Residuals:
    Min      1Q  Median      3Q     Max
-7915.8 -4199.0  -275.2  3819.2 11062.5

Coefficients:
                Estimate Std. Error t value Pr(>|t|)
(Intercept)        25374       2636   9.627 1.60e-08 ***
YearsExperience     9584        476  20.134 8.58e-14 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 5784 on 18 degrees of freedom
Multiple R-squared:  0.9575,    Adjusted R-squared:  0.9551
F-statistic: 405.4 on 1 and 18 DF,  p-value: 8.584e-14

> # Predict the salaries in the test set using the fitted linear model
> salary_predict = predict(regressor, newdata = test_set)
> # Print the actual and predicted salaries in the test set side by side
> print(data.frame(Salary = test_set$Salary, Salary_Predict = salary_predict))
   Salary Salary_Predict
2   46205       37833.33
8   54445       56042.67
13  56957       63709.76
17  66029       74252.01
19  81363       81919.10
23 101302      101086.83
24 113812      103961.99
25 109431      108753.92
26 105582      111629.08
28 112635      117379.40
> # Load the ggplot2 library for data visualization
> library(ggplot2)
> # Create a scatter plot with the training set points in green and test set points in red
> # Add a regression line (fitted line) in blue based on the training set
> ggplot() +
+   geom_point(aes(x = training_set$YearsExperience, y = training_set$Salary), colour = 'green') +
+   geom_point(aes(x = test_set$YearsExperience, y = test_set$Salary), colour = 'red') +
+   geom_line(aes(x = training_set$YearsExperience, y = predict(regressor, newdata = training_set)), colour = 'blue') +
+   ggtitle('Salary vs Experience (Green: Training Set, Red: Test Set)') +
+   xlab('Years of experience') +
+   ylab('Salary')
```



PRACTICAL NO 8

Aim-Implementation and analysis of Classification algorithms - Naive Bayesian

Code-

```
install.packages("e1071")
install.packages("caTools")
install.packages("caret")
install.packages("ggplot2")
install.packages("lattice")
library("e1071")
library("caTools")
library(ggplot2)
library(lattice)
library("caret")
# splitting data into train and test data
View(iris)
split<-sample.split(iris, SplitRatio = 0.7)
train_cl<- subset(iris, split == "TRUE")
test_cl<- subset(iris, split == "FALSE")
# Feature Scaling
train_scale <-scale(train_cl[,1:4])
test_scale <- scale(test_cl[,1:4])
# Fitting Naive Bayes Model to Training data set
set.seed(120) # setting seed
classifier_cl <- naiveBayes(Species ~ .,data = train_cl)
classifier_cl
# predicting on test data
ypred <- predict(classifier_cl, newdata = test_cl)
# confusion Matrix
cm<- table(test_cl$Species,ypred)
cm
# Model Evaluation
confusionMatrix(cm)
```

Output-

```
Console   Terminal ×   Background Jobs ×
Error fetching R version · C:/Users/Siddharth/Downloads/Rprac/

Naive Bayes Classifier for Discrete Predictors

Call:
naiveBayes.default(x = X, y = Y, laplace = laplace)

A-priori probabilities:
Y
    setosa versicolor  virginica
 0.3333333  0.3333333  0.3333333

Conditional probabilities:
            Sepal.Length
Y               [,1]      [,2]
  setosa     5.016667 0.3097088
  versicolor 5.916667 0.5414434
  virginica  6.740000 0.5763141

            Sepal.Width
Y               [,1]      [,2]
  setosa     3.456667 0.3490710
  versicolor 2.750000 0.3202908
  virginica  3.033333 0.2928261

            Petal.Length
Y               [,1]      [,2]
  setosa     1.466667 0.1881550
  versicolor 4.220000 0.4373904
  virginica  5.680000 0.5019960

            Petal.Width
Y               [,1]       [,2]
  setosa     0.220000 0.08051558
  versicolor 1.303333 0.20924055
  virginica  2.063333 0.29182344

> # predicting on test data
> ypred <- predict(classifier_cl, newdata = test_cl)
> # confusion Matrix
> cm<- table(test_cl$Species,ypred)
> cm
            ypred
             setosa versicolor virginica
  setosa         20          0         0
  versicolor      0         20         0
  virginica       0          2        18
> # Model Evaluation
```

```
> confusionMatrix(cm)
Confusion Matrix and Statistics

            ypred
             setosa versicolor virginica
  setosa         20          0         0
  versicolor      0         20         0
  virginica       0          2        18

Overall Statistics

               Accuracy : 0.9667
                 95% CI : (0.8847, 0.9959)
    No Information Rate : 0.3667
    P-Value [Acc > NIR] : < 2.2e-16

                  Kappa : 0.95

 Mcnemar's Test P-Value : NA

Statistics by Class:

                     Class: setosa Class: versicolor Class: virginica
Sensitivity                 1.0000            0.9091           1.0000
Specificity                 1.0000            1.0000           0.9524
Pos Pred Value              1.0000            1.0000           0.9000
Neg Pred Value              1.0000            0.9500           1.0000
Prevalence                  0.3333            0.3667           0.3000
Detection Rate              0.3333            0.3333           0.3000
Detection Prevalence        0.3333            0.3333           0.3333
Balanced Accuracy           1.0000            0.9545           0.9762
>
```

Filter

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa
7	4.6	3.4	1.4	0.3	setosa
8	5.0	3.4	1.5	0.2	setosa
9	4.4	2.9	1.4	0.2	setosa
10	4.9	3.1	1.5	0.1	setosa
11	5.4	3.7	1.5	0.2	setosa
12	4.8	3.4	1.6	0.2	setosa
13	4.8	3.0	1.4	0.1	setosa
14	4.3	3.0	1.1	0.1	setosa
15	5.8	4.0	1.2	0.2	setosa
16	5.7	4.4	1.5	0.4	setosa
17	5.4	3.9	1.3	0.4	setosa
18	5.1	3.5	1.4	0.3	setosa
19	5.7	3.8	1.7	0.3	setosa
20	5.1	3.8	1.5	0.3	setosa
21	5.4	3.4	1.7	0.2	setosa
22	5.1	3.7	1.5	0.4	setosa
23	4.6	3.6	1.0	0.2	setosa
24	5.1	3.3	1.7	0.5	setosa
25	4.8	3.4	1.9	0.2	setosa
26	5.0	3.0	1.6	0.2	setosa
27	5.0	3.4	1.6	0.4	setosa
28	5.2	3.5	1.5	0.2	setosa
29	5.2	3.4	1.4	0.2	setosa
30	4.7	3.2	1.6	0.2	setosa
31	4.8	3.1	1.6	0.2	setosa

PRACTICAL NO 9

Aim-Implementation and analysis of Classification algorithms - K-Nearest Neighbor

Code-

```
# Load necessary libraries
# e1071 for machine learning functions, caTools for data splitting, and class for KNN
library(e1071)
library(caTools)
library(class)

# Load the built-in Iris dataset
data(iris)  # Load the Iris dataset
head(iris)  # Display the first few rows to understand the structure of the dataset

# Split the dataset into training and testing sets
set.seed(123)  # Set a seed for reproducibility of random split
spl <- sample.split(iris, SplitRatio = 0.7)  # Split data: 70% for training, 30% for testing

# Create training and testing subsets based on the split
train_cl <- subset(iris, spl == TRUE)   # Training data
test_cl <- subset(iris, spl == FALSE)  # Testing data

# Feature Scaling
# Standardize the numerical features to have mean 0 and standard deviation 1
train_scale <- scale(train_cl[, 1:4])  # Scale the first 4 columns (features) of training data
test_scale <- scale(test_cl[, 1:4])    # Scale the first 4 columns (features) of testing data

# Apply K-Nearest Neighbors (KNN) algorithm
# k = 1 means considering the nearest neighbor for classification
classifier_knn <- knn(train = train_scale,    # Training data
                      test = test_scale,      # Testing data
                      cl = train_cl$Species,  # Class labels for training data
                      k = 1)                  # Number of neighbors to consider

# View the predictions for the testing data
print(classifier_knn)

# Create a confusion matrix to evaluate the model's performance
cm <- table(test_cl$Species, classifier_knn)  # Compare actual vs predicted classes
print("Confusion Matrix:")
print(cm)

# Interpretation of the Confusion Matrix
# Rows represent the actual classes, and columns represent the predicted classes
```

Output-

```
> # Load necessary libraries
> # e1071 for machine learning functions, caTools for data splitting, and class for KNN
> library(e1071)
> library(caTools)
> library(class)
> # Load the built-in Iris dataset
> data(iris)  # Load the Iris dataset
> head(iris)  # Display the first few rows to understand the structure of the dataset
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1          5.1         3.5          1.4         0.2  setosa
2          4.9         3.0          1.4         0.2  setosa
3          4.7         3.2          1.3         0.2  setosa
4          4.6         3.1          1.5         0.2  setosa
5          5.0         3.6          1.4         0.2  setosa
6          5.4         3.9          1.7         0.4  setosa
> # Split the dataset into training and testing sets
> set.seed(123)  # Set a seed for reproducibility of random split
> spl <- sample.split(iris, SplitRatio = 0.7)  # Split data: 70% for training, 30% for testing
> # Create training and testing subsets based on the split
> train_cl <- subset(iris, spl == TRUE)   # Training data
> test_cl <- subset(iris, spl == FALSE)  # Testing data
> # Feature Scaling
> # Standardize the numerical features to have mean 0 and standard deviation 1
> train_scale <- scale(train_cl[, 1:4])  # Scale the first 4 columns (features) of training data
> test_scale <- scale(test_cl[, 1:4])    # Scale the first 4 columns (features) of testing data
> # Apply K-Nearest Neighbors (KNN) algorithm
> # k = 1 means considering the nearest neighbor for classification
> classifier_knn <- knn(train = train_scale,    # Training data
+                       test = test_scale,      # Testing data
+                       cl = train_cl$Species,  # Class labels for training data
+                       k = 1)                  # Number of neighbors to consider
> # View the predictions for the testing data
> print(classifier_knn)
 [1] setosa     setosa     setosa     setosa     setosa     setosa     setosa     setosa     setosa     setosa
[11] setosa     setosa     setosa     setosa     setosa     setosa     setosa     setosa     setosa     setosa
[21] versicolor versicolor versicolor versicolor versicolor versicolor versicolor versicolor versicolor versicolor
[31] versicolor versicolor virginica  versicolor versicolor versicolor versicolor versicolor versicolor versicolor
[41] virginica  virginica  virginica  virginica  virginica  virginica  virginica  versicolor virginica  virginica
[51] virginica  virginica  versicolor versicolor virginica  virginica  virginica  virginica  virginica  virginica
Levels: setosa versicolor virginica
> # Create a confusion matrix to evaluate the model's performance
> cm <- table(test_cl$Species, classifier_knn)  # Compare actual vs predicted classes
> print("Confusion Matrix:")
[1] "Confusion Matrix:"
> print(cm)
            classifier_knn
             setosa versicolor virginica
  setosa         20          0         0
  versicolor      0         19         1
  virginica       0          3        17
>
> # Interpretation of the Confusion Matrix
>
```

PRACTICAL NO 10

Implementation and analysis of Classification algorithms - ID3, C4.5

Code-

```
# Load necessary libraries
install.packages("RWeka")
library(RWeka)

# Read the CSV file
data <- read.csv("weight-height.csv")

# Convert the class attribute to a factor
data$Gender <- as.factor(data$Gender)

# Build the C4.5 (J48) model
model_c45 <- J48(Gender ~ Height + Weight, data = data)

# Print the model
print(model_c45)

# Predict using the model
a <- data.frame(Height = c(190), Weight = c(85))
result <- predict(model_c45, a)
print(result)

# Visualization (J48 does not have a straightforward plot method like rpart)
# summary of the model
summary(model_c45)
```

Output-

```
Weight <= 162.676838
|   Weight <= 145.689533: Female (3592.0/91.0)
|   Weight > 145.689533
|   |   Weight <= 156.274297
|   |   |   Height <= 63.186865
|   |   |   |   Height <= 61.368045: Male (9.0)
|   |   |   |   Height > 61.368045
|   |   |   |   |   Weight <= 148.832458: Female (41.0/12.0)
|   |   |   |   |   Weight > 148.832458: Male (44.0/17.0)
|   |   |   Height > 63.186865: Female (882.0/144.0)
|   |   Weight > 156.274297
|   |   |   Height <= 65.605575: Male (204.0/64.0)
|   |   |   Height > 65.605575: Female (363.0/103.0)
Weight > 162.676838
|   Weight <= 179.181924
|   |   Weight <= 169.128653
|   |   |   Height <= 68.068922: Male (487.0/136.0)
|   |   |   Height > 68.068922: Female (107.0/34.0)
|   |   Weight > 169.128653: Male (964.0/132.0)
|   Weight > 179.181924: Male (3307.0/50.0)

Number of Leaves  :     11

Size of the tree :      21

> # Predict using the model
> a <- data.frame(Height = c(190), Weight = c(85))
> result <- predict(model_c45, a)
> print(result)
[1] Female
Levels: Female Male
> # Visualization (J48 does not have a straightforward plot method like rpart)
> # summary of the model
> summary(model_c45)

=== Summary ===

Correctly Classified Instances        9217               92.17   %
Incorrectly Classified Instances       783                7.83   %
Kappa statistic                          0.8434
Mean absolute error                      0.126
Root mean squared error                  0.251
Relative absolute error                 25.2074 %
Root relative squared error             50.207  %
Total Number of Instances            10000

=== Confusion Matrix ===

    a    b   <-- classified as
 4601  399 |    a = Female
  384 4616 |    b = Male
>
>
```

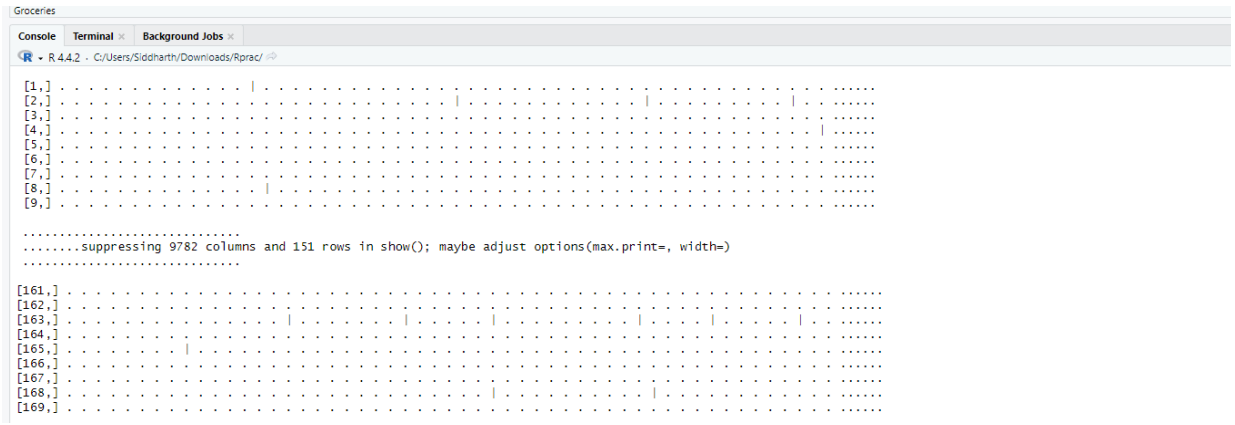
PRACTICAL NO 11

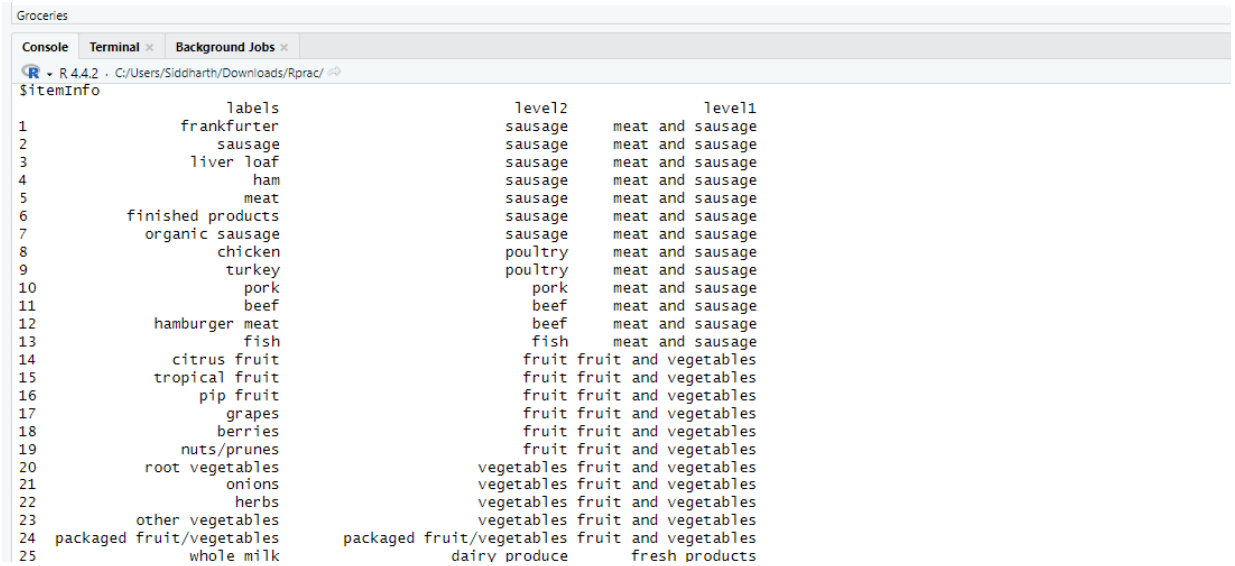
Aim- Implementation and analysis of Apriori Algorithm using Market Basket Analysis

Code-

```
# Install the arules package for association rule mining
install.packages("arules")

# Load the arules library
library(arules)

# Load the Groceries dataset from the arules package
data("Groceries")

# Display the attributes of the Groceries dataset
attributes(Groceries)

# Inspect the first 3 transactions in the Groceries dataset
inspect(head(Groceries, 3))

# Open the Groceries dataset in a spreadsheet-like viewer
View(Groceries)

# Display a summary of the Groceries dataset
summary(Groceries)

# Create an Apriori model with a minimum support of 0.001 and confidence of 0.15
model <- apriori(data = Groceries, parameter = list(support = 0.001, confidence = 0.15))

# Inspect the first 3 association rules generated by the Apriori model
inspect(head(model, 3))
```

Output-





	labels	level2	level1
1	frankfurter	sausage	meat and sausage
2	sausage	sausage	meat and sausage
3	liver loaf	sausage	meat and sausage
4	ham	sausage	meat and sausage
5	meat	sausage	meat and sausage
6	finished products	sausage	meat and sausage
7	organic sausage	sausage	meat and sausage
8	chicken	poultry	meat and sausage
9	turkey	poultry	meat and sausage
10	pork	pork	meat and sausage
11	beef	beef	meat and sausage
12	hamburger meat	beef	meat and sausage
13	fish	fish	meat and sausage
14	citrus fruit	fruit	fruit and vegetables
15	tropical fruit	fruit	fruit and vegetables
16	pip fruit	fruit	fruit and vegetables
17	grapes	fruit	fruit and vegetables
18	berries	fruit	fruit and vegetables
19	nuts/prunes	fruit	fruit and vegetables
20	root vegetables	vegetables	fruit and vegetables
21	onions	vegetables	fruit and vegetables
22	herbs	vegetables	fruit and vegetables
23	other vegetables	vegetables	fruit and vegetables
24	packaged fruit/vegetables	packaged fruit/vegetables	fruit and vegetables
25	whole milk	dairy produce	fresh products

```
Groceries
Console  Terminal  Background Jobs
R 4.4.2 · C:/Users/Siddharth/Downloads/Rprac/
 9835 rows (elements/itemsets/transactions) and
 169 columns (items) and a density of 0.02609146

most frequent items:
      whole milk other vegetables       rolls/buns             soda           yogurt          (Other)
            2513             1903             1809             1715             1372            34055

element (itemset/transaction) length distribution:
sizes
   1    2    3    4    5    6    7    8    9   10   11   12   13   14   15   16   17   18   19   20   21   22   23   24
2159 1643 1299 1005  855  645  545  438  350  246  182  117   78   77   55   46   29   14   14    9   11    4    6    1
  26   27   28   29   32
   1    1    1    3    1

   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  1.000   2.000   3.000   4.409   6.000  32.000

includes extended item information - examples:
       labels  level2          level1
1 frankfurter sausage meat and sausage
2     sausage sausage meat and sausage
3  liver loaf sausage meat and sausage
> # Create an Apriori model with a minimum support of 0.001 and confidence of 0.15
> model <- apriori(data = Groceries, parameter = list(support = 0.001, confidence = 0.15))
Apriori

Parameter specification:
 confidence minval smax arem  aval originalSupport maxtime support minlen maxlen target  ext
       0.15    0.1    1 none FALSE            TRUE       5   0.001      1     10  rules TRUE

Algorithmic control:
 filter tree heap memopt load sort verbose
    0.1 TRUE TRUE  FALSE TRUE    2    TRUE

Absolute minimum support count: 9

set item appearances ...[0 item(s)] done [0.00s].
set transactions ...[169 item(s), 9835 transaction(s)] done [0.00s].
sorting and recoding items ... [157 item(s)] done [0.00s].
creating transaction tree ... done [0.00s].
checking subsets of size 1 2 3 4 5 6 done [0.01s].
writing ... [26824 rule(s)] done [0.36s].
creating S4 object  ... done [0.01s].
> # Inspect the first 3 association rules generated by the Apriori model
```

```
Console  Terminal  Background Jobs
R 4.4.2 · C:/Users/Siddharth/Downloads/Rprac/

Parameter specification:
 confidence minval smax arem  aval originalSupport maxtime support minlen maxlen target  ext
       0.15    0.1    1 none FALSE            TRUE       5   0.001      1     10  rules TRUE

Algorithmic control:
 filter tree heap memopt load sort verbose
    0.1 TRUE TRUE  FALSE TRUE    2    TRUE

Absolute minimum support count: 9

set item appearances ...[0 item(s)] done [0.00s].
set transactions ...[169 item(s), 9835 transaction(s)] done [0.00s].
sorting and recoding items ... [157 item(s)] done [0.00s].
creating transaction tree ... done [0.00s].
checking subsets of size 1 2 3 4 5 6 done [0.01s].
writing ... [26824 rule(s)] done [0.36s].
creating S4 object  ... done [0.01s].
> # Inspect the first 3 association rules generated by the Apriori model
> inspect(head(model, 3))
    lhs    rhs                  support   confidence coverage lift count
[1] {}  => {soda}             0.1743772 0.1743772  1        1    1715
[2] {}  => {rolls/buns}       0.1839349 0.1839349  1        1    1809
[3] {}  => {other vegetables} 0.1934926 0.1934926  1        1    1903
>
>
```

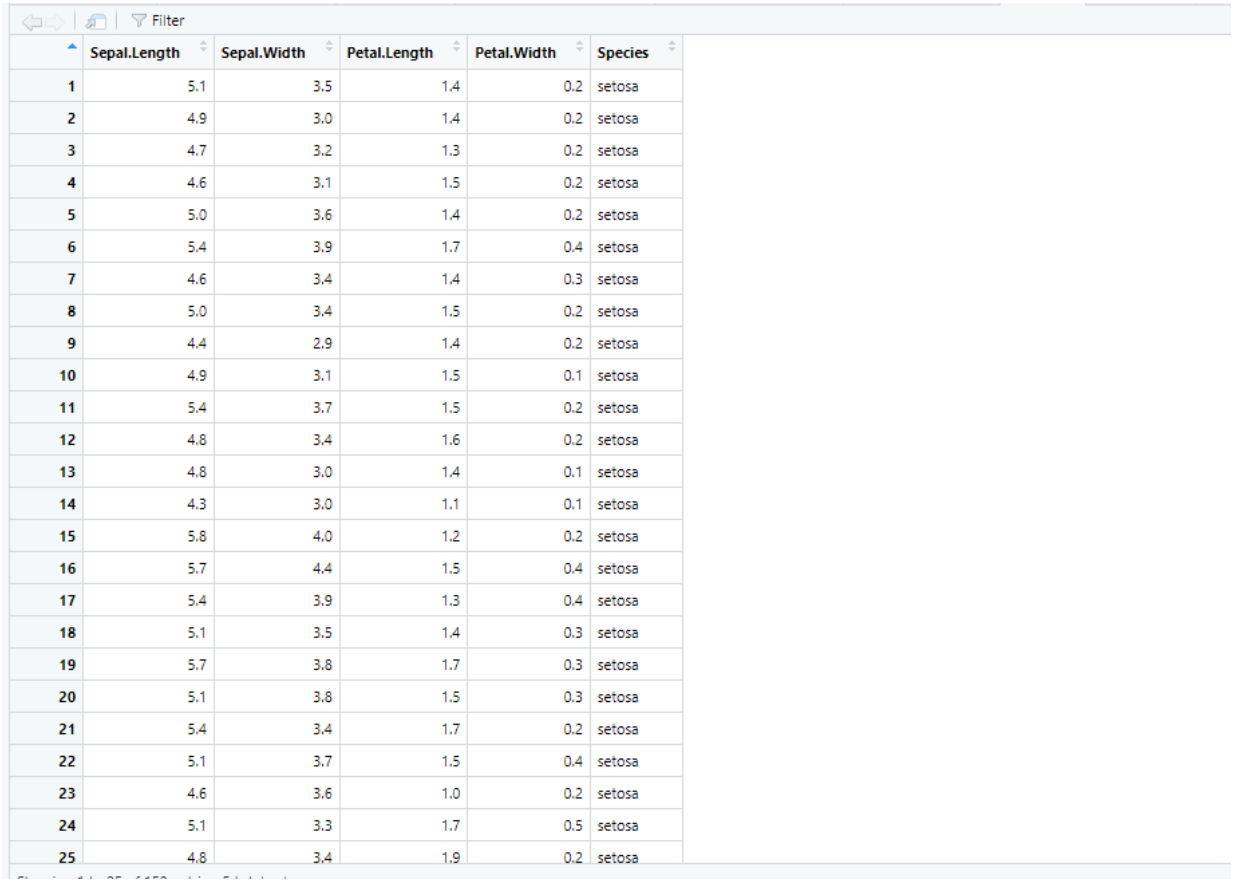
PRACTICAL NO 12

Aim-Implementation and analysis of clustering algorithms - K-Means

Code-

```
# Install necessary packages
install.packages("stats")     # For statistical operations
install.packages("dplyr")     # For data manipulation
install.packages("ggplot2")   # For data visualization
install.packages("ggfortify") # For ggplot2-based plotting of statistical results


# Load the libraries
library("stats")     # Load stats package
library("dplyr")     # Load dplyr package for data manipulation
library("ggplot2")   # Load ggplot2 package for data visualization
library("ggfortify") # Load ggfortify package for enhanced ggplot2 functionality


# View the built-in iris dataset
View(iris)


# Select the first four columns (sepal length, sepal width, petal length, petal width) from the iris dataset
mydata = select(iris, c(1, 2, 3, 4))


# Perform K-means clustering on the selected data with 2 clusters
km = kmeans(mydata, 2)


# Plot the clustering results using autoplot from ggfortify, with frames around clusters
autoplot(km, mydata, frame = TRUE)
```

Output-



Filter

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa
7	4.6	3.4	1.4	0.3	setosa
8	5.0	3.4	1.5	0.2	setosa
9	4.4	2.9	1.4	0.2	setosa
10	4.9	3.1	1.5	0.1	setosa
11	5.4	3.7	1.5	0.2	setosa
12	4.8	3.4	1.6	0.2	setosa
13	4.8	3.0	1.4	0.1	setosa
14	4.3	3.0	1.1	0.1	setosa
15	5.8	4.0	1.2	0.2	setosa
16	5.7	4.4	1.5	0.4	setosa
17	5.4	3.9	1.3	0.4	setosa
18	5.1	3.5	1.4	0.3	setosa
19	5.7	3.8	1.7	0.3	setosa
20	5.1	3.8	1.5	0.3	setosa
21	5.4	3.4	1.7	0.2	setosa
22	5.1	3.7	1.5	0.4	setosa
23	4.6	3.6	1.0	0.2	setosa
24	5.1	3.3	1.7	0.5	setosa
25	4.8	3.4	1.9	0.2	setosa

PRACTICAL NO 13

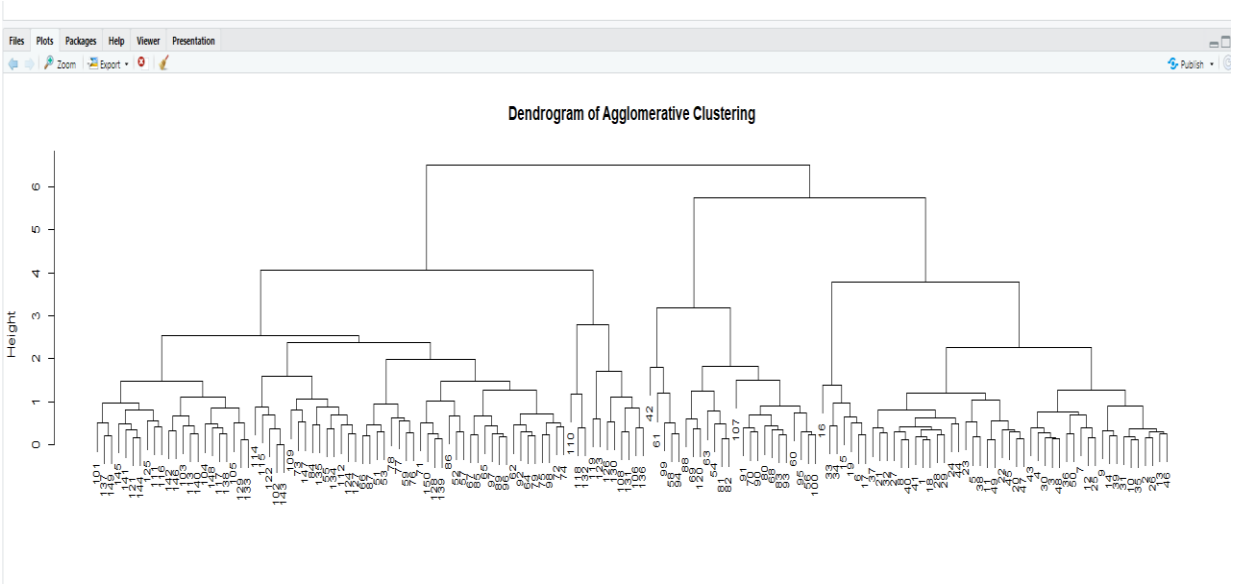
Implementation and analysis of clustering algorithms - Agglomerative

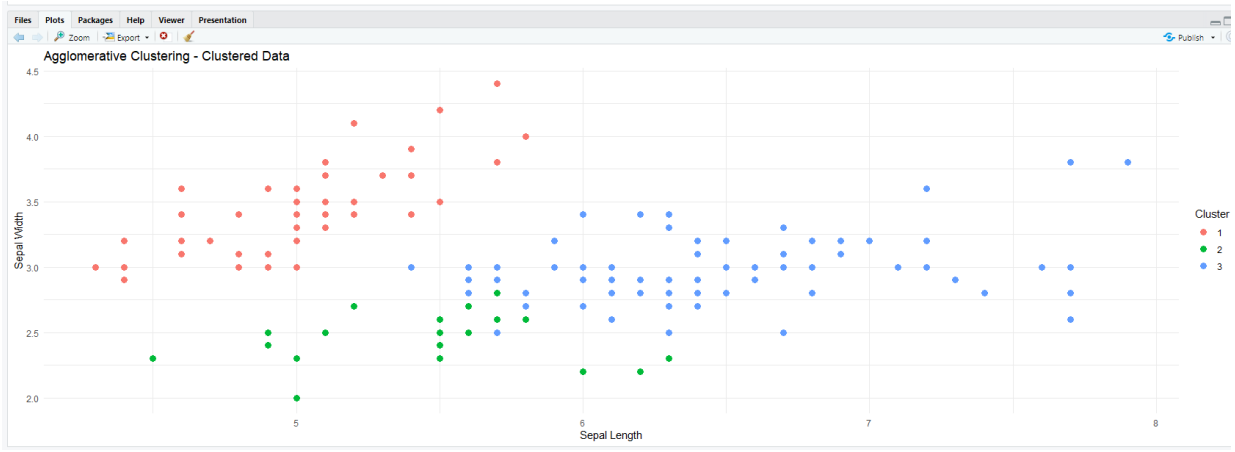
Code-

```
# Step 1: Load necessary libraries
library(datasets)   # To access the iris dataset
library(ggplot2)    # For visualization

# Step 2: Load and explore the dataset
data(iris)  # Load the iris dataset

# View the first few rows of the dataset
head(iris)

# Step 3: Preprocess the Data
# Exclude the Species column and normalize the data
iris_data <- iris[, -5]  # Remove the Species column
iris_scaled <- scale(iris_data)  # Normalize the features to have mean 0 and standard deviation 1

# Step 4: Perform Agglomerative Clustering
# Compute the distance matrix (Euclidean distance)
dist_matrix <- dist(iris_scaled, method = "euclidean")

# Perform hierarchical clustering using complete linkage
agg_clust <- hclust(dist_matrix, method = "complete")

# Step 5: Visualize the Dendrogram
# Plot the dendrogram to see how clusters are formed
plot(agg_clust, main = "Dendrogram of Agglomerative Clustering", xlab = "", sub = "", cex = 0.9)
```

```
# Step 6: Cut the Dendrogram to form 3 clusters
# Define the number of clusters as 3
clusters <- cutree(agg_clust, k = 3)

# Add the cluster labels to the iris dataset
iris$Cluster <- as.factor(clusters)

# View the first few rows of the dataset with cluster assignments
head(iris)

# Step 7: Visualize the Clusters
# Scatter plot using ggplot2 to visualize the clusters based on Sepal.Length and Sepal.Width
ggplot(iris, aes(x = Sepal.Length, y = Sepal.Width, color = Cluster)) +
  geom_point(size = 3) +
  labs(title = "Agglomerative Clustering - Clustered Data",
       x = "Sepal Length",
       y = "Sepal Width") +
  theme_minimal()
```

Output-


Files
Plots
Packages
Help
Viewer
Presentation
Zoom
Export
Publish
Dendrogram of Agglomerative Clustering
Height
0
1
2
3
4
5
6


Files
Plots
Packages
Help
Viewer
Presentation
Zoom
Export
Publish
Agglomerative Clustering - Clustered Data
Sepal Width
Sepal Length
4.5
4.0
3.5
3.0
2.5
2.0
5
6
7
8
Cluster
1
2
3